



Bao cao đatn lam-thi-ha-chau 2018 2385 3

ITS - Giao thông thông minh (Trường Đại học Công nghệ Giao thông Vận tải)



messages.pdf_cover_qr_code_label

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ



ĐỒ ÁN
TỐT NGHIỆP ĐẠI HỌC

Đề tài:

NGHIÊN CỨU KIẾN TRÚC MICROSERVICE,
TRIỂN KHAI ỨNG DỤNG MICROSERVICE SỬ
DỤNG DOCKER VÀ KUBERNETES

Sinh viên thực hiện: Lâm Thị Hà Châu
Lớp điện tử 07– K63
Giảng viên hướng dẫn: PGS.TS. Nguyễn Tài Hưng
TS. Tạ Thị Kim Huệ

Hà Nội, 7-2023

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN - ĐIỆN TỬ



ĐỒ ÁN
TỐT NGHIỆP ĐẠI HỌC

Đề tài:

**NGHIÊN CỨU KIẾN TRÚC MICROSERVICE,
TRIỂN KHAI ỨNG DỤNG MICROSERVICE SỬ
DỤNG DOCKER VÀ KUBERNETES**

Sinh viên thực hiện: Lâm Thị Hà Châu
Lớp điện tử 07– K63
Giảng viên hướng dẫn: PGS.TS. Nguyễn Tài Hưng
TS. Tạ Thị Kim Huệ
Cán bộ phản biện:

Hà Nội, 7-2023

ĐÁNH GIÁ QUYỀN ĐỒ ÁN TỐT NGHIỆP

(Dùng cho giảng viên hướng dẫn)

Tên giảng viên đánh giá:

Họ và tên Sinh viên: Lâm Thị Hà Châu. MSSV: 20182385

Tên đồ án: Nghiên cứu kiến trúc microservice, triển khai ứng dụng microservice sử dụng Docker và Kubernetes.

Chọn các mức điểm phù hợp cho sinh viên trình bày theo các tiêu chí dưới đây:

Rất kém (1); Kém (2); Đạt (3); Giỏi (4); Xuất sắc (5)

Có sự kết hợp giữa lý thuyết và thực hành (20)					
1	Nêu rõ tính cấp thiết và quan trọng của đề tài, các vấn đề và các giả thuyết (bao gồm mục đích và tính phù hợp) cũng như phạm vi ứng dụng của đồ án	1	2	3	4 5
2	Cập nhật kết quả nghiên cứu gần đây nhất (trong nước/quốc tế)	1	2	3	4 5
3	Nêu rõ và chi tiết phương pháp nghiên cứu/giải quyết vấn đề	1	2	3	4 5
4	Có kết quả mô phỏng/thực nghiệm và trình bày rõ ràng kết quả đạt được	1	2	3	4 5
Có khả năng phân tích và đánh giá kết quả (15)					
5	Kế hoạch làm việc rõ ràng bao gồm mục tiêu và phương pháp thực hiện dựa trên kết quả nghiên cứu lý thuyết một cách có hệ thống	1	2	3	4 5
6	Kết quả được trình bày một cách logic và dễ hiểu, tất cả kết quả đều được phân tích và đánh giá thỏa đáng.	1	2	3	4 5
7	Trong phần kết luận, tác giả chỉ rõ sự khác biệt (nếu có) giữa kết quả đạt được và mục tiêu ban đầu đề ra đồng thời cung cấp lập luận để đề xuất hướng giải quyết có thể thực hiện trong tương lai.	1	2	3	4 5
Kỹ năng viết quyền đồ án (10)					
8	Đồ án trình bày đúng mẫu quy định với cấu trúc các chương logic và đẹp mắt (bảng biểu, hình ảnh rõ ràng, có tiêu đề, được đánh số thứ tự và được giải thích hay đề cập đến trong đồ án, có căn lề, dấu cách sau dấu chấm, dấu phẩy v.v), có mở đầu chương và kết luận chương, có liệt kê tài liệu tham khảo và có trích dẫn đúng quy định	1	2	3	4 5
9	Kỹ năng viết xuất sắc (cấu trúc câu chuẩn, văn phong khoa học, lập luận logic và có cơ sở, từ vựng sử dụng phù hợp v.v.)	1	2	3	4 5
Thành tựu nghiên cứu khoa học (5) (chọn 1 trong 3 trường hợp)					
10a	Có bài báo khoa học được đăng hoặc chấp nhận đăng/đạt giải SVNC khoa học giải 3 cấp Viện trở lên/các giải thưởng khoa học (quốc tế/trong nước) từ giải 3 trở lên/ Có đăng ký bằng phát minh sáng chế	5			
10b	Được báo cáo tại hội đồng cấp Viện trong hội nghị sinh viên nghiên cứu khoa học nhưng không đạt giải từ giải 3 trở lên/Đạt giải khuyến khích trong các kỳ thi quốc gia và quốc tế khác về chuyên ngành như TI contest.	2			
10c	Không có thành tích về nghiên cứu khoa học	0			
Điểm tổng		/50			
Điểm tổng quy đổi về thang 10					

Nhận xét khác (về thái độ và tinh thần làm việc của sinh viên)

.....
.....
.....
.....
.....
.....

Ngày: ... / ... / 20...

Người nhận xét

(Ký và ghi rõ họ tên)

ĐÁNH GIÁ QUYỀN ĐỒ ÁN TỐT NGHIỆP

(Dùng cho cán bộ phản biện)

Giảng viên đánh giá:.....

Họ và tên sinh viên: Lâm Thị Hà Châu. MSSV: 20182385

Tên đồ án: Nghiên cứu kiến trúc microservice, triển khai ứng dụng microservice sử dụng Docker và Kubernetes.

Chọn các mức điểm phù hợp cho sinh viên trình bày theo các tiêu chí dưới đây:

Rất kém (1); Kém (2); Đạt (3); Giỏi (4); Xuất sắc (5)

Có sự kết hợp giữa lý thuyết và thực hành (20)					
1	Nêu rõ tính cấp thiết và quan trọng của đề tài, các vấn đề và các giả thuyết (bao gồm mục đích và tính phù hợp) cũng như phạm vi ứng dụng của đồ án	1	2	3	4 5
2	Cập nhật kết quả nghiên cứu gần đây nhất (trong nước/quốc tế)	1	2	3	4 5
3	Nêu rõ và chi tiết phương pháp nghiên cứu/giải quyết vấn đề	1	2	3	4 5
4	Có kết quả mô phỏng/thực nghiệm và trình bày rõ ràng kết quả đạt được	1	2	3	4 5
Có khả năng phân tích và đánh giá kết quả (15)					
5	Kế hoạch làm việc rõ ràng bao gồm mục tiêu và phương pháp thực hiện dựa trên kết quả nghiên cứu lý thuyết một cách có hệ thống	1	2	3	4 5
6	Kết quả được trình bày một cách logic và dễ hiểu, tất cả kết quả đều được phân tích và đánh giá thỏa đáng.	1	2	3	4 5
7	Trong phần kết luận, tác giả chỉ rõ sự khác biệt (nếu có) giữa kết quả đạt được và mục tiêu ban đầu đề ra đồng thời cung cấp lập luận để đề xuất hướng giải quyết có thể thực hiện trong tương lai.	1	2	3	4 5
Kỹ năng viết quyền đồ án (10)					
8	Đồ án trình bày đúng mẫu quy định với cấu trúc các chương logic và đẹp mắt (bảng biểu, hình ảnh rõ ràng, có tiêu đề, được đánh số thứ tự và được giải thích hay đề cập đến trong đồ án, có căn lề, dấu cách sau dấu chấm, dấu phẩy v.v), có mở đầu chương và kết luận chương, có liệt kê tài liệu tham khảo và có trích dẫn đúng quy định	1	2	3	4 5
9	Kỹ năng viết xuất sắc (cấu trúc câu chuẩn, văn phong khoa học, lập luận logic và có cơ sở, từ vựng sử dụng phù hợp v.v.)	1	2	3	4 5
Thành tựu nghiên cứu khoa học (5) (chọn 1 trong 3 trường hợp)					
10a	Có bài báo khoa học được đăng hoặc chấp nhận đăng/đạt giải SVNC khoa học giải 3 cấp Viện trở lên/các giải thưởng khoa học (quốc tế/trong nước) từ giải 3 trở lên/ Có đăng ký bằng phát minh sáng chế	5			
10b	Được báo cáo tại hội đồng cấp Viện trong hội nghị sinh viên nghiên cứu khoa học nhưng không đạt giải từ giải 3 trở lên/Đạt giải khuyến khích trong các kỳ thi quốc gia và quốc tế khác về chuyên ngành như TI contest.	2			
10c	Không có thành tích về nghiên cứu khoa học	0			
Điểm tổng					/50
Điểm tổng quy đổi về thang 10					

Nhận xét khác của cán bộ phản biện

.....
.....
.....
.....
.....
.....

Ngày: ... / ... / 20...

Người nhận xét

(Ký và ghi rõ họ tên)

LỜI NÓI ĐẦU

Trong thời đại công nghệ hiện đại, việc xây dựng các ứng dụng phần mềm có tính mở rộng cao, linh hoạt và dễ dàng bảo trì là một trong những yêu cầu cần thiết. Và để đáp ứng yêu cầu đó, kiến trúc microservice đã trở thành một trong những phương pháp được sử dụng phổ biến trong thiết kế và triển khai ứng dụng phần mềm.

Nghiên cứu về kiến trúc microservice là một chủ đề đang thu hút sự quan tâm của rất nhiều nhà phát triển và các chuyên gia trong lĩnh vực công nghệ thông tin. Kiến trúc microservice cho phép các ứng dụng phần mềm được chia thành các thành phần nhỏ hơn, độc lập với nhau và có khả năng hoạt động độc lập. Điều này giúp cho việc phát triển, triển khai và bảo trì ứng dụng trở nên dễ dàng hơn, đồng thời cũng giúp cải thiện hiệu suất, tính sẵn sàng và khả năng mở rộng của ứng dụng.

Đồ án này sẽ nghiên cứu về kiến trúc microservice, bao gồm định nghĩa, ưu điểm, hạn chế, cách triển khai và các công cụ hỗ trợ cho việc triển khai kiến trúc này. Từ đó, đồ án sẽ đưa ra các giải pháp và lời khuyên cho việc triển khai kiến trúc microservice trong các ứng dụng phần mềm.

Trong suốt quá trình học tập và thực hiện đề tài này, em đã nhận được sự giúp đỡ, chỉ dạy tận tình của PGS. TS. Nguyễn Tài Hưng và TS. Tạ Thị Kim Huệ - giảng viên Trường Điện- Điện tử, Đại học Bách Khoa Hà Nội. Sự chỉ dạy tận tình của thầy và cô đã giúp đỡ em rất nhiều trong quá trình thực hiện đề tài.

Trong quá trình thực hiện đề tài em vẫn còn nhiều sai sót do đó em rất mong nhận được những nhận xét, đánh giá của thầy/cô để đề tài có thể phát triển và hoàn thiện hơn.

Em xin chân thành cảm ơn!

LỜI CAM ĐOAN

Tôi là Lâm Thị Hà Châu, mã số sinh viên 20182385, sinh viên lớp Điện tử 07, khóa 63. Người hướng dẫn là PGS.TS. Nguyễn Tài Hưng và TS. Tạ Thị Kim Huệ. Chúng tôi xin cam đoan toàn bộ nội dung được trình bày trong đồ án '*Nghiên cứu kiến trúc microservice, triển khai ứng dụng microservice sử dụng Docker và Kubernetes*' là kết quả quá trình tìm hiểu và nghiên cứu của chúng tôi. Các dữ liệu được nêu trong đồ án là hoàn toàn trung thực, phản ánh đúng kết quả thực tế. Mọi thông tin trích dẫn đều tuân thủ các quy định về sở hữu trí tuệ; các tài liệu tham khảo được liệt kê rõ ràng. Chúng tôi xin chịu hoàn toàn trách nhiệm với những nội dung được viết trong đồ án này.

Hà Nội, ngày tháng năm 2022

Người cam đoan

Lâm Thị Hà Châu

MỤC LỤC

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT	i
DANH MỤC HÌNH VẼ.....	ii
DANH MỤC BẢNG BIỂU.....	iv
TÓM TẮT ĐỒ ÁN.....	v
ABSTRACT	vi
PHẦN MỞ ĐẦU	1
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT	3
1.1 Tổng quan về kiến trúc hệ thống	3
1.2 Kiến trúc nguyên khối (Monolithic).....	4
1.3 Kiến trúc hướng dịch vụ (SOA)	5
1.4 Kiến trúc dịch vụ siêu nhỏ (Microservice)	6
1.4.1 Khái niệm.....	7
1.4.2 Các thành phần cơ bản của microservice.....	8
1.4.3 Các mẫu kiến trúc microservice.....	10
1.4.4 Ưu nhược điểm của kiến trúc microservice	17
1.4.5 Ứng dụng microservice dựa trên container.....	19
CHƯƠNG 2. CÁC GIẢI PHÁP CÔNG NGHỆ.....	21
2.1 Docker	21
2.1.1 Khái niệm container	21
2.1.2 Tổng quan về Docker.....	22
2.1.3 Kiến trúc và thành phần của Docker	23
2.1.4 So sánh Docker Container với máy ảo (Virtual Machine).....	25
2.1.5 Quy trình thực thi của một hệ thống sử dụng Docker.....	26
2.1.6 Container Orchestration	27
2.2 Kubernetes.....	30
2.2.1 Tổng quan về Kubernetes	30
2.2.2 Kiến trúc của Kubernetes.....	31
2.2.3 Kubernetes Object.....	33
2.2.4 Kubernetes Addon.....	35

2.2.5 Minikube	35
CHƯƠNG 3. TRIỂN KHAI ỨNG DỤNG MICROSERVICE	37
<i>3.1 Nghiên cứu kiến trúc ứng dụng microservice được triển khai.....</i>	<i>37</i>
3.1.1 Kiến trúc	40
3.1.2 Front-ent.....	40
3.1.3 Order service	41
3.1.4 Payment service.....	42
3.1.5 User service	42
3.1.6 Catalogue service	43
3.1.7 Các service khác	44
<i>3.2 Container hóa các service trong ứng dụng</i>	<i>45</i>
<i>3.3 Triển khai ứng dụng trên Kubernetes</i>	<i>54</i>
3.3.1 Thiết lập cụm Kubernetes cơ bản với minikube.....	54
3.3.2 Triển khai ứng dụng trên Kubernetes	57
<i>3.4 Đánh giá</i>	<i>61</i>
KẾT LUẬN.....	63
<i>Kết luận chung</i>	<i>63</i>
<i>Hướng phát triển.....</i>	<i>63</i>
TÀI LIỆU THAM KHẢO	65
PHỤ LỤC.....	66

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

STT	Chữ viết tắt	Chữ viết đầy đủ
1	VM	Virtual Machine
2	SOA	Service-Oriented Architecture
3	API	Application Programming Interface
4	HTTP	Hypertext Transfer Protocol
5	REST	Representational State Transfer
6	CDN	Content Delivery Network
7	IoT	Internet of Things
8	CRUD	Create, Read, Update, Delete
9	CQRS	Command and Query Responsibility Segregation
10	2PC	Two-Phase Commit
11	OS	Operating System

DANH MỤC HÌNH VẼ

Hình 1.1 Mô hình kiến trúc Monolithic	5
Hình 1.2 Mô hình kiến trúc Microservice	7
Hình 1.3 Kiến trúc ứng dụng microservice điển hình	8
Hình 1.4 Kỹ thuật Service Discovery	10
Hình 1.5 Mô tả trạng thái trong mô hình Circuit Breaker	12
Hình 1.6 Kiến trúc ứng dụng sử dụng API gateway	13
Hình 1.7 Mô hình CQRS	14
Hình 2.1 Container hóa ứng dụng với Docker	22
Hình 2.2 Kiến trúc của Docker	24
Hình 2.3 Các container dùng chung nhân OS	25
Hình 2.4 So sánh Docker container và Virtual Machines	26
Hình 2.5 Quy trình thực thi hệ thống chạy container.....	27
Hình 2.6 Kiến trúc của Kubernetes	31
Hình 2.7 Minikube architecture	36
Hình 3.1 Kiến trúc ứng dụng	40
Hình 3.2 Các API của Orders service	42
Hình 3.3 Các API của Payment service	42
Hình 3.4 Các API của User service	43
Hình 3.5 Các API của Catalogue service	43
Hình 3.6 Thư mục chứa source code và Dockerfile của từng service	45
Hình 3.7 Log chạy front-end service container.....	47
Hình 3.8 Log chạy Edge-router service container	47
Hình 3.9 Log chạy Orders service container.....	48
Hình 3.10 Log chạy Payment service container	48
Hình 3.11 Log chạy User service container	49
Hình 3.12 Log chạy Carts service container	49
Hình 3.13 Log chạy Catalogue service container	50

Hình 3.14 Log chạy Shipping service container	50
Hình 3.15 Log chạy Queue-master service container	50
Hình 3.16 Giao diện chính của ứng dụng.....	51
Hình 3.17 Đăng ký tài khoản mua hàng.....	51
Hình 3.18 Giao diện danh mục sản phẩm.....	52
Hình 3.19 Giao diện thêm địa chỉ giao hàng	52
Hình 3.20 Giao diện thêm thông tin thẻ thanh toán	53
Hình 3.21 Giao diện order sản phẩm.....	53
Hình 3.22 Giao diện danh sách các đơn hàng đã đặt.....	54
Hình 3.23 Cài đặt thành công minikube	56
Hình 3.24 Kết quả lệnh thiết lập cluster	56
Hình 3.25 Nodes, deployment và pod hệ thống	56
Hình 3.26 Thư mục chứa tệp cấu hình manifest.....	57
Hình 3.27 Kết quả triển khai ứng dụng sử dụng tệp manifest.....	57
Hình 3.28 Các pod của ứng dụng	58
Hình 3.29 Các deployment của ứng dụng	58
Hình 3.30 Các service chạy trên cluster	58
Hình 3.31 Kết quả chạy lệnh thêm Kubernetes Dashboard	59
Hình 3.32 Giao diện Kubernetes Dashboard.....	59
Hình 3.33 URL của các service	60
Hình 3.34 URL của Dashboard service	60
Hình 3.35 URL Front-end service	60
Hình 3.36 Giao diện chính của ứng dụng chạy trên cụm Kubernetes.....	61

DANH MỤC BẢNG BIỂU

TÓM TẮT ĐỒ ÁN

Đồ án này em sẽ tập trung vào nghiên cứu về kiến trúc microservice và triển khai ứng dụng microservice sử dụng Docker và Kubernetes. Đầu tiên, đồ án trình bày định nghĩa, ưu điểm và hạn chế của kiến trúc microservice. Sau đó, đồ án sẽ đi sâu vào các công nghệ và công cụ hỗ trợ cho triển khai kiến trúc này, bao gồm Docker và Kubernetes.

Đồ án đưa ra một bài toán về triển khai một ứng dụng web sử dụng kiến trúc microservice bằng Docker và Kubernetes. Mô tả các bước cần thiết để triển khai ứng dụng web này, bao gồm việc xây dựng các container Docker, triển khai các container đó sử dụng Kubernetes và kiểm tra hoạt động của ứng dụng.

Kết quả của đồ án là việc triển khai thành công ứng dụng web sử dụng kiến trúc microservice bằng Docker và Kubernetes. Đồ án cũng đưa ra các đánh giá và so sánh về hiệu suất của kiến trúc microservice so với kiến trúc monolith truyền thống, cũng như lợi ích và nhược điểm của việc sử dụng Docker và Kubernetes cho việc triển khai kiến trúc microservice.

Đề tài cung cấp một cái nhìn tổng quan về kiến trúc hệ thống, chi tiết hơn về kiến trúc microservice và cách mà ứng dụng dựa trên kiến trúc này được triển khai với hai công nghệ nổi bật và phát triển mạnh mẽ trong thời gian gần đây là Docker và Kubernetes.

ABSTRACT

This thesis will focus on researching microservice architecture and implementing microservice applications using Docker and Kubernetes. Firstly, the thesis presents the definition, advantages, and limitations of microservice architecture. Then, the thesis will delve into the technologies and tools that support the implementation of this architecture, including Docker and Kubernetes.

The thesis presents a problem of deploying a web application using microservice architecture with Docker and Kubernetes. It describes the necessary steps to deploy this web application, including building Docker containers, deploying those containers using Kubernetes, and testing the application's functionality.

The result of the thesis is the successful deployment of a web application using microservice architecture with Docker and Kubernetes. The thesis also provides an evaluation and comparison of the performance of microservice architecture compared to traditional monolithic architecture, as well as the benefits and drawbacks of using Docker and Kubernetes for microservice architecture deployment.

The topic provides an overview of system architecture, more details about microservice architecture, and how applications based on this architecture are deployed with two prominent and rapidly developing technologies, Docker and Kubernetes.

PHẦN MỞ ĐẦU

1. Đặt vấn đề

Trong thời đại công nghệ 4.0, việc phát triển các hệ thống phần mềm đang trở nên phức tạp hơn bao giờ hết. Để đáp ứng nhu cầu ngày càng tăng, các nhà phát triển phần mềm cần xây dựng các hệ thống có tính linh hoạt, có khả năng mở rộng và dễ dàng bảo trì. Trong bối cảnh đó, kiến trúc microservice đã trở thành một trong những giải pháp phổ biến nhất để giải quyết vấn đề này. Bên cạnh đó là sự phát triển của các công nghệ mang tính đột phá như công nghệ ảo hóa và container. Nhận thấy được lợi ích tuyệt vời mà các giải pháp công nghệ này đem lại cùng với khả năng kết hợp của chúng trong thực tế, với việc chúng được áp dụng trong rất nhiều hệ thống phần mềm lớn trên thế giới hiện nay. Với mong muốn hiểu rõ hơn về các giải pháp công nghệ hiện đại này và cách chúng được thực hiện trong bối cảnh thực tế, em xin trình bày đồ án với đề tài “Nghiên cứu kiến trúc microservice, triển khai ứng dụng microservice sử dụng Docker và Kubernetes”.

2. Mục đích của đề tài

Đề tài này được thực hiện nhằm cung cấp một cái nhìn tổng quan về kiến trúc hệ thống, các công nghệ ảo hóa và cách chúng được áp dụng và triển khai thông qua nghiên cứu, thử nghiệm và đánh giá dựa trên phần mềm thực tế.

3. Giới hạn đề tài

Phạm vi của đề tài sẽ tập trung vào tìm hiểu, nghiên cứu về kiến trúc microservice, các công nghệ như Docker và Kubernetes và áp dụng các công nghệ đó trong việc triển khai một ứng dụng dựa trên kiến trúc microservice. Do giới hạn về mặt thời gian và khối lượng kiến thức lớn cần phải chuẩn bị và nghiên cứu, nên của em kết quả đồ án chỉ đáp ứng được một phần trong tổng thể giải pháp lớn và còn có nhiều hạn chế.

4. Cấu trúc đề tài

Trong cấu trúc của đề tài, ngoài phần mở đầu và kết luận, đề tài được chia thành ba chương với các nội dung chính sau đây:

- Chương 1: Cơ sở lý thuyết

Nội dung chương 1 sẽ đi vào tổng quan về kiến trúc hệ thống và đi sâu vào nghiên cứu, tìm hiểu kiến trúc microservice.

- Chương 2: Các giải pháp công nghệ

Nội dung của chương 2 tập trung làm rõ về khái niệm container, những lợi ích khi sử dụng container và các công nghệ dựa trên container như Docker hay Kubernetes.

- Chương 3: Triển khai ứng dụng microservice

Trong chương 3, một ứng dụng microservice mã nguồn mở đơn giản được triển khai sử dụng Docker và Kubernetes. Nội dung chương sẽ giải thích về kiến trúc ứng dụng được mô phỏng, các công nghệ được sử dụng để phát triển nên ứng dụng, quy trình container hóa ứng dụng với Docker và triển khai các container chứa các thành phần của ứng dụng microservice đó trên nền tảng điều phối vùng chứa là Kubernetes.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

Trong thời đại kinh tế số hiện nay, các ứng dụng phần mềm ngày càng trở nên quan trọng và đóng vai trò không nhỏ trong việc đưa ra các giải pháp kinh doanh và cải thiện trải nghiệm người dùng. Tuy nhiên, với sự phát triển nhanh chóng của các ứng dụng và quy mô ngày càng lớn, việc thiết kế và triển khai hệ thống phần mềm đòi hỏi sự chú ý đặc biệt đến kiến trúc hệ thống. Kiến trúc hệ thống là một khái niệm rất phức tạp, bao gồm nhiều thành phần khác nhau như máy chủ, mạng, phần mềm và cơ sở dữ liệu. Một kiến trúc hệ thống tốt sẽ đảm bảo rằng các thành phần này hoạt động hiệu quả và tương tác với nhau một cách chính xác để đáp ứng nhu cầu của người dùng.

Trong các kiến trúc hệ thống, microservice đang trở thành một lựa chọn phổ biến cho các hệ thống phân tán, đặc biệt là trong các hệ thống có quy mô lớn hoặc cần độ linh hoạt trong việc triển khai và mở rộng hệ thống. Kiến trúc microservice giúp các nhà phát triển có thể xây dựng các dịch vụ độc lập nhau và có thể được triển khai và quản lý độc lập nhau, giúp tăng tính linh hoạt và khả năng mở rộng hệ thống.

Nội dung chương này sẽ đi vào tìm hiểu chi tiết hơn về kiến trúc hệ thống, về kiến trúc microservice và cách mà việc sử dụng microservice có thể giúp cải thiện khả năng linh hoạt và mở rộng của hệ thống phần mềm. Đầu tiên sẽ đi vào tổng quan về kiến trúc hệ thống và các loại kiến trúc hệ thống, sau đó tìm hiểu về các lợi ích và thách thức của việc sử dụng kiến trúc microservice. Cuối cùng sẽ xem xét các quy trình và công cụ để triển khai và quản lý các dịch vụ microservice.

1.1 Tổng quan về kiến trúc hệ thống

Quá trình tạo và xác định cấu trúc và hành vi chung của một hệ thống phức tạp được gọi là kiến trúc hệ thống. Nó đòi hỏi phải phân tách hệ thống thành các mô-đun và thành phần riêng biệt và tìm ra cách các mô-đun này sẽ tương tác với nhau như thế nào để tạo ra chức năng hệ thống mong muốn. Ngoài khả năng mở rộng, bảo trì và bảo mật, một kiến trúc được thiết kế tốt có thể đảm bảo rằng hệ thống đáp ứng cả yêu cầu chức năng và phi chức năng của người dùng.

Các hệ thống phần mềm, hệ thống phần cứng và thậm chí cả các hệ thống phức tạp có cả các thành phần phần cứng và phần mềm đều có thể hưởng lợi từ việc sử dụng kiến

trúc hệ thống. Nó đòi hỏi phải tính đến mọi khía cạnh của hệ thống, bao gồm đầu vào, đầu ra, giao diện và cách các bộ phận tương tác với nhau.

Một kiến trúc hệ thống phần mềm thường được thiết kế qua nhiều bước. Giai đoạn đầu tiên là xác định các yêu cầu chức năng và phi chức năng của hệ thống. Hiểu các yêu cầu của người dùng cũng như bất kỳ hạn chế hoặc giới hạn kỹ thuật nào là cần thiết cho việc này. Nhóm thiết kế kiến trúc hệ thống có thể bắt đầu xác định các thành phần hệ thống khác nhau và tương tác của chúng sau khi các yêu cầu đã được xác định.

Việc áp dụng các mẫu thiết kế và kiểu kiến trúc là một thành phần quan trọng của kiến trúc hệ thống phần mềm. Đây là các bản sửa lỗi đã được thử và đúng cho các sự cố thiết kế phần mềm điển hình có thể giúp đảm bảo hệ thống phần mềm được thiết kế tốt và đáp ứng các thông số kỹ thuật cần thiết.

Có nhiều loại kiến trúc hệ thống được sử dụng trong phát triển phần mềm có thể kể đến như kiến trúc dịch vụ siêu nhỏ (Microservice), kiến trúc hướng dịch vụ (SOA), kiến trúc nguyên khối (Monolithic), kiến trúc máy chủ - khách (Client-Server) và nhiều loại kiến trúc khác. Nội dung chương này sẽ trình bày về một số kiến trúc hệ thống phổ biến và được sử dụng nhiều trong thực tế.

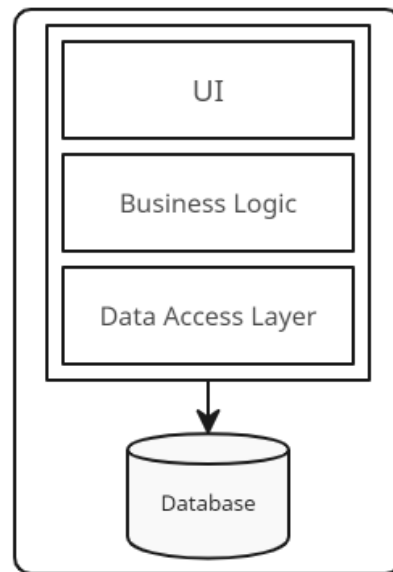
1.2 Kiến trúc nguyên khối (Monolithic)

Kiến trúc nguyên khối (monolithic architecture), chính tên gọi cũng diễn tả được phần nào đặc điểm của nó. Đây là một dạng kiến trúc mà tất cả các chức năng của hệ thống được tích hợp trong chỉ một ứng dụng duy nhất. Toàn bộ hệ thống được phát triển và triển khai như một khối đơn lẻ. Các thành phần của ứng dụng được phát triển dựa trên kiến trúc này được liên kết chặt chẽ với nhau và có tính phụ thuộc cao. Mô hình kiến trúc được minh họa ở Hình 1.1.

Với kiến trúc đơn giản, Monolithic là một mô hình dễ dàng để phát triển và gỡ lỗi. Kiến trúc nguyên khối giúp đơn giản hóa việc bảo trì và nâng cấp ứng dụng bằng cách chỉ cần triển khai và cập nhật một tệp duy nhất vì hầu hết project đều nằm trên tệp này. Nó thường được sử dụng trong xây dựng các chương trình phần mềm truyền thống, các ứng dụng desktop, web đơn giản, các hệ thống có quy mô nhỏ và ít yêu cầu mở rộng.

Tuy nhiên, ngoài những ưu điểm kể trên kiến trúc nguyên khối vẫn tồn tại một số nhược điểm. Đầu tiên, kiến trúc này khó để mở rộng khi ứng dụng phát triển lớn hơn và phải xử lý nhiều yêu cầu hơn cùng một lúc. Việc mở rộng sẽ phải thực hiện bằng cách

thêm các chức năng vào khối hiện có hoặc tạo ra các khối mới. Mã nguồn ứng dụng khó để tái sử dụng cho các ứng dụng khác, các nhà phát triển sẽ phải tạo ra các tính năng mới hoặc các ứng dụng mới một cách độc lập gây mất thời gian và làm giảm tính hiệu quả. Việc triển khai liên tục (Continuous Deployment) có thể trở nên khó khăn với kiến trúc nguyên khối do việc triển khai sẽ ảnh hưởng đến toàn bộ ứng dụng. Các thành phần của ứng dụng liên kết chặt chẽ và có tính phụ thuộc cao nên khi một thành phần gặp vấn đề hoặc cần chỉnh sửa thì sẽ gây ảnh hưởng đến cả hệ thống.



Hình 1.1 Mô hình kiến trúc Monolithic

1.3 Kiến trúc hướng dịch vụ (SOA)

Kiến trúc hướng dịch vụ (Service-Oriented Architecture - SOA) là một phương pháp thiết kế phần mềm trong đó các thành phần ứng dụng cung cấp dịch vụ cho các thành phần khác thông qua một giao thức truyền thông trên mạng. Nguyên tắc của SOA độc lập với nhà cung cấp và các công nghệ khác. Trong kiến trúc này, các dịch vụ tương tác với nhau bằng cách truyền dữ liệu hoặc phối hợp một hoạt động giữa hai hoặc nhiều dịch vụ khác nhau. Mỗi dịch vụ đóng vai trò là một phần của hệ thống toàn cục và có thể được sử dụng để cung cấp các tính năng khác nhau của ứng dụng. Các dịch vụ này có thể được sử dụng lại trong nhiều ứng dụng khác nhau. Có sáu điểm cốt lõi của SOA, bao gồm giá trị kinh doanh, mục tiêu chiến lược, khả năng tương tác bên trong, dịch vụ chia sẻ, tính linh hoạt và cải tiến tiến hóa. SOA thường được sử dụng để triển khai các dịch vụ web, giúp cho các khối xây dựng chức năng trở nên có thể truy cập được qua các giao thức tiêu chuẩn trên internet. SOAP, viết tắt của Simple Object Access Protocol,

là một ví dụ về tiêu chuẩn dịch vụ web. Các tùy chọn khác để triển khai SOA bao gồm Jini, COBRA hoặc REST (Representational State Transfer). Tuy nhiên, các kiến trúc có thể hoạt động độc lập với các công nghệ cụ thể, điều đó có nghĩa là chúng có thể được triển khai theo nhiều cách khác nhau, bao gồm truyền thông, chẳng hạn như ActiveMQ, Apache Thrift và SORCER.

SOA mang lại nhiều lợi ích cho các doanh nghiệp, bao gồm khả năng tạo mã có thể tái sử dụng, thúc đẩy tương tác, cho phép mở rộng và giảm chi phí trong khi vẫn duy trì mức đầu ra mong muốn. Trong SOA, các dịch vụ được phân tách một cách độc lập và có thể tái sử dụng được. Điều đó giúp giảm thời gian phát triển và tăng tính linh động trong việc thay đổi chức năng của hệ thống. Đặc điểm trên còn đem lại một ưu thế nữa đó là một dịch vụ có thể được thay đổi và nâng cấp một cách dễ dàng mà không làm ảnh hưởng đến các thành phần khác của ứng dụng. SOA có thể hoạt động kết hợp hoặc không kết hợp với cloud computing, tuy nhiên, một trong những thách thức mà các doanh nghiệp phải đối mặt khi kết hợp cloud computing và SOA là tích hợp dữ liệu và hệ thống hiện có vào giải pháp cloud.

1.4 Kiến trúc dịch vụ siêu nhỏ (Microservice)

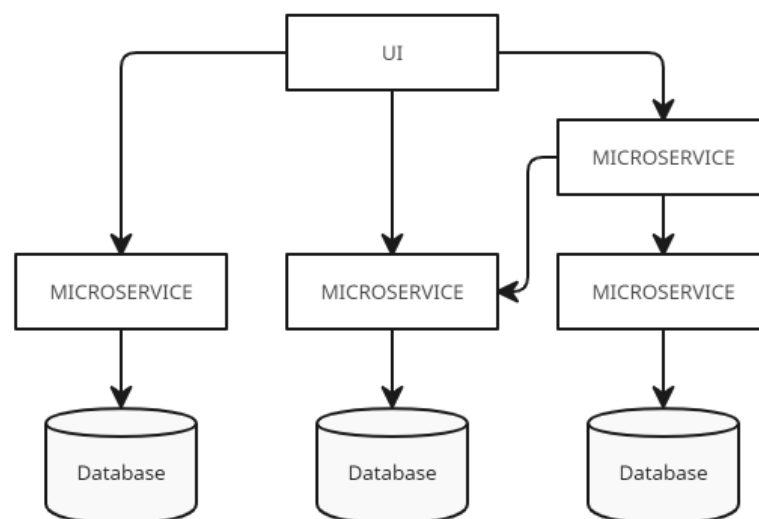
Microservice dịch ra có nghĩa là dịch vụ siêu nhỏ. Kiến trúc Microservice đề cập đến vấn đề phân tách hệ thống thành các service nhỏ, mỗi service thực hiện các chức năng, nhiệm vụ khác nhau và có thể triển khai một cách độc lập. Các service được kết nối bằng các liên kết lỏng lẻo (loosely coupled link) vừa đảm bảo các service có thể giao tiếp với nhau nhưng vẫn giữ tính độc lập giữa chúng.

So với Monolithic và SOA, Microservice nổi trội hơn ở tính độc lập, các thành phần liên kết lỏng lẻo, nhỏ gọn và có khả năng tự quản lý cao. Trong kiến trúc Microservice, các dịch vụ được phát triển và triển khai một cách độc lập. Trong khi đó, trong kiến trúc Monolithic và SOA, các dịch vụ phụ thuộc vào nhau hơn, vì vậy tính độc lập của các dịch vụ không cao như Microservice. Mỗi Microservice thực hiện một chức năng cụ thể, giúp giảm thiểu sự phức tạp của ứng dụng. Còn trong Monolithic và SOA, các dịch vụ thường lớn hơn và thực hiện nhiều chức năng khác nhau. Microservice sử dụng các cơ chế truyền thông đơn giản, như HTTP (Hypertext Transfer Protocol) hoặc RESTful API, trong khi Monolithic và SOA sử dụng các cơ chế truyền thông phức tạp hơn, như SOAP (Simple Object Access Protocol) hoặc JMS (Java Message Service). So với việc các ứng

dụng được xây dựng dựa trên Monolithic và SOA yêu cầu một trung tâm quản lý dịch vụ thì Microservice có tính tự quản lý cao hơn, mỗi Microservice đảm nhận việc quản lý và triển khai riêng của mình. Ngoài ra, Microservice cho phép các dịch vụ được triển khai trên các nền tảng độc lập, giúp tăng khả năng di chuyển của ứng dụng giữa các môi trường khác nhau. Microservices và SOA tương đồng ở một số khía cạnh, nhưng điểm khác biệt nằm ở chính chức năng của chúng.

1.4.1 Khái niệm

Microservice là một kiến trúc phần mềm dựa trên các dịch vụ nhỏ, độc lập và tuân theo nguyên tắc phân tách trách nhiệm (Separation of Concerns). Trong kiến trúc Microservice, mỗi dịch vụ thực hiện một chức năng cụ thể và có thể được triển khai và quản lý độc lập. Các dịch vụ này giao tiếp với nhau thông qua các giao thức chuẩn như HTTP hoặc sử dụng RESTful API để thực hiện các chức năng phức tạp hơn. Các dịch vụ trong kiến trúc Microservice có thể được phát triển bằng các ngôn ngữ lập trình khác nhau, sử dụng các công nghệ khác nhau và được triển khai trên các nền tảng độc lập. Một trong những ưu điểm của kiến trúc Microservice là tính độc lập của các dịch vụ, giúp tăng tính linh hoạt và giảm thiểu sự phụ thuộc giữa các dịch vụ. Điều này giúp phát triển và triển khai các tính năng mới một cách nhanh chóng hơn, làm tăng khả năng mở rộng của hệ thống. Mô hình kiến trúc hệ thống Microservice được mô tả dựa trên hình dưới đây.

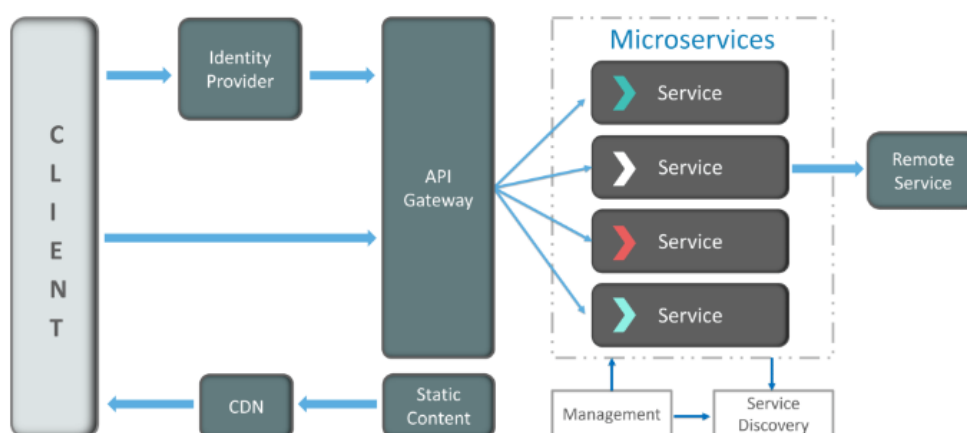


Hình 1.2 Mô hình kiến trúc Microservice

Microservice đã thực sự nổi lên một cách mạnh mẽ trong những năm gần đây nhờ vào những lợi ích nó mang lại. Trên thực tế, nó đã được ứng dụng vào rất nhiều hệ thống từ các ứng dụng web, ứng dụng di động, IoT (Internet of Things) đến các hệ thống tài chính và ngân hàng. Kiến trúc microservice được sử dụng để phát triển các ứng dụng web có lượng truy cập lớn, một vài trang web phổ biến như Netflix, Amazon, Uber,... Các ứng dụng di động, IoT đòi hỏi về băng thông rộng và độ trễ thấp cũng được xây dựng dựa trên microservice, nhờ khả năng tối ưu hóa truyền thông giữa các thành phần hệ thống giúp giảm thiểu độ trễ và tăng tốc độ xử lý. Đối với các hệ thống ngân hàng và tài chính đòi hỏi tính bảo mật và độ tin cậy cao, Microservice cho phép phân tách các chức năng kinh doanh thành các dịch vụ độc lập, giúp tăng tính an toàn và giảm thiểu tác động của các vấn đề bảo mật đến toàn bộ hệ thống. Ngoài ra, nó cũng được sử dụng cho các ứng dụng trí tuệ nhân tạo và machine learning đòi hỏi xử lý dữ liệu lớn và đòi hỏi tính mở rộng cao. Một số ứng dụng trí tuệ nhân tạo và machine learning sử dụng kiến trúc microservice có thể kể đến như Amazon SageMaker hay TensorFlow. Tuy nhiên, kiến trúc Microservice cũng có một số thách thức, bao gồm việc quản lý các dịch vụ độc lập và đảm bảo tính nhất quán của hệ thống khi các dịch vụ phát triển và triển khai độc lập với nhau.

1.4.2 Các thành phần cơ bản của microservice

Một kiến trúc Microservice điển hình bao gồm nhiều thành phần cơ bản như sau: Client, Identity Provider, API Gateway, Messaging Formats, Databases, Static Content, Management, Service Discovery,... Mỗi quan hệ của các thành phần và giao tiếp giữa chúng được biểu diễn được mô tả dưới Hình 1.3.



Hình 1.3 Kiến trúc ứng dụng microservice điển hình [16]

- **Client (Máy khách):** Đây là thành phần cơ bản nhất của mọi kiến trúc microservice, Client là các thiết bị khác nhau, cố gắng thực hiện các tác vụ khác nhau từ sử dụng dịch vụ đến quản trị hệ thống.
- **Identity Providers (Cung cấp định danh):** Khi client gửi yêu cầu, yêu cầu đó sẽ được chuyển tiếp tới một nhà cung cấp định danh để xác thực người dùng. Sau đó, yêu cầu sẽ được chuyển tới API Gateway để tiếp tục xử lý.
- **API Gateway:** client không gửi yêu cầu trực tiếp tới các service mà thông qua API Gateway. Đây là một điểm đầu vào cho phép chuyển tiếp các yêu cầu từ client tới các microservice tương ứng. Các microservice cũng có thể trực tiếp trao đổi với nhau hoặc thông qua API Gateway để xử lý các yêu cầu của client.
- **Databases (Cơ sở dữ liệu):** Mỗi microservice có thể có một cơ sở dữ liệu riêng để lưu trữ thông tin liên quan đến chức năng hoặc nghiệp vụ của nó. Các API tương ứng được sử dụng để tương tác với cơ sở dữ liệu này.
- **Static Content:** Khi client gửi yêu cầu, kết quả sau khi xử lý sẽ được trả về cho client. Các nội dung này có thể được lưu trữ trên các dịch vụ đám mây và được gửi đến client thông qua các CDN (Content Delivery Network).
- Trong kiến trúc microservice, CDN có thể được sử dụng để lưu trữ tĩnh nội dung, chẳng hạn như hình ảnh, video hoặc trang HTML, và phục vụ chúng trực tiếp cho client, giảm tải cho các dịch vụ backend và cải thiện hiệu suất tổng thể của hệ thống. CDN cũng có thể được sử dụng để phân phối nội dung động, chẳng hạn như phản hồi API.
- **Managerment:** Thành phần quản lý có nhiệm vụ cân bằng tải các microservice đang hoạt động trên các node hoặc instance và xác định lỗi. Việc quản lý các dịch vụ và tài nguyên trong kiến trúc Microservice là rất quan trọng. Các công cụ quản lý dịch vụ như Kubernetes, Docker và AWS giúp quản lý các dịch vụ và tài nguyên trong hệ thống một cách hiệu quả.
- **Service Registry và Service Discovery (Truy tìm dịch vụ):** Service Registry giúp xác định địa chỉ của một microservice khi nó khởi chạy và hủy đăng ký khi nó bị tắt. Service Discovery được sử dụng để tìm kiếm các microservice đã đăng ký và có sẵn để sử dụng.
- **Remote service** là một thành phần quan trọng trong kiến trúc microservice, cho phép các microservice hoạt động một cách độc lập và đồng thời giao tiếp với nhau một cách hiệu quả. Nó thường được triển khai dưới dạng các API hoặc giao thức mạng, cho phép các microservice trao đổi thông tin và thực hiện các tác vụ

phức tạp. Các phương pháp triển khai remote service khác nhau có thể được sử dụng tùy thuộc vào yêu cầu của hệ thống và các tác vụ cần thực hiện.

Tóm lại, đây là các thành phần cơ bản của một kiến trúc Microservice điển hình. Tất cả các thành phần này đều đóng vai trò quan trọng, đảm bảo hệ thống hoạt động hiệu quả, đáp ứng các yêu cầu được đặt ra.

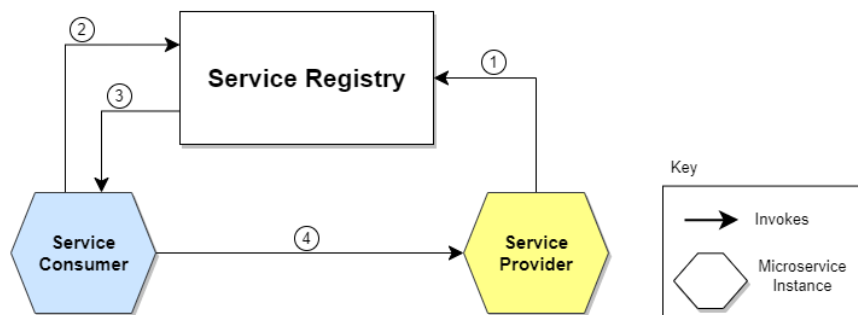
1.4.3 Các mẫu kiến trúc microservice

Hiện nay, có nhiều kiến trúc Microservice khác nhau được sử dụng trong các ứng dụng phần mềm. Các mẫu kiến trúc microservice là các mẫu thiết kế được sử dụng để xây dựng và tổ chức các microservice theo dạng modul và có khả năng mở rộng. Dưới đây là một số mẫu kiến trúc Microservice phổ biến:

Service Discovery

Microservices là một cách phổ biến để xây dựng các ứng dụng phức tạp. Tuy nhiên, khi xây dựng một ứng dụng dựa trên microservices, một trong những thách thức là quản lý tính động của môi trường. Đó là lý do tại sao khái niệm về Service Discovery ra đời.

Service Discovery là một cơ chế cho phép các microservices giao tiếp với nhau mà không cần biết chính xác vị trí của chúng. Trong một ứng dụng dựa trên microservices, các dịch vụ thường chạy trong môi trường ảo hóa hoặc container, và vị trí của chúng có thể thay đổi động. Do đó, chúng ta cần một cách để theo dõi vị trí của từng phiên bản dịch vụ, và đó là lúc Service Discovery giúp ích. Service Discovery giúp chúng ta biết được vị trí của từng phiên bản dịch vụ bằng cách hoạt động như một đăng ký trong đó các địa chỉ của tất cả các phiên bản được theo dõi. Khi một khách hàng muốn thực hiện yêu cầu đến dịch vụ, họ phải sử dụng cơ chế Service Discovery để xác định vị trí của phiên bản dịch vụ mà họ cần giao tiếp.



Hình 1.4 Kỹ thuật Service Discovery [5]

Nhà cung cấp dịch vụ sẽ gửi địa chỉ của mình đến Cơ quan đăng ký dịch vụ (một cơ sở dữ liệu chứa tất cả các địa chỉ của các phiên bản dịch vụ đang có sẵn). Người dùng dịch vụ sẽ hỏi Máy chủ khám phá dịch vụ để tìm vị trí của Nhà cung cấp dịch vụ. Máy chủ khám phá dịch vụ sẽ tìm kiếm địa chỉ của Nhà cung cấp dịch vụ trong cơ sở dữ liệu của Cơ quan đăng ký dịch vụ và trả lại địa chỉ đó cho Người dùng dịch vụ. Sau đó, Người dùng dịch vụ có thể gửi yêu cầu trực tiếp đến Nhà cung cấp dịch vụ.

Có hai mô hình chính để triển khai Service Discovery: Client-Side Discovery và Server-Side Discovery. Với Client-Side Discovery, khách hàng chịu trách nhiệm xác định vị trí mạng của các phiên bản dịch vụ có sẵn và cân bằng tải yêu cầu giữa chúng. Trong khi đó, với Server-Side Discovery, một trung gian cân bằng tải được sử dụng để định tuyến yêu cầu đến các phiên bản dịch vụ có sẵn.

Để đăng ký và hủy đăng ký các phiên bản dịch vụ với Service Registry, có hai cách tiếp cận khả dụng: self-registration và third-party registration. Với self-registration, một phiên bản dịch vụ chịu trách nhiệm đăng ký và hủy đăng ký chính nó trong Service Registry. Trong khi đó, với third-party registration, Service Registry chịu trách nhiệm đăng ký và hủy đăng ký các phiên bản dịch vụ.

Tóm lại, Service Discovery là một cơ chế quan trọng để quản lý microservices trong một môi trường động. Bằng cách sử dụng Service Registry để theo dõi các phiên bản dịch vụ, khách hàng có thể xác định vị trí của các dịch vụ mà họ cần giao tiếp, ngay cả khi vị trí của chúng thay đổi động.

Circuit Breaker

Mô hình Circuit Breaker là một mô hình thiết kế thuộc danh mục các mô hình thiết kế bền vững, cho phép các nhà phát triển ngăn chặn sự cố lan truyền trong kiến trúc Microservices bằng cách gọi các dịch vụ từ xa thông qua một proxy.

Trong kiến trúc Microservices, các dịch vụ giao tiếp với nhau và đôi khi xảy ra sự cố dịch vụ hoặc mất nhiều thời gian để phản hồi yêu cầu do vấn đề kết nối mạng. Mô hình Circuit Breaker hoạt động dựa trên ba trạng thái đó là Closed, Open, Haft Open. Ở trạng thái Closed, Circuit Breaker cho phép các service giao tiếp với nhau như bình thường và đếm số lỗi giữa các Microservices. Nếu số lỗi vượt quá giá trị ngưỡng nó sẽ chuyển sang trạng thái Open, khi đó nó sẽ tự động dừng gửi yêu cầu trong một khoảng thời gian cụ thể. Sau khi thời gian chờ kết thúc, Circuit Breaker sẽ chuyển sang trạng

thái Half-Open. Ở trạng thái này, nó sẽ cho phép một số yêu cầu giới hạn để kiểm tra xem Microservice có hoạt động hay không. Nếu các yêu cầu đó thành công, trạng thái sẽ chuyển sang Closed, cho phép các Microservices tiếp tục hoạt động bình thường. Nếu không, nó sẽ lại chặn các yêu cầu trong khoảng thời gian chờ đã xác định. Các trạng thái được mô tả qua Hình 1.5.



Hình 1.5 Mô tả trạng thái trong mô hình Circuit Breaker [6]

Ý tưởng của mô hình Circuit Breaker rất đơn giản và nó có các thư viện bên thứ ba chuyên biệt cho hầu hết các ngôn ngữ lập trình phổ biến để giảm lược việc triển khai. Tuy nhiên, triển khai mô hình Circuit Breaker yêu cầu việc quản lý cơ sở hạ tầng tốt để duy trì các Circuit Breaker và có thể xảy ra các vấn đề về thông lượng nếu các Circuit Breaker không được cấu hình đúng cách. Bên cạnh đó, kiểm thử triển khai Circuit Breaker có thể gặp khó khăn.

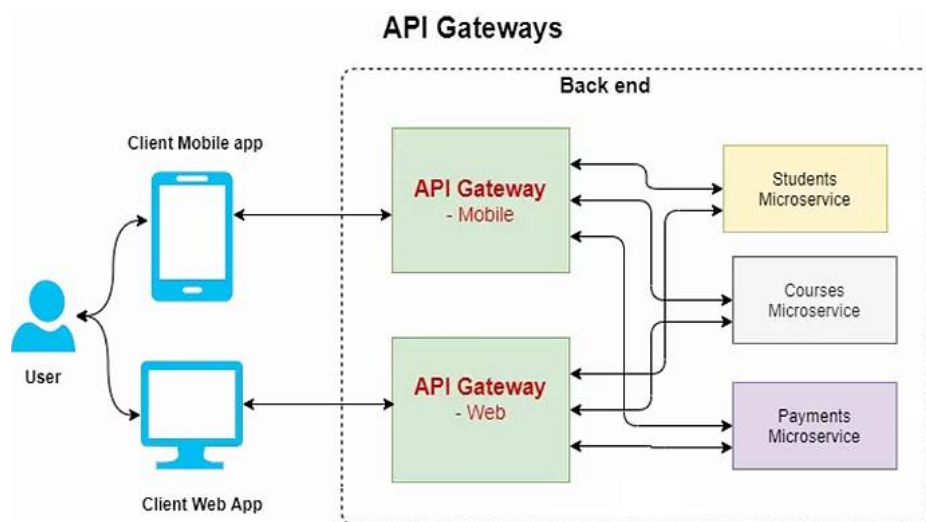
API Gateway

Mô hình kiến trúc Microservice thường xuyên sử dụng API Gateway để giải quyết vấn đề truy cập đến các dịch vụ Microservices. API Gateway là một điểm đầu vào duy nhất cho tất cả các client và giúp họ truy cập vào các dịch vụ Microservices.

Ví dụ, khi xây dựng một cửa hàng trực tuyến sử dụng kiến trúc Microservices và triển khai trang chi tiết sản phẩm. Trang web cần hiển thị thông tin chi tiết sản phẩm từ nhiều dịch vụ khác nhau, chẳng hạn như dịch vụ cung cấp thông tin sản phẩm, dịch vụ giá cả, dịch vụ đặt hàng và dịch vụ đánh giá sản phẩm. Vì vậy, mã hiển thị trang chi tiết sản phẩm phải truy xuất thông tin từ tất cả các dịch vụ này. Để giải quyết vấn đề này,

API Gateway xử lý các yêu cầu theo hai cách. Một số yêu cầu được định tuyến đến dịch vụ phù hợp, trong khi các yêu cầu khác được phân phối đến nhiều dịch vụ. API Gateway cung cấp API tối ưu cho từng client một cách khác nhau. Nhờ việc sử dụng API Gateway, client không cần quan tâm đến cách dịch vụ được phân chia ra thành các Microservices hay phải tìm kiếm các vị trí của các phiên bản dịch vụ. Thay vào đó, client chỉ cần gọi API Gateway để truy cập vào các dịch vụ cần thiết. Ngoài ra, API Gateway còn có thể thực hiện bảo mật, ví dụ như xác minh client đã được ủy quyền để thực hiện yêu cầu.

Một biến thể khác của mô hình này là mô hình Backends for Frontends (BFFs), nơi mỗi loại client có một API Gateway riêng. Ví dụ, có ba loại client: ứng dụng web, ứng dụng di động và ứng dụng bên thứ ba. Có ba API Gateway khác nhau, mỗi API Gateway cung cấp API cho khách hàng của nó.



Hình 1.6 Kiến trúc ứng dụng sử dụng API gateway [19]

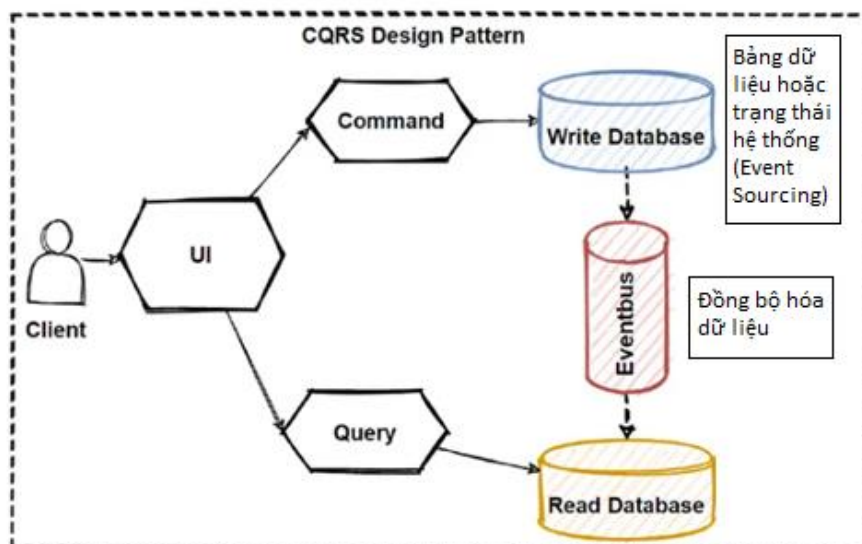
Việc sử dụng API Gateway có nhiều lợi ích như giảm số lượng yêu cầu/roundtrips và cung cấp API tối ưu cho từng khách hàng. Tuy nhiên, mô hình này có một số hạn chế như phức tạp hóa hệ thống và tăng thời gian phản hồi do phải qua một mạng thêm.

CQRS

CQRS (Command and Query Responsibility Segregation) là một mẫu thiết kế thường được sử dụng trong kiến trúc Microservice. Nó đề xuất việc tách các hoạt động đọc và ghi cho cơ sở dữ liệu.

Trong các ứng dụng monolithic truyền thống, chỉ có một cơ sở dữ liệu phục vụ cả hoạt động đọc và ghi. Điều này có nghĩa là cơ sở dữ liệu đang xử lý các truy vấn kết nối

phức tạp và thực hiện các hoạt động CRUD (Create, Read, Update, Delete). Tuy nhiên, khi ứng dụng trở nên phức tạp hơn, các hoạt động CRUD này trở nên khó quản lý. Ví dụ, khi đọc cơ sở dữ liệu, nếu ứng dụng yêu cầu một truy vấn cần phải kết hợp hơn 10 bảng, điều này sẽ khóa cơ sở dữ liệu do độ trễ tính toán của truy vấn. Ngoài ra, nếu cần thực hiện các hoạt động CRUD, sẽ cần thực hiện các kiểm tra và xử lý logic phức tạp, gây ra các hoạt động khóa cơ sở dữ liệu. Để giải quyết vấn đề đó, CQRS đề xuất sử dụng nguyên tắc "tách biệt trách nhiệm" và phân tách các hoạt động đọc và ghi cho cơ sở dữ liệu thành hai cơ sở dữ liệu khác nhau. Như vậy, có thể sử dụng các cơ sở dữ liệu khác nhau cho các loại hoạt động đọc và ghi khác nhau, chẳng hạn như sử dụng NoSQL cho các hoạt động đọc và sử dụng cơ sở dữ liệu SQL cho các hoạt động CRUD. CQRS phân tách hoạt động đọc và ghi thành hai cơ sở dữ liệu khác nhau. Các lệnh thực hiện cập nhật dữ liệu và các truy vấn thực hiện đọc dữ liệu. Do đó, các lệnh có thể được xử lý bằng hệ thống đẩy tin nhắn để xử lý các lệnh bất đồng bộ. Các truy vấn không bao giờ thay đổi cơ sở dữ liệu và luôn trả về dữ liệu JSON. Như vậy, có thể cô lập lệnh và truy vấn. Để cô lập các lệnh và truy vấn, thực hành tốt nhất là tách biệt các cơ sở dữ liệu đọc và ghi thành hai cơ sở dữ liệu vật lý khác nhau. Như vậy, nếu ứng dụng đọc nhiều hơn là ghi, có thể xác định các lược địa dữ liệu tùy chỉnh để tối ưu hóa cho các hoạt động truy vấn. Materialized view là một ví dụ điển hình để triển khai cơ sở dữ liệu đọc. Từ đó có thể tránh các kết hợp phức tạp và ánh xạ dữ liệu với định hướng tốt cho các hoạt động truy vấn.



Hình 1.7 Mô hình CQRS [9]

Bằng cách phân tách các cơ sở dữ liệu đọc và ghi trong mô hình CQRS, mối quan tâm chính là đồng bộ hóa đúng giữa hai cơ sở dữ liệu này. Nên đồng bộ hóa hai cơ sở dữ liệu này và duy trì đồng bộ hóa. Điều này có thể được giải quyết bằng cách sử dụng chiến lược sao chép dữ liệu, chẳng hạn như kiến trúc dựa trên sự kiện hoặc kiến trúc đẩy tin nhắn. Trong các kiến trúc này, có thể sử dụng một trình điều khiển tin nhắn để xử lý sao chép dữ liệu giữa hai cơ sở dữ liệu. Như vậy, chúng ta có thể đảm bảo rằng cả hai cơ sở dữ liệu luôn được đồng bộ hóa.

Saga Parttern

Giao dịch phân tán trong kiến trúc microservice đặt ra hai thách thức chính. Thứ nhất là duy trì tính ACID (Atomicity, Consistency, Isolation, and Durability) để đảm bảo tính chính xác của giao dịch là rất quan trọng. Thách thức thứ hai là quản lý mức độ cô lập giao dịch, xác định lượng dữ liệu hiển thị trong một giao dịch khi các dịch vụ khác truy cập cùng một dữ liệu đồng thời. Một mô hình rộng rãi được sử dụng để triển khai giao dịch phân tán trong kiến trúc microservice là giao thức Two-Phase Commit (2PC) gồm hai giai đoạn là Prepare Phase và Commit Phase. Trong Prepare Phase, Điều phối viên hỏi các node tham gia xem chúng đã sẵn sàng thực hiện giao dịch chưa. Những node trả lời có hoặc không. Chuyển sang Commit Phase, nếu tất cả các node tham gia phản hồi có trong Prepare Phase, điều phối viên sẽ yêu cầu tất cả các node đó cam kết. Nếu ít nhất một node trả về giá trị âm, điều phối viên sẽ yêu cầu tất cả những node tham gia hoàn nguyên các giao dịch cục bộ của chúng. Tuy nhiên, 2PC có một số điểm yếu, chẳng hạn như nút điều phối trở thành điểm thất bại đơn lẻ và các vấn đề về tính mở rộng và hiệu suất trong kiến trúc dịch vụ nhỏ.

Mô hình Saga sử dụng một chuỗi các giao dịch cục bộ để thay thế cho giao thức Two-Phase Commit (2PC) truyền thống trong việc quản lý các giao dịch phân tán. Trong mô hình Saga, mỗi dịch vụ nhỏ trong kiến trúc microservice thực hiện một giao dịch cục bộ để thực hiện một chức năng cụ thể. Nếu một giao dịch cục bộ thất bại, SEC (Saga Execution Coordinator) sẽ triển khai một giao dịch bù để hoàn tất các giao dịch đã hoàn thành trước đó và đảm bảo tính toàn vẹn dữ liệu. Mô hình Saga cung cấp hai cách tiếp cận để triển khai: Saga Choreography và Saga Orchestration. Trong Saga Choreography, mỗi dịch vụ nhỏ có thể nhúng với SEC và tương tác trực tiếp với các dịch vụ khác để thực hiện các giao dịch cục bộ. Trong Saga Orchestration, có một bộ điều khiển đơn lẻ chịu trách nhiệm quản lý trạng thái của toàn bộ giao dịch phân tán.

Mô hình Saga có nhiều lợi ích so với 2PC. Nó giúp giảm thiểu các thách thức về tính mở rộng và hiệu suất trong kiến trúc phân tán, đồng thời cung cấp một cách tiếp cận linh hoạt và có tính mở rộng để quản lý các giao dịch phân tán trong một hệ thống phân tán.

Bulkhead Pattern

Mô hình Bulkhead trong kiến trúc microservice là một giải pháp để giảm thiểu các tác động tiêu cực của một dịch vụ lỗi đến các dịch vụ khác trong cùng một kiến trúc microservice. Mô hình Bulkhead tách các dịch vụ và ngăn cách chúng bằng cách sử dụng các tường ngăn (bulkhead) để giới hạn tác động của một dịch vụ lỗi đến các dịch vụ khác.

Trong kiến trúc microservice, các dịch vụ thường chia sẻ các tài nguyên như CPU, bộ nhớ và băng thông mạng. Khi một dịch vụ gặp sự cố, nó có thể tiêu tốn tài nguyên quan trọng và ảnh hưởng đến các dịch vụ khác trong cùng kiến trúc microservice. Điều này có thể dẫn đến hiện tượng "domino effect", khi một sự cố nhỏ lan rộng và gây ra sự cố lớn hơn.

Mô hình Bulkhead sử dụng các tường ngăn để tách các dịch vụ và giới hạn tác động của một dịch vụ lỗi đến các dịch vụ khác. Các tường ngăn được sử dụng để chia sẻ các tài nguyên giữa các dịch vụ, nhưng với một giới hạn nhất định. Khi một dịch vụ gặp sự cố và tiêu tốn nhiều tài nguyên, các tường ngăn sẽ ngăn chặn sự cố lan rộng đến các dịch vụ khác. Như vậy, các dịch vụ khác sẽ tiếp tục hoạt động mà không bị ảnh hưởng bởi sự cố của dịch vụ lỗi.

Mô hình Bulkhead cũng có thể được sử dụng để giảm thiểu tác động của các tấn công mạng đến hệ thống. Nó có thể giúp giới hạn tác động của các tấn công và ngăn chặn chúng lan rộng đến các dịch vụ khác trong kiến trúc microservice.

Nhìn chung, mô hình Bulkhead là một giải pháp hữu ích để giảm thiểu tác động của các sự cố và tấn công đến các dịch vụ khác trong kiến trúc microservice. Nó sử dụng các tường ngăn để tách các dịch vụ và giới hạn tác động của một dịch vụ lỗi đến các dịch vụ khác, đồng thời giúp giảm thiểu hiện tượng "domino effect" trong kiến trúc phân tán.

Trên đây chỉ là một số ví dụ về các mẫu kiến trúc microservice phổ biến. Bằng cách sử dụng những mẫu kiến trúc này, có thể tạo ra các kiến trúc microservice theo hướng mô hình hóa, có khả năng mở rộng và khả năng chịu lỗi.

1.4.4 Ưu nhược điểm của kiến trúc microservice

Kiến trúc microservice là một kiến trúc phần mềm đang được sử dụng rộng rãi trong các ứng dụng phần mềm hiện đại. Kiến trúc này có nhiều ưu điểm nổi trội giúp mang lại những lợi ích đáng kể trong xây dựng và phát triển hệ thống phần mềm.

Đầu tiên phải kể đến là Loose coupling, tạm dịch là khớp nối lỏng lẻo. Đây là một nguyên tắc chính trong kiến trúc microservices và nó đề cập đến mức độ phụ thuộc lẫn nhau giữa các thành phần phần mềm. Ý tưởng là giảm mức độ kết nối giữa các thành phần để những thay đổi trong một thành phần có tác động tối thiểu đến các thành phần khác. Có thể tiếp cận nó bằng cách sử dụng API để giao tiếp giữa các vi dịch vụ. Mỗi microservice hiển thị một bộ API mà các microservice khác có thể sử dụng để tương tác với nó. Bằng cách sử dụng API, các dịch vụ siêu nhỏ có thể giao tiếp với nhau theo cách tách rời mà không cần biết hoạt động bên trong của nhau. Một cách tiếp cận khác để triển khai khớp nối lỏng lẻo là sử dụng hàng đợi tin nhắn. Theo cách tiếp cận này, các dịch vụ siêu nhỏ giao tiếp với nhau bằng cách gửi tin nhắn đến hàng đợi tin nhắn, đóng vai trò trung gian. Trong kiến trúc microservice, mỗi microservice có bộ lưu trữ dữ liệu riêng. Điều này cho phép các dịch vụ siêu nhỏ hoạt động độc lập và giảm mức độ liên kết giữa chúng. Phân cấp lưu trữ dữ liệu cũng có thể giúp cải thiện khả năng mở rộng, vì mỗi dịch vụ siêu nhỏ có thể mở rộng quy mô lưu trữ dữ liệu của riêng mình một cách độc lập. Và còn nhiều cách tiếp cận khác để thực hiện Loose coupling tuy nhiên không phải tất cả chúng đều khả thi. Về tổng quan, khớp nối lỏng lẻo trong kiến trúc microservice, giúp hệ thống linh hoạt hơn, có thể mở rộng quy mô và dễ bảo trì hơn.

Ưu điểm thứ hai chính là tính độc lập, là một trong những yếu tố quan trọng của kiến trúc microservice. Nó được thể hiện ở chỗ kiến trúc Microservice cho phép các dịch vụ độc lập phát triển và triển khai một cách độc lập. Điều này giúp tăng tính linh hoạt, giảm thiểu sự phụ thuộc và giúp các nhà phát triển có thể phát triển các tính năng mới một cách nhanh chóng hơn. Vì mỗi dịch vụ hoạt động độc lập, các dịch vụ có thể được mở rộng một cách độc lập để đáp ứng nhu cầu tăng trưởng của hệ thống. Mỗi dịch vụ hoạt động độc lập, do đó, việc bảo trì và cập nhật các dịch vụ cũng dễ dàng hơn. Nếu một dịch vụ cần được sửa chữa hoặc nâng cấp, các dịch vụ khác vẫn có thể tiếp tục hoạt động mà không bị ảnh hưởng. . Với tính độc lập trong microservice, các vấn đề có thể được giới hạn trong một dịch vụ cụ thể, không ảnh hưởng đến các dịch vụ khác. Điều này giúp đảm bảo tính tin cậy và sẵn sàng của hệ thống. Bên cạnh đó, các dịch vụ có thể

được triển khai trên nhiều nền tảng khác nhau và có thể được chuyển đổi qua lại giữa các nền tảng một cách dễ dàng giúp làm tăng tính linh hoạt.

Tính nhỏ gọn cũng là một trong những ưu điểm của Microservice. Các dịch vụ trong kiến trúc Microservice được thiết kế để thực hiện một chức năng cụ thể, giúp giảm thiểu sự phức tạp và đảm bảo tính đơn giản của ứng dụng phần mềm.

Cuối cùng là tính độc lập về cơ sở dữ liệu. Đối với cách tiếp cận truyền thống, chỉ có một cơ sở dữ liệu tập trung duy nhất, thường là một cơ sở dữ liệu SQL chuẩn hóa được sử dụng cho toàn bộ ứng dụng và tất cả các hệ thống con bên trong của nó. Còn trong Microservice, mỗi dịch vụ nhỏ sở hữu dữ liệu của riêng nó, được quản lý trong một cơ sở dữ liệu riêng và không phụ thuộc vào cơ sở dữ liệu chung của hệ thống. Việc đóng gói dữ liệu đảm bảo rằng các vi dịch vụ được liên kết lỏng lẻo và có thể phát triển độc lập với nhau. Mỗi dịch vụ có cơ sở dữ liệu độc lập giúp đảm bảo tính bảo mật của hệ thống. Nếu một dịch vụ bị tấn công hoặc dữ liệu bị lộ ra ngoài, các dịch vụ khác vẫn được bảo vệ.

Mặc dù kiến trúc Microservice có nhiều ưu điểm, nhưng nó cũng tồn tại một số nhược điểm như về độ phức tạp, độ trễ, chi phí cao và khó khăn trong việc kiểm soát các phiên bản của ứng dụng. Kiến trúc Microservice là một kiến trúc phức tạp, yêu cầu kỹ năng và kinh nghiệm lập trình viên để triển khai và quản lý. Các dịch vụ Microservice phải được phát triển, triển khai và quản lý độc lập, tạo ra thách thức trong việc quản lý và vận hành hệ thống. Vì các dịch vụ Microservice phải giao tiếp với nhau qua mạng, việc truyền thông giữa các dịch vụ có thể tạo ra độ trễ. Điều này có thể làm giảm hiệu suất của hệ thống và làm tăng thời gian phản hồi. Triển khai kiến trúc Microservice có thể tốn kém hơn so với kiến trúc monolithic, vì mỗi dịch vụ phải có cơ sở hạ tầng, cơ sở dữ liệu và các tài nguyên khác của riêng nó. Chi phí vận hành và quản lý hệ thống có thể từ đó mà tăng lên. Mặt khác, việc các dịch vụ được phát triển một cách độc lập, khiến cho công việc kiểm soát phiên bản có thể trở nên phức tạp. Nếu các dịch vụ khác nhau sử dụng các phiên bản khác nhau của các thư viện hoặc phần mềm khác nhau, có thể dẫn đến các vấn đề tương thích và độ tin cậy của hệ thống.

Kiến trúc Microservice có nhiều ưu điểm như tính độc lập, nhỏ gọn, linh hoạt, khả năng mở rộng và độc lập về cơ sở dữ liệu. Nhưng cũng có những nhược điểm và thách thức cần được xem xét và giải quyết khi triển khai và quản lý.

1.4.5 Ứng dụng microservice dựa trên container

Microservice dựa trên container là một trong những cách triển khai kiến trúc Microservice phổ biến hiện nay. Trong cách triển khai này, mỗi dịch vụ Microservice được đóng gói trong một container độc lập, với tất cả các phụ thuộc và cấu hình cần thiết để chạy dịch vụ đó. Container là một phương tiện để đóng gói và chạy các ứng dụng và dịch vụ một cách độc lập, có thể triển khai và chạy trên bất kỳ nền tảng nào mà không cần phải cài đặt lại các phần mềm. Một số ứng dụng của Microservice dựa trên container bao gồm: triển khai và quản lý các dịch vụ độc lập, khả năng di chuyển ứng dụng linh hoạt giữa các môi trường, tăng tính hiệu quả của việc triển khai và quản lý hệ thống, dễ dàng cập nhật và mở rộng hệ thống.

Mỗi container đóng gói một dịch vụ Microservice độc lập, giúp đảm bảo tính độc lập giữa các dịch vụ và tránh các xung đột giữa chúng. Việc quản lý các dịch vụ độc lập này trở nên dễ dàng hơn. Container cho phép triển khai và di chuyển các dịch vụ Microservice giữa các môi trường khác nhau một cách dễ dàng, làm tối ưu hiệu suất và giảm thiểu chi phí. Các ứng dụng và dịch vụ được đóng gói bởi container theo các tiêu chuẩn chung, đảm bảo tính nhất quán và đồng bộ giữa các dịch vụ Microservice. Nó cũng cung cấp một môi trường đóng gói độc lập và bảo mật, giúp đảm bảo tính bảo mật của các dịch vụ Microservice. Microservice dựa trên container giúp tăng tính hiệu quả của việc triển khai và quản lý hệ thống bằng cách cho phép các container được quản lý một cách tự động bởi các công cụ như Kubernetes. Điều này giúp giảm thiểu thời gian và công sức cần thiết cho việc triển khai và quản lý hệ thống. Không những thế, nó còn cho phép các dịch vụ được cập nhật và mở rộng một cách độc lập làm khả năng mở rộng của hệ thống.

Microservice dựa trên container là một cách triển khai kiến trúc Microservice hiệu quả và tiện lợi, giúp tăng tính độc lập, tính khả chuyển và tính linh hoạt của hệ thống dựa trên các công nghệ container như Docker và Kubernetes. Những công nghệ này giúp microservice phát huy tối đa những ưu điểm vốn có kết hợp với những lợi ích tuyệt vời mà container đem lại. Giải pháp container hóa ứng dụng, Docker hay kubernetes sẽ được giải thích rõ hơn trong chương tiếp theo.

Tổng kết lại, nội dung chương một đã là những hiểu biết và nghiên cứu của em về khái niệm kiến trúc hệ thống, các loại kiến trúc hệ thống phổ biến và đi sâu vào kiến trúc microservice. Trong chương tiếp theo, em sẽ trình bày về các công nghệ dựa trên

container, đưa ra cơ sở ứng dụng những công nghệ đó trong việc triển khai thử nghiệm ứng dụng microservice trong Chương 3.

CHƯƠNG 2. CÁC GIẢI PHÁP CÔNG NGHỆ

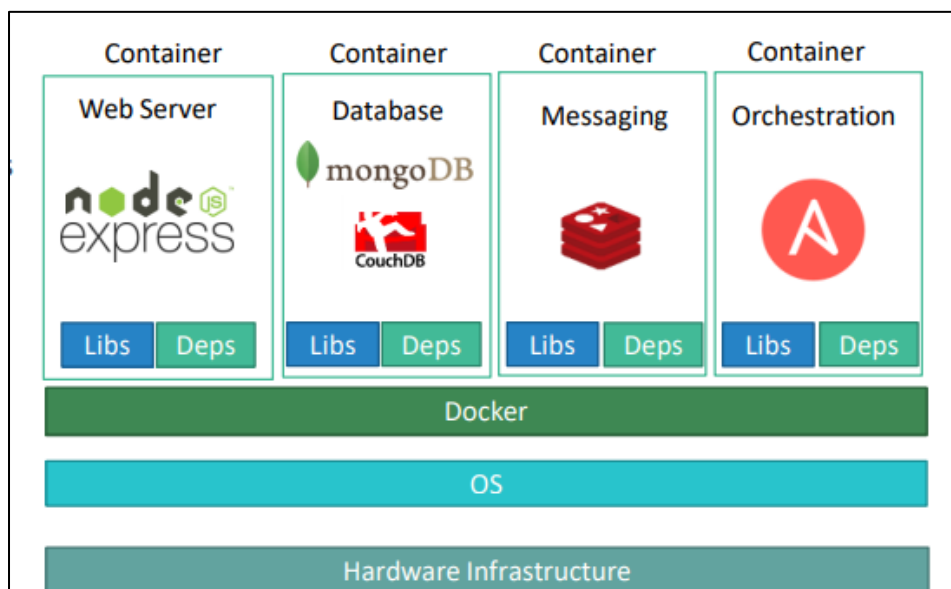
Công nghệ ngày nay đang phát triển với tốc độ chóng mặt và ảnh hưởng đến nhiều lĩnh vực, trong đó có phát triển phần mềm. Các giải pháp công nghệ mới đang được đưa vào sử dụng để tăng cường hiệu suất, tính bảo mật và giảm chi phí trong quá trình phát triển phần mềm. Trong các giải pháp công nghệ đó, container đang được sử dụng rộng rãi trong việc triển khai các ứng dụng phân tán. Nội dung chương 2 này sẽ đưa ra cái nhìn từ tổng quan đến chi tiết về các công nghệ, giải pháp mã nguồn mở áp dụng trong đề án này như container, Docker và Kubernetes và cách chúng được ứng dụng trong phát triển và vận hành của các hệ thống phần mềm hiện nay. Đầu tiên container là một cách để đóng gói và vận chuyển ứng dụng cùng với các phụ thuộc của nó, trong một môi trường cô lập. Docker là một nền tảng giúp quản lý các container, cho phép ta dễ dàng tạo, triển khai và chạy các ứng dụng trên các máy chủ khác nhau. Kubernetes là một hệ thống quản lý container mã nguồn mở, giúp quản lý và tự động hóa việc triển khai, mở rộng và quản lý các ứng dụng container trên một mạng lưới các máy chủ.

2.1 Docker

2.1.1 *Khái niệm container*

Khái niệm container trong ngành công nghiệp vận tải là một thùng chứa vật lý để vận chuyển hàng hóa một cách an toàn và phù hợp với nhiều phương thức vận chuyển khác nhau. Trong lĩnh vực công nghệ phần mềm, khái niệm container cũng mang nhiều ý nghĩa tương đồng, container là một đơn vị đóng gói tiêu chuẩn của phần mềm, chứa tất cả thành phần phụ thuộc để chạy một ứng dụng như code, runtime, libraries, dependencies, binaries,... ; container giúp triển khai các ứng dụng một cách độc lập, đáng tin cậy và trên nhiều môi trường khác nhau khác nhau. Ứng dụng được đóng gói trong container được cách ly với môi trường và các ứng dụng được đóng gói trên các container khác trên cùng một hệ điều hành dùng chung (Linux, Windows,...). Quá trình đóng gói tất cả thành phần cần thiết để chạy một ứng dụng được gọi là containerization, đây là một cách tiếp cận mang nhiều lợi ích to lớn trong phát triển phần mềm. Có nhiều loại container khác nhau như LXC, LXD, LXCFS,...

Theo cách truyền thống, để chạy một ứng dụng trên máy tính cần cài một phiên bản hệ điều hành phù hợp kèm theo đó là các thư viện và các thành phần phụ thuộc. Khi chạy nhiều ứng dụng trên cùng một cơ sở hạ tầng phần cứng, thứ nhất cần đảm bảo phiên bản hệ điều hành phải phù hợp với tất cả các ứng dụng; thứ hai cần kiểm tra tính tương thích giữa các service, thư viện và các thành phần phụ thuộc trên hệ điều hành. Sự cố có thể xảy ra khi một service yêu cầu một phiên bản của thư viện phụ thuộc nhưng service khác lại yêu cầu một phiên bản khác. Ứng dụng liên tục cập nhật, các thành phần kiến trúc thay đổi theo thời gian và được nâng cấp lên phiên bản mới hơn, các thay đổi về cơ sở dữ liệu,... cứ mỗi một lần thay đổi lại dẫn đến một quy trình kiểm tra lại tính tương thích giữa các thành phần khác nhau chưa kể việc chuyển ứng dụng từ môi trường này sang môi trường khác cũng phát sinh các sự cố do khác biệt cấu hình của thư viện và các thành phần phụ thuộc khác. Yêu cầu về hướng tiếp cận mới giúp giải quyết những vấn đề về tương thích, sửa đổi cập nhật trong các thành phần mà không ảnh hưởng đến các thành phần khác được đặt ra. Và containerization đã thực sự giải quyết được những vấn đề đó. Theo hướng tiếp cận này, các container là những môi trường hoàn toàn biệt lập giống như Virtual Machine nhưng khác chỗ chúng cùng chia sẻ một nhân hệ điều hành.



Hình 2.1 Container hóa ứng dụng với Docker [2]

2.1.2 Tổng quan về Docker

Docker là một dự án mã nguồn mở dùng để building, deploying và running ứng dụng dễ dàng hơn bằng cách sử dụng các containers (trên nền tảng ảo hóa). Docker giúp

tự động hóa việc triển khai các ứng dụng dưới dạng container có thể chạy trên nền tảng cloud hoặc on-premises. Docker sử dụng container LXC và LXK. Ban đầu ứng dụng được viết bằng Python tuy nhiên sau đó đã được chuyển sang Go-lang.

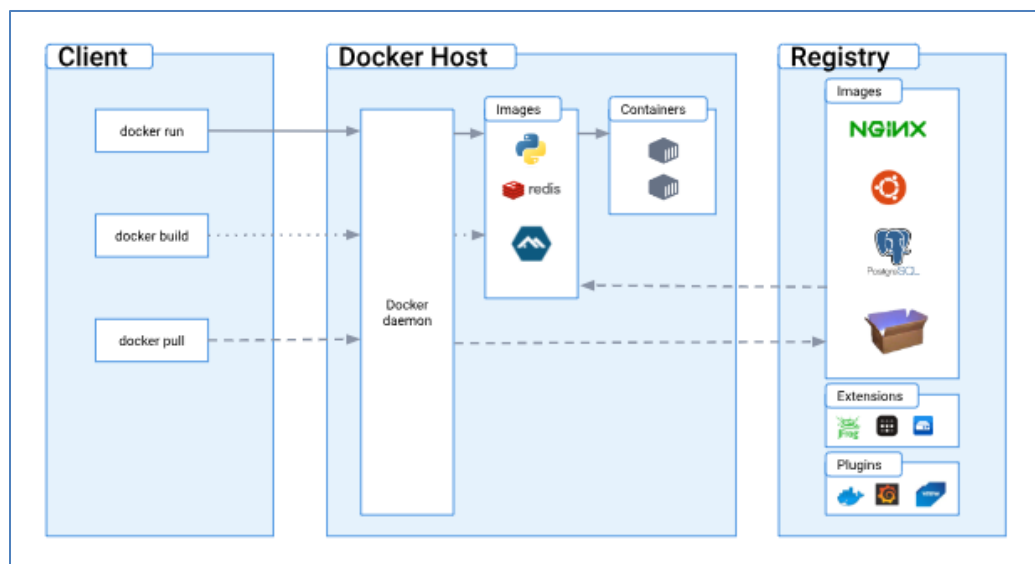
Docker cung cấp một lớp trừu tượng và tự động ảo hóa dựa trên hệ điều hành Linux. Docker sử dụng những tài nguyên cốt lõi của Linux như cgroups, kernel, quản lý tệp để cho phép các container chạy độc lập trên nền hệ điều hành Linux. Các thay đổi được lưu trữ trong các Docker image, các lớp tệp hệ thống được tạo ra và lưu lại dựa theo từng lớp (layer). Điều này giúp cho Docker image giảm dung lượng đáng kể so với máy ảo (VM). Ban đầu, các ứng dụng muốn chạy bằng Docker phải là ứng dụng chạy được trên Linux nhưng giờ đây Docker đã hỗ trợ cả các ứng dụng chạy trên hệ điều hành Windows trong các Windows container. Để chạy Docker trên máy chủ Windows hoặc Mac, thay vì phải chạy thông qua máy ảo quản lý bởi Hyper-V hoặc máy ảo VirtualBox như trước, Docker đã có phiên bản ứng dụng có thể chạy trên hệ điều hành Windows và Mac có tên là Docker Desktop. Tuy nhiên cần cài đặt thêm Windows Subsystem for Linux giúp tạo môi trường để chạy các container Linux mà không cần qua máy ảo, lúc này Docker cũng chạy nhanh và ổn định hơn so với việc sử dụng Hyper-V.

2.1.3 Kiến trúc và thành phần của Docker

Kiến trúc Docker sử dụng là client-server. Trong đó, Docker client giao tiếp với Docker Daemon để thực hiện các công việc như build, run, distribute đối với Docker container. Docker client và Docker daemon có thể chạy trên cùng một hệ thống, hoặc có thể được kết nối từ xa. Chúng giao tiếp bằng REST API, qua UNIX socket hoặc giao diện mạng. Một Docker client khác đó là Docker Compose, cho phép build, run, distribute đối với một tập hợp nhiều container. Kiến trúc của Docker được thể hiện như ở Hình 2.2.

Docker daemon (dockerd) nhận yêu cầu từ Docker API và quản lý những thành phần của Docker như images, containers, networks, volumes,... Giữa các daemon cũng có sự giao tiếp để quản lý dịch vụ Docker. Người dùng tương tác với Docker thông qua Docker client, sử dụng lệnh như docker run, docker ps,... Những lệnh này sử dụng Docker API, được Docker client gửi đến dockerd để thực thi yêu cầu của người dùng tương ứng với lệnh mà nó nhận được.

Registries là nơi lưu trữ và phân phối các Docker images. Một registry phổ biến đó là Docker Hub, được quản lý bởi Docker, registry công cộng mà bất cứ ai cũng có thể sử dụng, mặc định Docker sẽ tìm kiếm images trên DockerHub. Registry cũng được cung cấp dưới dạng dịch vụ bởi các nhà cung cấp dịch vụ cloud như Amazon Elastic Container Registry (ECR), Google Container Registry (GCR) hay Azure Container Registry (ACR).



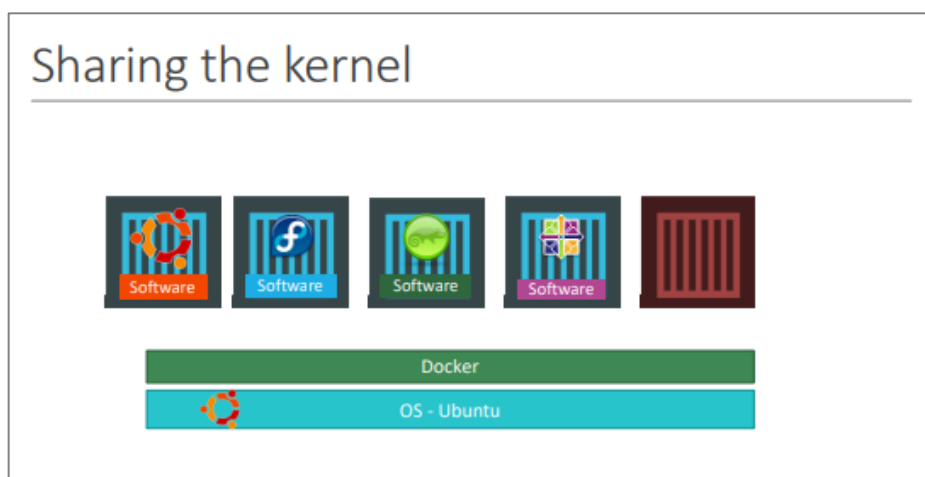
Hình 2.2 Kiến trúc của Docker [18]

Các thành phần như images, containers, volumes,... được gọi là Docker object. Image là một “read-only template”, hiểu đơn giản là một khuôn mẫu để tạo ra container. Thường thì image sẽ dựa trên một image có sẵn với những tùy chỉnh thêm. Ví dụ, có thể dựng một image chạy Nginx dựa trên image Centos mẫu có sẵn thêm vào những tùy chỉnh để cấu hình một ứng dụng web có thể chạy được. Cũng có thể tự xây dựng một image riêng hoặc sử dụng những image được chia sẻ từ cộng đồng Docker Hub. Một image sẽ được build dựa trên những chỉ dẫn của Dockerfile, là những cú pháp đơn giản để định nghĩa những bước cần thiết để tạo và chạy một image. Docker container hoạt động giống như một thư mục (directory), chứa tất cả những thứ cần thiết để một ứng dụng có thể chạy được. Mỗi một docker container được tạo ra từ một docker image. Các thao tác có thể được thực hiện đối với một container là: run, start, stop, move, remove,... Volumes là phần dữ liệu được tạo ra khi container được khởi tạo. Dockerfile là một file chứa tập hợp các lệnh để Docker có thể đọc và thực hiện để đóng gói một image theo yêu cầu người dùng. Docker compose là công cụ dùng để định nghĩa và chạy nhiều

container cho Docker application. Với compose, file YAML được sử dụng để cấu hình các services cho ứng dụng, sau đó dùng lệnh để tạo và chạy từ những cấu hình đó.

2.1.4 So sánh Docker Container với máy ảo (Virtual Machine)

Có một đặc điểm về container đã được nhắc đến trong phần trước đó là các container cùng chia sẻ một nhân hệ điều hành, trước khi đi vào so sánh container với máy ảo (VM), đặc điểm này cần được làm rõ bởi nó một phần tạo nên sự khác biệt giữa container và VM.

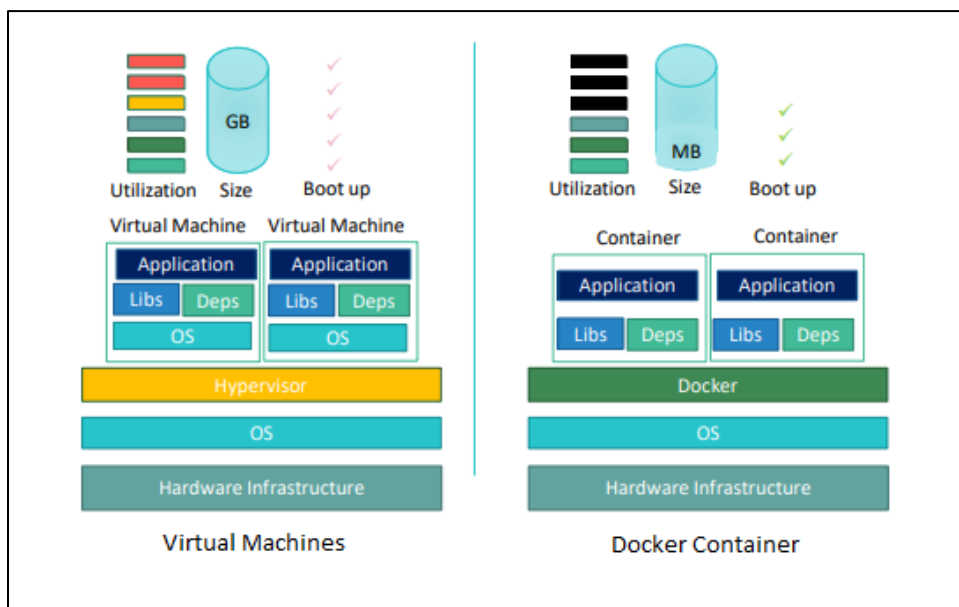


Hình 2.3 Các container dùng chung nhân OS [2]

Các hệ điều hành như Ubuntu, Fedora, Suse hay Centos,... đều bao gồm hai thành phần đó là một nhân hệ điều hành và một bộ phần mềm. Nhân hệ điều hành trong trường hợp này là Linux, có nhiệm vụ tương tác với phần cứng bên dưới. Các hệ điều hành kể trên đều có chung phần nhân Linux, điều tạo nên sự khác biệt giữa chúng là các bộ phần mềm phía trên. Docker có thể chạy bất kỳ loại OS nào miễn là tất cả chúng đều chạy dựa trên cùng một nhân. Ví dụ hệ điều hành cơ bản là Ubuntu, Docker có thể chạy các container dựa trên các bản phân phối khác như Fedora, Centos,... mỗi container chỉ cần có thêm phần mềm bổ sung tương ứng với các bản phân phối còn phần nhân hệ điều hành sẽ được chia sẻ và dùng chung trên nhân máy chủ cơ bản phía dưới. Điều này dẫn đến sự khác biệt giữa container và máy ảo (VM).

Như có thể thấy, trong Hình 2.4, phía bên trái là những mô tả về máy ảo (VMs), đầu tiên ta có là cơ sở hạ tầng phần cứng cơ bản, hệ điều hành được cài trên đó và tiếp đến là Hypervisor - phần mềm chịu trách nhiệm quản lý, giám sát các máy ảo trên nó. Mỗi máy ảo đều có hệ điều hành riêng, thư viện và gói phụ trợ để chạy ứng dụng, do đó mỗi máy ảo đều chiếm một lượng tài nguyên đáng kể, dung lượng đĩa cao và mất nhiều thời

gian để khởi động. Trong trường hợp của Docker container, ta cũng có một hệ điều hành chạy trên cơ sở hạ tầng phần cứng cơ bản, Docker được cài đặt trên đó và quản lý các container. Mỗi container dựa trên nguyên lý Kernel Sharing, chỉ cần bao gồm thư viện và phần phụ trợ của riêng nó. Đó là nguyên nhân khiến Docker container nổi trội hơn so với máy ảo nhờ kích thước nhỏ, chiếm ít tài nguyên phần cứng và khởi động nhanh chóng hơn trong thời gian chỉ mất vài giây trong khi máy ảo mất đến vài phút để khởi động vì nó cần khởi động toàn bộ hệ điều hành của riêng nó. Vì chiếm ít tài nguyên hơn nên mật độ container sẽ cao hơn so với máy ảo xét trên cùng một đơn vị phần cứng, điều đó có nghĩa là nó cho phép chạy nhiều dịch vụ hơn từ đó giúp giảm chi phí. Cũng dựa theo Kernel Sharing, Docker ít bị cô lập hơn vì nhiều tài nguyên được chia sẻ giữa container và kernel còn các máy ảo thì hoàn toàn độc lập với nhau.



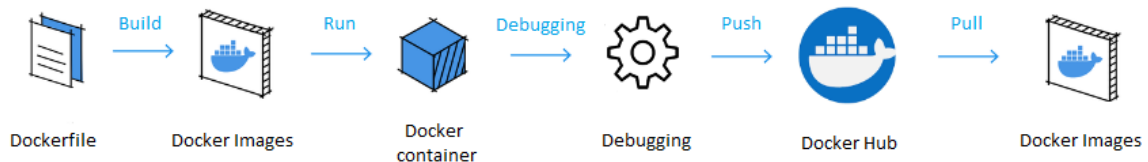
Hình 2.4 So sánh Docker container và Virtual Machines [2]

2.1.5 Quy trình thực thi của một hệ thống sử dụng Docker

Vậy quy trình để một ứng dụng có thể chạy trên các Docker container như thế nào? Một quy trình thực thi được thực hiện với các bước chính đó là build, run, debugging, push và pull.

Đầu tiên một Dockerfile được khởi tạo, Dockerfile này sẽ được build tại một máy chủ đã cài đặt Docker để tạo ra Docker image với lệnh “docker build”. Sau đó, sử dụng lệnh "docker run" để khởi chạy Docker container từ Docker image đã tạo ở bước trước. Trong lệnh này, bạn có thể định cấu hình các thông số như cổng mạng, biến môi trường,

tên container,... Sau khi Docker container được khởi chạy, tiến hành kiểm tra ứng dụng bên trong container bằng cách truy cập vào nó thông qua một cổng mạng hoặc giao diện dòng lệnh. Sau khi thực hiện kiểm tra và sửa lỗi, Docker images sẽ được đẩy lên registry qua tiến trình push. Có thể sử dụng lệnh “docker pull” để tải images trên registry từ bất cứ server nào và chạy Docker container từ images đã được pull xuống server đó.



Hình 2.5 Quy trình thực thi hệ thống chạy container

Từ quy trình trên có thể thấy được lợi thế của việc sử dụng container. Theo truyền thống, các nhà phát triển sau khi phát triển các ứng dụng, họ sẽ bàn giao nó cho nhóm Vận hành để triển khai và quản lý nó trong môi trường product. Điều đó được thực hiện bằng cách, các nhà phát triển sẽ cung cấp một bộ hướng dẫn, chẳng hạn như thông tin về cách thiết lập máy chủ, những điều kiện tiên quyết cần cài đặt trên máy chủ và cách định cấu hình các phần phụ thuộc, ... Vì nhóm Vận hành không phát triển ứng dụng nên họ gặp nhiều khó khăn trong việc thiết lập nó. Khi gặp sự cố, họ sẽ làm việc với các nhà phát triển để giải quyết vấn đề đó. Với Docker, phần lớn công việc liên quan đến việc thiết lập cơ sở hạ tầng hiện nằm trong tay các nhà phát triển dưới dạng tệp Docker. Hướng dẫn mà các nhà phát triển đã xây dựng trước đây để thiết lập cơ sở hạ tầng giờ đây có thể dễ dàng tập hợp vào Dockerfile để tạo image cho ứng dụng. Image này hiện có thể chạy trên bất kỳ nền tảng container nào và được đảm bảo chạy theo cùng một cách ở mọi nơi. Vì vậy, nhóm Vận hành giờ đây có thể chỉ cần sử dụng image để triển khai ứng dụng. Vì image đã hoạt động khi nhà phát triển xây dựng nó và các hoạt động không sửa đổi nó, nên nó tiếp tục hoạt động như cũ khi được triển khai trong sản xuất.

2.1.6 Container Orchestration

Container Orchestration là một công cụ giúp quản lý, triển khai và điều phối hoạt động các container trong một môi trường phân tán. Container Orchestration được sử dụng để quản lý và điều phối các container trong một hệ thống lớn, bao gồm các tính

năng như tự động triển khai, mở rộng, cân bằng tải, và phục hồi lỗi, đảm bảo chúng hoạt động một cách hiệu quả nhất.

Container Orchestration hoạt động như sau. Công cụ này sử dụng các file cấu hình viết bằng ngôn ngữ YAML hoặc JSON để mô tả cấu hình của một ứng dụng. Các cấu hình này bao gồm nơi lưu trữ các Docker image, cách thiết lập mạng giữa các container, cách mount volume lưu trữ và nơi lưu trữ log cho các container đó. Nhóm phát triển viết và kiểm soát phiên bản của các tệp cấu hình này để có thể triển khai các ứng dụng giống nhau trên các môi trường thử nghiệm và phát triển khác nhau trước khi triển khai chúng vào các cụm trong môi trường product. Các container được triển khai trên các máy chủ, thường là trong các Replicated groups. Sau đó, Container Orchestration sẽ lên lịch triển khai và tìm kiếm một máy chủ phù hợp để đặt container, dựa trên một số ràng buộc được định nghĩa trong tệp cấu hình, chẳng hạn như tính khả dụng của CPU hoặc bộ nhớ. Các container có thể được đặt theo labels, metadata hoặc theo mức độ phù hợp của chúng so với các máy chủ khác. Sau khi container được thiết lập và chạy trên máy chủ, Container Orchestration phải quản lý vòng đời của container theo các thông số kỹ thuật được định nghĩa trong tệp định nghĩa của container (ví dụ như Dockerfile).

Công cụ điều phối container cho phép các nhà phát triển và quản trị viên hệ thống quản lý và triển khai các ứng dụng phức tạp trên nhiều máy chủ và môi trường khác nhau. Nó có khả năng tự động mở rộng số lượng container để đáp ứng nhu cầu tải và đảm bảo tính sẵn sàng cao của ứng dụng. Để đảm bảo các tài nguyên được sử dụng hiệu quả và giảm thiểu thời gian chết của ứng dụng, Container Orchestration cung cấp các công cụ để cân bằng tải giữa các container và node khác nhau. Nó cũng cung cấp các công cụ để quản lý tài nguyên của các container và node, giúp người dùng có thể dễ dàng điều phối các tài nguyên đến các container. Ngoài ra, Container Orchestration còn hỗ trợ tự động phát hiện và phục hồi lỗi, giúp tăng tính sẵn sàng và độ tin cậy của ứng dụng. Một số công nghệ điều phối container phổ biến như: Kubernetes, Docker Swarm, Mesos.

Docker Swarm là một nhóm các máy chạy Docker và tập hợp lại với nhau thành một cluster. Không như docker, sau khi các máy này tập hợp vào Swarm, mọi câu lệnh Docker sẽ được thực thi trên Swarm manager. Các máy tham gia vào swarm được gọi là worker node. Các node này chỉ có khả năng cung cấp khả năng hoạt động chứ không có quyền quản lý các node khác. Docker Swarm có khả năng khởi chạy các container

trên nhiều máy (cluster - máy ảo hoặc máy vật lý) hoặc trên một máy duy nhất (standalone). Một số tính năng của Docker Swarm bao gồm khả năng tự động phát hiện và phục hồi lỗi, khả năng quản lý các máy chủ và container trên một cụm, khả năng mở rộng dễ dàng và khả năng tùy chỉnh các cấu hình để đáp ứng các yêu cầu của ứng dụng. Docker Swarm cũng hỗ trợ tính năng bảo mật cao, cho phép người dùng thiết lập các chính sách bảo mật và chứng thực cho các container và máy chủ Docker trong swarm. Đây là một công nghệ đơn giản và dễ sử dụng, tuy nhiên nó lại thiếu một số tính năng tự động thay đổi quy mô nâng cao cần có cho các ứng dụng phức tạp.

Apache Mesos là một hệ thống quản lý tài nguyên mã nguồn mở được thiết kế để quản lý các tài nguyên máy tính trong một hệ thống phân tán. Apache Mesos được phát triển bởi các nhà nghiên cứu của Đại học California, Berkeley và được phát hành dưới giấy phép Apache. Mesos cung cấp một giao diện API để quản lý các tài nguyên máy tính và cho phép các ứng dụng sử dụng các tài nguyên đó một cách hiệu quả và linh hoạt. Nó cũng hỗ trợ nhiều khung ứng dụng (framework) như Apache Hadoop, Apache Spark và Marathon. Mesos được thiết kế để chạy trên các cụm máy tính lớn, có thể có hàng nghìn máy chủ và hàng triệu container. Nó cung cấp tính linh hoạt cao và khả năng mở rộng để quản lý các ứng dụng phức tạp và lớn trong các môi trường phân tán. Mesos cũng cung cấp tính năng hiệu suất cao và độ tin cậy để đảm bảo rằng các ứng dụng chạy trơn tru và liên tục. Tuy nhiên Mesos lại khá khó để thiết lập và khởi chạy.

Kubernetes là một hệ thống quản lý container mã nguồn mở được phát triển bởi Google. Nó được thiết kế để tự động triển khai, mở rộng và quản lý các ứng dụng container trên một cụm máy chủ. Kubernetes cung cấp một lớp trừu tượng để quản lý tài nguyên của các container và cung cấp các công cụ để triển khai, quản lý, cân bằng tải và phục hồi lỗi các ứng dụng container. Kubernetes hoạt động bằng cách quản lý các pod, là một nhóm các container chia sẻ cùng một không gian tên mạng và tài nguyên. Nó có thể tự động phát hiện và phục hồi lỗi trong các pod và triển khai các phiên bản mới của các container một cách dễ dàng. Nó cũng cung cấp tính năng tự động mở rộng, giúp đáp ứng nhu cầu tải của các ứng dụng, và tính năng cân bằng tải để đảm bảo rằng các tài nguyên được sử dụng hiệu quả. Kubernetes hơi khó thiết lập và bắt đầu nhưng nó cung cấp rất nhiều tùy chọn để tùy chỉnh triển khai và hỗ trợ triển khai các kiến trúc phức tạp. Kubernetes hiện được hỗ trợ trên tất cả các nhà cung cấp dịch vụ đám mây

công cộng như GCP, Azure và AWS và dự án kubernetes là một trong những dự án được xếp hạng hàng đầu trên Github.

Kubernetes và Docker Swarm đều được thiết kế để quản lý các container trong khi Apache Mesos còn có thể quản lý nhiều loại tài nguyên khác nhau, bao gồm cả container, máy ảo và các ứng dụng truyền thống. Về tính năng, Kubernetes và Docker Swarm đều cung cấp các tính năng cơ bản như tự động triển khai, cân bằng tải và phục hồi lỗi. Còn Mesos có thêm tính năng quản lý tài nguyên đa dạng và hỗ trợ đa khung ứng dụng. Mesos có tính năng quản lý tài nguyên tốt hơn so với Kubernetes và Docker Swarm. Nó cung cấp một phương pháp quản lý tài nguyên trừu tượng, cho phép quản lý tài nguyên của các máy chủ trong cụm một cách hiệu quả và linh hoạt. Kubernetes và Mesos đều có khả năng mở rộng tốt hơn so với Docker Swarm. Kubernetes cung cấp tính năng tự động mở rộng, giúp đáp ứng nhu cầu tải của các ứng dụng, và Mesos cũng có khả năng mở rộng linh hoạt để quản lý các ứng dụng phức tạp và lớn trong các môi trường phân tán. Kubernetes và Docker Swarm đều có cộng đồng lớn với nhiều tài liệu và hỗ trợ. Mesos cũng có một cộng đồng lớn, nhưng không phổ biến như Kubernetes và Docker Swarm. Kubernetes, Docker Swarm và Apache Mesos đều là các hệ thống quản lý container được phát triển để quản lý các container trên các cụm máy chủ nhưng khác nhau ở một số đặc điểm và chức năng. Mỗi công cụ đều có những ưu nhược điểm riêng, cần cân nhắc dựa trên yêu cầu và đặc điểm của từng dự án phần mềm mà lựa chọn giải pháp phù hợp nhất. Trong phạm vi đề án này, Kubernetes sẽ là công cụ Container Orchestration được sử dụng và sẽ được trình bày chi tiết hơn về thành phần kiến trúc và nguyên lý hoạt động trong các phần sau.

2.2 Kubernetes

2.2.1 Tổng quan về Kubernetes

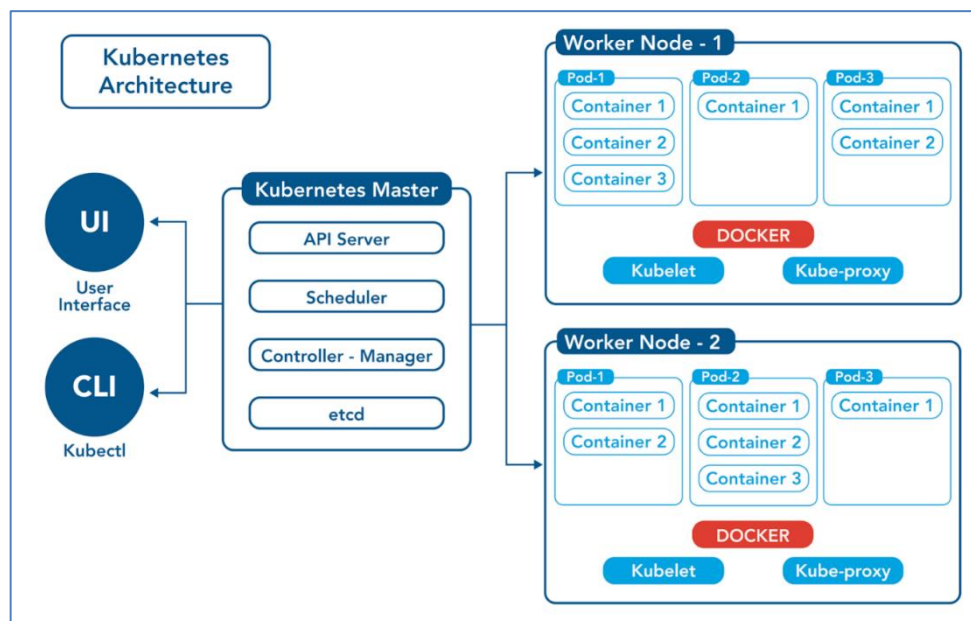
Kubernetes hay k8s là một nền tảng nguồn mở, di động, có thể mở rộng để quản lý các ứng dụng và dịch vụ đóng gói, tạo điều kiện cấu hình và tự động hóa việc triển khai ứng dụng. Kubernetes là một hệ sinh thái lớn và đang phát triển nhanh chóng. Các dịch vụ, hỗ trợ và công cụ có sẵn rộng rãi. Như được nhắc đến ở phần trên, Kubernetes là một công cụ container orchestration giúp tự động hoá việc quản lý, scaling và triển khai ứng dụng dưới dạng container. Nó được phát triển đầu tiên bởi Google và hiện nay được maintain bởi cộng đồng.

Kubernetes không phải là một hệ thống PaaS (Platform as a Service) bởi nó hoạt động ở tầng container chứ không phải ở tầng phần cứng. Nó cung cấp các một vài tính năng thường được áp dụng cho hệ thống PaaS: triển khai, nhân rộng, cân bằng tải, ghi nhật ký và giám sát.

Kubernetes có nhiều lợi thế khác nhau của việc điều phối container. Đầu tiên, nó giúp ứng dụng có tính khả dụng cao, lỗi phần cứng không làm ngừng hoạt động vì có nhiều phiên bản của ứng dụng chạy trên các node khác nhau. Thứ hai, lưu lượng người dùng được cân bằng tải trên các vùng chứa khác nhau. Khi nhu cầu tăng lên, nó triển khai nhiều phiên bản của ứng dụng một cách liền mạch. Thứ ba, khi hết tài nguyên phần cứng, việc tăng hoặc giảm số lượng các node được thực hiện mà không cần gỡ ứng dụng xuống. Bên cạnh đó còn nhiều lợi thế khác như lên lịch để chạy các container, tự phục hồi,...

2.2.2 Kiến trúc của Kubernetes

Thành phần cơ bản của một Kubernetes Cluster chính là nodes. Node là một máy vật lý hoặc máy ảo, trên đó kubernetes đã được cài đặt. Một node là một worker machine và đây là các container sẽ được đưa ra bởi Kubernetes. Kubernetes Cluster bao gồm Master Nodes và Worker Nodes. Kiến trúc và các thành phần của Kubernetes được thể hiện rõ qua Hình



Hình 2.6 Kiến trúc của Kubernetes [2]

Master

Master đóng vai trò là mặt phẳng điều khiển, chịu trách nhiệm quản lý toàn bộ hoạt động trên cụm. Master gồm các thành phần là: Kubernetes API Server, Scheduler, Controller Manager và etcd.

Trong đó, Kubernetes API Server được xem là một hub trung tâm, có nhiệm vụ kết nối các thành phần khác. Nó giao tiếp với kubectl thông qua HTTP REST API, giao tiếp với các thành phần khác trong Master sử dụng gRPC và với các thành phần trong cụm bằng TLS. Api-server chịu trách nhiệm hiển thị API endpoint của cụm và xử lý tất cả các yêu cầu API. Không những thế, nó còn phụ trách điều phối tất cả các quy trình giữa Master node và các worker node.

Thành phần tiếp theo là bộ lập lịch Scheduler, đây là thành phần chịu trách nhiệm lên lịch cho các pod trên Worker node. Khi triển khai một pod, các yêu cầu về pod như CPU, memory, tolerations, persistent volumes (PV),... sẽ được chỉ định. Nhiệm vụ chính của trình lập lịch chính là xác định các yêu cầu tạo và chọn node tốt nhất cho một nhóm pod để đáp ứng các yêu cầu.

Controller Manager thực hiện quản lý các Worker node. Bộ điều khiển sẽ chạy các vòng điều kiện vô hạn, liên tục theo dõi trạng thái thực tế và trạng thái mong muốn của các đối tượng. Khi nó phát hiện điểm khác nhau giữa hai trạng thái này nó sẽ cố gắng di chuyển trạng thái cụm về trạng thái mong muốn. Trình quản lý bộ điều khiển là một thành phần quản lý tất cả các bộ điều khiển Kubernetes. Các tài nguyên, đối tượng của Kubernetes như pod, namespace, job, replicaset được quản lý bởi các bộ điều khiển tương ứng. Ngoài ra, bộ lập lịch kube cũng là một bộ điều khiển được quản lý bởi trình quản lý bộ điều khiển Controller Manager.

Và cuối cùng là etcd, là một hệ cơ sở dữ liệu phân tán hỗ trợ hiệu quả cho hoạt động của Kubernetes và được xem như là bộ não của Cluster. Etcd là một kho lưu trữ phân tán key-value, mã nguồn mở và có tính nhất quán mạnh mẽ. Tính nhất quán mạnh mẽ được thể hiện ở chỗ, khi một node thực hiện một bản cập nhật thì ngay lập tức bản cập nhật này sẽ được thực hiện với tất cả các node trong cụm. Etcd lưu trữ tất cả các cấu hình, trạng thái và siêu dữ liệu của các Kubernetes Objects. Kubernetes api-server sử dụng chức năng theo dõi của etcd để theo dõi sự thay đổi trạng thái của một đối tượng.

Worker node

Các pod chứa các container sẽ được chạy trên Worker node dưới sự điều khiển của Master. Trên Worker node có các thành phần bao gồm container runtime, kubelet và Kubernetes Service Proxy.

Đầu tiên là về container runtime, cũng giống như Java Runtime – phần mềm cần thiết để chạy các chương trình java trên máy chủ, container runtime giúp chạy các ứng dụng dưới dạng container. Nó chạy trên tất cả các node trong cụm, chịu trách nhiệm lấy image từ registry, chạy container và phân bổ tài nguyên cho chúng. Kubernetes hỗ trợ nhiều container runtime tuân thủ Container Runtime Interface (CRI) như CRI-O, Docker Engine, containerd, ...

Thành phần tiếp theo là kubelet. Kubelet là thành phần để worker node giao tiếp với Kubernetes API Server, và cũng tham gia quản lý các container. Kubelet chạy dưới dạng daemon và được quản lý bởi systemd. Nó chịu trách nhiệm đăng ký worker node với Kubernetes API Server và làm việc với podSpec. PodSpec sẽ xác định các container chạy bên trong pod, tài nguyên của chúng (CPU, memory), các thiết lập khác như biến môi trường, volumes, và nhân và tạo các container. Kubelet sử dụng plugin CNI được định cấu hình trong cụm để phân bổ địa chỉ IP của pod.

Trong Worker node, Kubernetes Service Proxy là thành phần đảm nhiệm việc phân tải giữa các ứng dụng. Đây là một thành phần proxy để triển khai Kubernetes Service cho các pod. Nó chạy trên tất cả các node dưới dạng daemonset.

2.2.3 Kubernetes Object

Kubernetes Object được sử dụng để thể hiện trạng thái của cụm. Từ chúng có thể biết được ứng dụng nào đang chạy trên node nào, các tài nguyên có sẵn, các thiết lập của ứng dụng,... Có thể tác động tới các Object thông qua Kubernetes API bằng cách thực hiện lệnh gọi API trực tiếp hoặc sử dụng kubectl. Có nhiều loại Object khác nhau, các loại phổ biến nhất có thể kể đến như: pod, deployment, replicaset, service, ...

Trong đó, pod là đơn vị cơ bản nhất trong Kubernetes, là một nhóm gồm một hoặc nhiều container chia sẻ cùng một không gian lưu trữ và tài nguyên mạng. Pod được sử dụng để triển khai các ứng dụng và các tiến trình nằm trên Worker Node.

ReplicaSet giúp quản lý các pod, đảm bảo một số lượng nhất định bản sao của một pod. Nó kiểm tra và đảm bảo rằng số lượng Pod đang chạy trên cụm luôn bằng với số lượng được định nghĩa trong ReplicaSet. Khi có thay đổi về số lượng Pod hoặc Pod bị lỗi, ReplicaSet sẽ tự động tạo hoặc xóa Pod để đảm bảo rằng số lượng được định nghĩa được duy trì.

Deployment là một đối tượng để quản lý một phiên bản của ứng dụng và đảm bảo rằng phiên bản đó luôn chạy trên cụm. Khi có yêu cầu, để cập nhật phiên bản ứng dụng, Deployment sẽ tạo ra một ReplicaSet mới với phiên bản mới và tắt ReplicaSet cũ. Sau đó, nó đảm bảo rằng ReplicaSet mới được triển khai và đang chạy trên cụm. Deployment cũng cung cấp các tính năng như xác nhận và phục hồi lỗi để đảm bảo rằng phiên bản ứng dụng mới được triển khai một cách an toàn và hiệu quả.

Còn service là một Kubernetes object cho phép tạo điểm truy cập đến các pod nằm trong cùng một replicaset hoặc deployment. Khi một Service được tạo ra, Kubernetes sẽ tạo ra một địa chỉ IP duy nhất cho Service đó và cung cấp một cách để các ứng dụng bên ngoài có thể truy cập vào các Pod đang chạy trong Service đó. Có 4 loại service: ClusterIP là điểm truy cập chỉ cho phép truy cập từ trong cluster, Nodeport giúp tạo điểm truy cập tại một port trên tất cả các node worker (default port range 30000-32768), Loadbalancer tự động cấu hình loadbalancer trỏ đến worker node, External Name trỏ đến một DNS nằm ngoài cluster. Dữ liệu của pod được lưu trên Volume và chỉ là lưu trữ tạm thời.

Volume là một đối tượng được sử dụng để quản lý và cung cấp tài nguyên lưu trữ cho các container trong các Pod. Volume cung cấp một cách để lưu trữ dữ liệu trong các Pod và đảm bảo rằng dữ liệu đó không bị mất khi các container bị xóa hoặc khởi động lại.

ConfigMap là một API object để lưu trữ các dữ liệu không bí mật trong ứng dụng dưới dạng key-value. Các dữ liệu này có thể là các biến môi trường hoặc tệp cấu hình. ConfigMap cung cấp một cách để lưu trữ các cấu hình và cho phép chúng được sử dụng trong các Pod và Containers trên cụm.

Namespace giúp phân vùng các tài nguyên thành các nhóm nhằm dễ dàng trong quản lý và nhận dạng. Namespace cho phép tách biệt các dự án, môi trường... Có thể giới hạn quyền truy cập của người dùng trong một namespace cụ thể sử dụng RBAC (Role Based

Access Control). Các Namespace đặc biệt bao gồm: default - Namespace mặc định, kube-system chứa tài nguyên hệ thống quan trọng, kube-public chứa tài nguyên public có thể truy cập từ toàn bộ cụm và kube-node-lease.

2.2.4 Kubernetes Addon

Ngoài các thành phần chính, Kubernetes còn có thêm các thành phần phụ trợ được lựa chọn dựa trên mục đích sử dụng và yêu cầu của dự án. Các thành phần addon phổ biến có thể kể đến như CoreDNS, CNI (Container Network Interface), CSI (Container Storage Interface),...

CoreDNS hoạt động như một DNS server trong cụm kubernetes. Nó cung cấp tính năng phân giải tên miền cho các service, pod và có thể cấu hình được. CoreDNS được sử dụng để khám phá các service dựa trên DNS.

CNI (Container Network Interface) cung cấp các giải pháp định tuyến mạng cho các container trong Kubernetes. Các giải pháp dựa trên CNI được gọi là CNI-Plugins. CNI-Plugins hỗ trợ kết nối mạng giữa các pod trên cùng một node hoặc trên các node khác nhau. Ngoài ra, nó còn có chức năng cách ly và bảo mật pod network thông qua Network Policies nhằm kiểm soát luồng lưu lượng giữa các pod và các namespace. Một số CNI-Plugins phổ biến gồm có: Amazon VPC CNI, Calico, Cilium, Flannel, WeaveNet, Multus,...

CSI (Container Storage Interface): là chức năng cho phép quản lý lưu trữ volume. Thường được cung cấp bởi các bên thứ ba như: AWS EBS CSI, GCE persistent disk CSI, Azure Disks CSI, Cinder CSI, Ceph CSI, Longhorn...

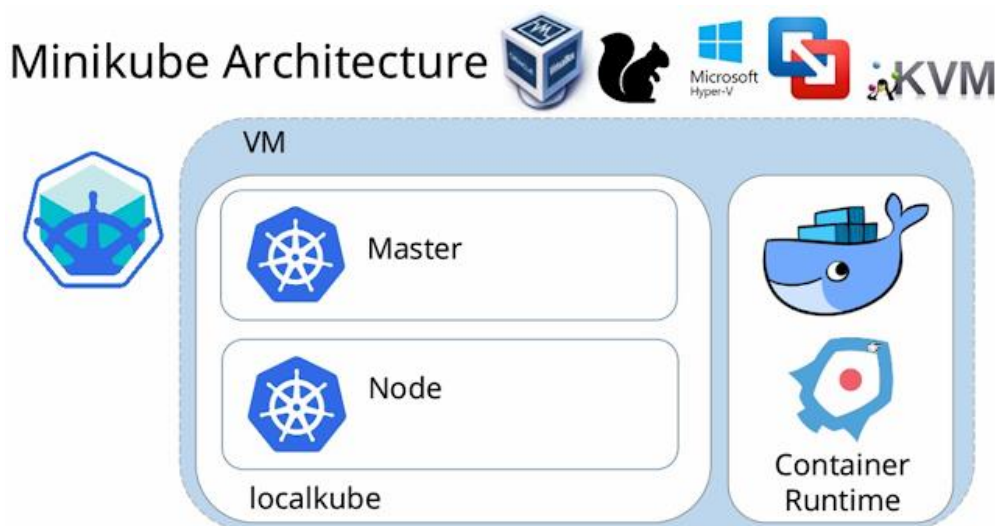
2.2.5 Minikube

Minikube là một công cụ giúp người dùng triển khai Kubernetes trên máy tính cá nhân để phát triển và kiểm thử các ứng dụng Kubernetes một cách dễ dàng. Minikube cung cấp một môi trường Kubernetes đơn giản, độc lập và có khả năng tái tạo để giúp người dùng tập trung vào việc phát triển ứng dụng thay vì quản lý hạ tầng.

Minikube sử dụng công nghệ ảo hóa để tạo ra một môi trường Kubernetes trên máy tính cá nhân của người dùng. Người dùng có thể chọn sử dụng các công nghệ ảo hóa khác nhau như VirtualBox, VMware, Docker, Hyperkit, hoặc KVM để tạo ra một môi trường Kubernetes trên máy tính cá nhân của mình. Việc thiết lập một cụm Kubernetes

sử dụng kubeadm thường rất khó khăn, nó cần nhiều bước cài đặt và xảy ra nhiều lỗi. Minikube khiến cho việc cài đặt Kubernetes trở nên vô cùng dễ dàng chỉ với vài thao tác mà không tốn quá nhiều tài nguyên. Phù hợp để phát triển và kiểm thử ứng dụng chạy trên môi trường Kubernetes. Minikube cung cấp các tính năng như tạo và quản lý một môi trường Kubernetes đơn giản, triển khai ứng dụng trên môi trường Kubernetes của mình và cung cấp các công cụ để kiểm thử ứng dụng của mình. Ngoài ra, Minikube cũng tích hợp các công cụ như kubectl, dashboard và các plugin để người dùng có thể quản lý và tương tác với môi trường Kubernetes của mình một cách dễ dàng. Minikube được phát triển bởi cộng đồng Kubernetes và là một phần của dự án Kubernetes.

Nhìn chung, đây là một công cụ hữu ích và tiện lợi để triển khai và kiểm thử các ứng dụng trên một môi trường Kubernetes. Và có thể được sử dụng trên các hệ điều hành khác nhau như Linux, macOS và Windows vô cùng linh hoạt.



Hình 2.7 Minikube architecture [10]

Tóm lại, nội dung chương là cái nhìn tổng quan về ảo hóa và công nghệ container. Em đã cố gắng để đưa ra những góc nhìn trực quan nhất về những khái niệm có phần trừu tượng này. Nội dung chương đã đi qua khái niệm container, tổng quan về Docker, các công cụ điều phối container trong đó có Kubernetes. Việc triển khai và thực thi một hệ thống dựa trên Docker và Kubernetes sẽ được thực hiện trong chương tiếp theo. Từ đó sẽ có những hình dung rõ hơn về những công nghệ nổi bật này.

CHƯƠNG 3. TRIỂN KHAI ỨNG DỤNG MICROSERVICE

Nội dung chương này sẽ thực hiện trình bày quá trình triển khai một ứng dụng microservice open-sources sử dụng các công nghệ phổ biến như Docker, Kubernetes. Các công việc thực hiện trong chương này bao gồm tìm hiểu về kiến trúc của ứng dụng được triển khai, container hóa các service trong ứng dụng, lên phương án triển khai các container chạy các service trên cụm Kubernetes.

3.1 Nghiên cứu kiến trúc ứng dụng microservice được triển khai

Ứng dụng là một cửa hàng bán hàng trực tuyến nhằm mục đích thử nghiệm kiến trúc microservice và các giải pháp công nghệ như Docker và Kubernetes. Ứng dụng được xây dựng bằng Spring Boot, Go kit và Node.js và được đóng gói trong các thùng chứa Docker (Docker container). Các ngôn ngữ lập trình được sử dụng là Java và Golang. Ứng dụng được phát triển với mục đích mô phỏng các phương pháp hay nhất về microservice, thể hiện tính đa nền tảng khi ứng dụng có thể được triển khai với tất cả các bộ điều phối vùng chứa và cung cấp ứng dụng có thể kiểm tra thực tế cho các nền tảng điều phối đó. Chi tiết về các framework và ngôn ngữ lập trình được trình bày ngay dưới đây.

Spring Boot

Spring Boot là một framework phát triển ứng dụng Java, được xây dựng trên nền tảng Spring Framework. Nó cung cấp cho người dùng một cách nhanh chóng và dễ dàng để phát triển các ứng dụng Java, bao gồm các ứng dụng web, microservices và các ứng dụng đám mây. Spring Boot giúp giảm thiểu việc cấu hình và cung cấp nhiều tính năng như phát hiện tự động của các thành phần ứng dụng, tạo và quản lý các kết nối cơ sở dữ liệu, và cung cấp các cơ chế xử lý lỗi và ghi nhật ký.

Một số tính năng của Spring Boot bao gồm:

Cấu hình tự động: Spring Boot sử dụng các giá trị mặc định và các cấu hình để tự động cấu hình một số thành phần của ứng dụng, giúp giảm thiểu việc phải cấu hình thủ công.

- Tích hợp nhiều thành phần: Spring Boot tích hợp nhiều thành phần của Spring Framework để cung cấp cho người dùng một cách tiếp cận dễ dàng để phát triển và triển khai các ứng dụng.
- Tích hợp với các công nghệ khác: Spring Boot cung cấp các tính năng tích hợp với các công nghệ khác như Thymeleaf, Hibernate, và Jackson, giúp giảm thiểu việc phải tìm hiểu và tích hợp các công nghệ này vào ứng dụng.
- Quản lý phụ thuộc: Spring Boot sử dụng Maven hoặc Gradle để quản lý các phụ thuộc và cung cấp các công cụ để giải quyết các xung đột phụ thuộc.
- Hỗ trợ đám mây: Spring Boot tích hợp các tính năng để phát triển các ứng dụng đám mây, bao gồm tích hợp với các dịch vụ đám mây như Netflix Eureka và Spring Cloud Config.

Trong tổng quát, Spring Boot là một framework phát triển ứng dụng Java mạnh mẽ, cung cấp nhiều tính năng tiện ích và làm giảm thiểu việc phải cấu hình và tích hợp các thành phần của ứng dụng.

Go kit

Go kit là một bộ công cụ (toolkit) phát triển ứng dụng microservices bằng ngôn ngữ lập trình Go. Nó cung cấp cho người dùng các công cụ để xây dựng các dịch vụ microservices có khả năng mở rộng, linh hoạt và dễ dàng bảo trì. Go kit được thiết kế để giúp tăng tốc độ phát triển và giảm độ phức tạp của việc xây dựng và triển khai các dịch vụ microservices.

Go kit bao gồm các tính năng như:

- Hỗ trợ chuẩn: Go kit tuân theo các chuẩn phổ biến như HTTP, JSON và gRPC để tăng tính tương thích và giảm độ phức tạp trong việc tích hợp với các công nghệ khác.
- Khả năng mở rộng: Go kit hỗ trợ các mô hình phát triển dịch vụ microservices khác nhau và có khả năng mở rộng để đáp ứng nhu cầu tăng trưởng của ứng dụng.
- Cấu trúc tách biệt: Go kit hỗ trợ cấu trúc tách biệt (separation of concerns) giữa logic ứng dụng và các chi tiết triển khai, giúp tăng tính bảo trì và dễ dàng mở rộng ứng dụng.
- Quản lý lỗi: Go kit cung cấp các công cụ để quản lý lỗi trong các dịch vụ microservices, bao gồm xử lý lỗi, ghi nhật ký và giám sát.

- Kiểm tra: Go kit cung cấp các công cụ để kiểm tra và đảm bảo tính đúng đắn của các dịch vụ microservices.
- Tích hợp dịch vụ: Go kit hỗ trợ tích hợp với các dịch vụ bên ngoài như các dịch vụ đám mây và các dịch vụ bảo mật.

Go kit là một bộ công cụ phát triển ứng dụng microservices bằng Go, cung cấp cho người dùng các công cụ để xây dựng các dịch vụ microservices có khả năng mở rộng, linh hoạt và dễ dàng bảo trì. Nó được thiết kế để giảm độ phức tạp và tăng tốc độ phát triển ứng dụng.

Node.js

Node.js là một nền tảng phát triển ứng dụng máy chủ được xây dựng trên nền tảng JavaScript. Nó cho phép các lập trình viên sử dụng JavaScript để xây dựng các ứng dụng backend, chẳng hạn như các ứng dụng web và dịch vụ web. Node.js sử dụng mô hình sự kiện (event-driven) và không chặn (non-blocking) để xử lý các yêu cầu, cho phép nó xử lý hàng ngàn kết nối cùng một lúc mà không làm giảm hiệu suất. Điều này làm cho Node.js trở thành một lựa chọn phổ biến cho các ứng dụng yêu cầu khả năng mở rộng cao như các ứng dụng trò chơi trực tuyến và các ứng dụng đa người dùng.

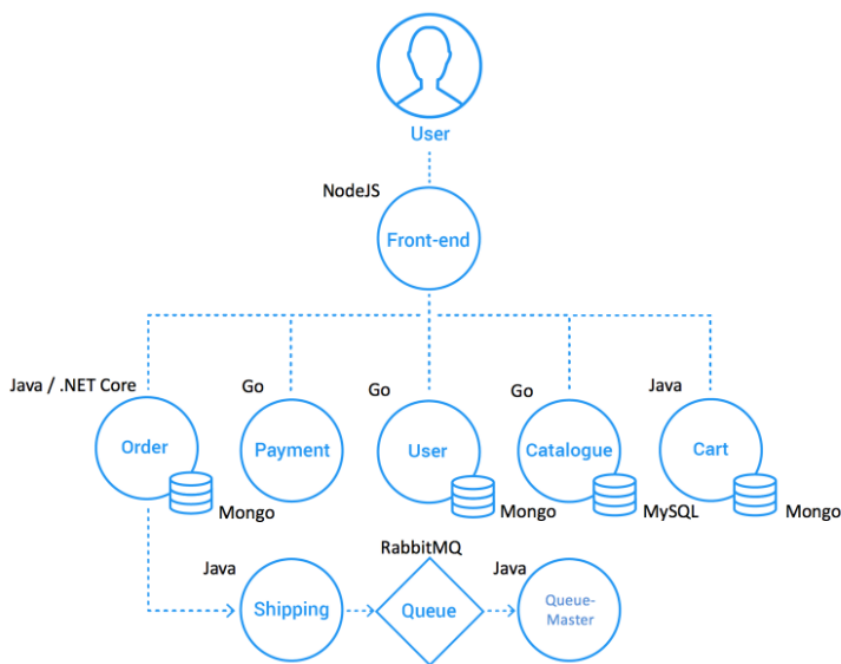
Node.js có một số tính năng quan trọng như:

- Có tính năng xử lý không chặn (non-blocking) để xử lý các yêu cầu mà không ảnh hưởng đến hiệu suất của ứng dụng.
- Sử dụng mô hình sự kiện (event-driven) và mô hình bất đồng bộ (asynchronous) để xử lý các yêu cầu.
- Cho phép lập trình viên sử dụng JavaScript cả phía client và server, giúp tăng tính liên tục và đơn giản hoá quá trình phát triển.
- Có một cộng đồng lớn và tích cực phát triển các thư viện và module để giải quyết các vấn đề và thách thức khác nhau khi phát triển ứng dụng.
- Dễ dàng triển khai và vận hành trên các nền tảng khác nhau.

Một số ứng dụng phổ biến được xây dựng bằng Node.js bao gồm Paypal, Netflix, Uber và LinkedIn. Tóm lại, Node.js là một nền tảng phát triển ứng dụng máy chủ được xây dựng trên nền tảng JavaScript, sử dụng mô hình sự kiện và không chặn để xử lý các yêu cầu. Nó là một lựa chọn phổ biến cho các ứng dụng yêu cầu khả năng mở rộng cao và có cộng đồng lớn và tích cực phát triển các thư viện và module.

3.1.1 Kiến trúc

Các microservice được xác định dựa trên các chức năng cơ bản của một trang Web thương mại điện tử. Ứng dụng bao gồm các microservice: Front-end, Order, Payment, User, Catalogue, Cart, Shipping, Queue-Master. Các microservice sử dụng DNS để tìm kiếm các service khác. Chúng giao tiếp bằng REST qua HTTP nhằm đơn hóa việc phát triển và thử nghiệm. Các service Order, User, Catalogue và Cart đều có riêng các cơ sở dữ liệu bao gồm cả SQL và NoSQL. Có thể thấy các dịch vụ được phát triển từ các ngôn ngữ lập trình và các framework khác nhau. Điều này làm nổi khả năng phát triển một cách độc lập giữa các dịch vụ của kiến trúc Microservice. Ứng dụng có thể được phát triển bởi một nhóm các nhà phát triển, các dịch vụ khác nhau trong hệ thống có thể được phát triển bằng nhiều ngôn ngữ lập trình khác nhau, cho phép các nhà phát triển sử dụng ngôn ngữ lập trình mà họ có chuyên môn và kinh nghiệm để phát triển dịch vụ của mình. Đây là một ứng dụng microservice cơ bản nhằm mô phỏng kiến trúc microservice, mang đến cái nhìn trực quan hơn về kiến trúc này sau những tìm hiểu và nghiên cứu nó về mặt lý thuyết.



Hình 3.1 Kiến trúc ứng dụng [15]

3.1.2 Front-end

Front-end (giao diện người dùng) là phần của một ứng dụng web đóng vai trò giao tiếp với người dùng thông qua các trình duyệt web. Nó bao gồm các thành phần như trang HTML, CSS và JavaScript, cùng với các thư viện và framework để xây dựng các

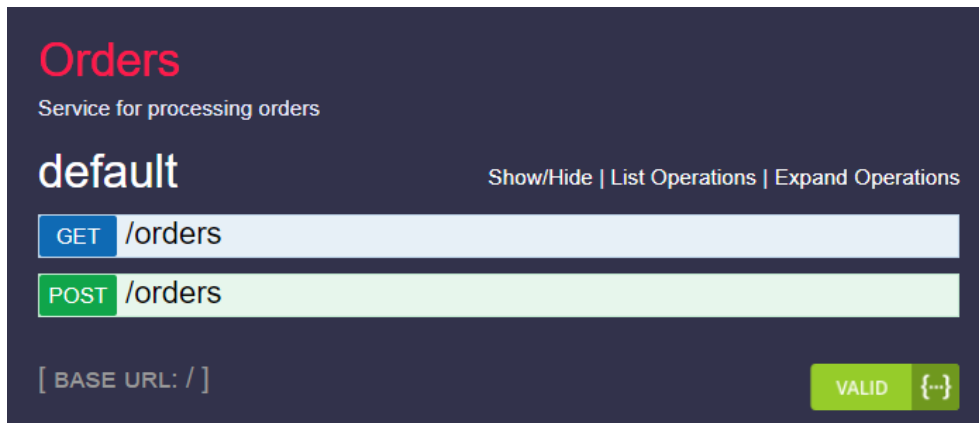
giao diện người dùng (UI) tương tác. HTML là ngôn ngữ đánh dấu siêu văn bản (hypertext markup language) được sử dụng để xác định cấu trúc của trang web. Ngôn ngữ định dạng CSS (cascading style sheets) được sử dụng để xác định kiểu dáng và bố cục của trang web. JavaScript là ngôn ngữ lập trình được sử dụng để tạo ra các tính năng tương tác trên trang web. Service này được phát triển dựa trên NodeJS.

Front-end service là dịch vụ chịu trách nhiệm xử lý tất cả các yêu cầu từ phía người dùng và hiển thị giao diện người dùng (UI) tương ứng. Front-end service được phát triển độc lập với các dịch vụ back-end khác, và sử dụng các API để truy cập dữ liệu và chức năng từ các dịch vụ back-end. Trong kiến trúc microservice, front-end service thường được thiết kế để chịu trách nhiệm xử lý các yêu cầu từ phía người dùng, như trang chủ, đăng nhập, đăng ký, xem chi tiết sản phẩm, thêm sản phẩm vào giỏ hàng, thanh toán, v.v. Front-end service cũng cung cấp giao diện người dùng cho người dùng tương tác với ứng dụng, và đảm bảo tính thân thiện và dễ sử dụng cho người dùng. Front-end service trong kiến trúc microservice có nhiều lợi ích, bao gồm tính độc lập và linh hoạt, tính tương thích, và tối ưu hóa hiệu suất.

3.1.3 Order service

Trong một ứng dụng mua sắm dựa trên kiến trúc microservice, order service là một trong các dịch vụ back-end quan trọng, chịu trách nhiệm xử lý các đơn hàng của khách hàng. Order service sử dụng các API để truy cập các dịch vụ khác, như payment service, product service, customer service, v.v. để xử lý các đơn hàng của khách hàng. Trong ứng dụng này, service gồm chức năng xử lý đơn hàng, kiểm tra sản phẩm và số lượng, cập nhật trạng thái đơn hàng. Order service nhận yêu cầu từ phía front-end service và xử lý các đơn hàng của khách hàng, bao gồm tạo đơn hàng mới, cập nhật thông tin đơn hàng. Nó sẽ gọi đến các service khác như user, cart, payment, shipping để thực hiện các chức năng của mình. Trước khi tạo đơn hàng, order service cần kiểm tra xem sản phẩm và số lượng có đủ để đáp ứng yêu cầu của khách hàng hay không. Sau khi khách hàng đặt hàng, order service gửi yêu cầu thanh toán đến payment service để xử lý thanh toán. Sau đó chuyển đến shipping để xử lý vận chuyển. Thông tin order được lưu vào cơ sở dữ liệu. Service sử dụng cơ sở dữ liệu là MongoDB để lưu trữ thông tin về đơn hàng và trạng thái đơn hàng. MongoDB là một hệ quản trị cơ sở dữ liệu phi quan hệ (NoSQL) mã nguồn mở, được phát triển bởi MongoDB Inc. MongoDB sử dụng định dạng JSON

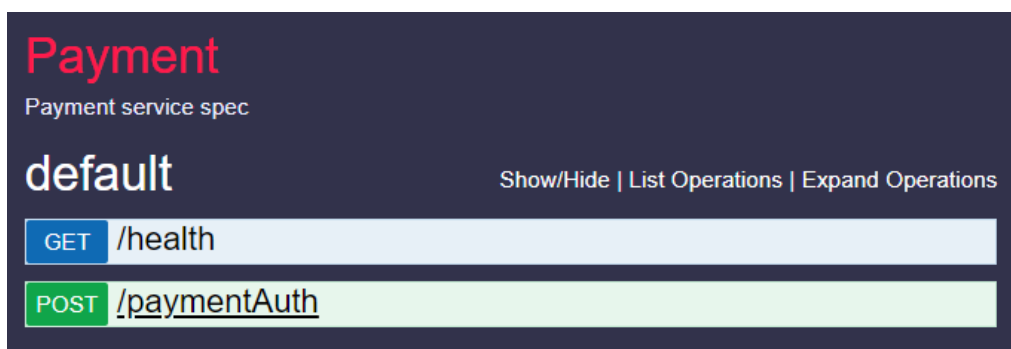
để lưu trữ dữ liệu và cung cấp các tính năng như index, truy vấn, thống kê,... MongoDB được sử dụng rộng rãi trong các ứng dụng web, big data và IoT.



Hình 3.2 Các API của Orders service [14]

3.1.4 Payment service

Payment service chịu trách nhiệm xử lý thanh toán của khách hàng. Nó sử dụng các API để truy cập các dịch vụ khác để xử lý thanh toán của khách hàng. Các chức năng của payment service bao gồm xác thực thanh toán, xử lý thanh toán, quản lý thông tin thanh toán và hỗ trợ các phương thức thanh toán khác nhau. Payment service được phát triển bằng Golang.



Hình 3.3 Các API của Payment service [14]

3.1.5 User service

Userservice chịu trách nhiệm quản lý thông tin của khách hàng. Được phát triển bằng Golang. Nó sử dụng cơ sở dữ liệu MongoDB để lưu trữ thông tin người dùng như: thông tin đăng nhập, thẻ, địa chỉ.

User	
Provide Customer login, register, retrieval, as well as card and address retrieval	
default Show/Hide List Operations Expand Operations	
GET	/login
POST	/register
GET	/customers
DELETE	/customers/{id}
GET	/customers/{id}
GET	/customers/{id}/cards
GET	/customers/{id}/addresses
GET	/cards
POST	/cards
DELETE	/cards/{id}
GET	/cards/{id}
GET	/addresses
POST	/addresses
DELETE	/addresses/{id}
GET	/addresses/{id}
[BASE URL: /] VALID { }	

Hình 3.4 Các API của User service [14]

3.1.6 Catalogue service

Catalogue service có nhiệm vụ quản lý thông tin về sản phẩm bao gồm tên, mô tả, ảnh,... Nó cung cấp các API để các dịch vụ khác trong hệ thống có thể truy cập với các thông tin này. Service được phát triển bằng ngôn ngữ lập trình Golang và sử dụng cơ sở dữ liệu quan hệ MySQL để lưu các thông tin về sản phẩm.

Catalogue resources	
default Show/Hide List Operations Expand Operations	
GET	/catalogue
GET	/catalogue/{id}
GET	/catalogue/size
GET	/tags
[BASE URL: /] VALID { }	

Hình 3.5 Các API của Catalogue service [14]

3.1.7 Các service khác

Cart service được phát triển bằng Java, nhằm quản lý thông tin thanh toán của khách hàng. Các thông tin này được lưu trong MongoDB dưới dạng key-value.

Shipping service cũng được phát triển bằng Java, nó thực hiện quản lý thông tin vận chuyển đơn hàng sau khi đơn hàng được xử lý tại dịch vụ order. Nó sử dụng message broker để gửi dữ liệu vận chuyển tới Queue-Master, đơn hàng sẽ được đưa vào hàng đợi để được xử lý vận chuyển. Hệ thống message broker trong ứng dụng này được cung cấp bởi RabbitMQ.

Message broker là một phần mềm trung gian được sử dụng để truyền thông tin giữa các ứng dụng khác nhau. Nó là một thành phần quan trọng trong kiến trúc phân tán, cho phép các ứng dụng giao tiếp và trao đổi thông tin một cách đáng tin cậy và hiệu quả. Message broker hoạt động bằng cách nhận thông tin từ một ứng dụng và gửi nó đến một hoặc nhiều ứng dụng khác. Message broker có thể được sử dụng để xử lý các nhiệm vụ như lưu trữ tin nhắn, gửi tin nhắn, đăng ký và phân phối các sự kiện, và nhiều hơn nữa. Một số message broker phổ biến được sử dụng trong các hệ thống phân tán hiện nay bao gồm Apache Kafka, RabbitMQ, ActiveMQ, Redis, và Amazon SQS. Mỗi message broker có các đặc tính và ưu điểm riêng, và được sử dụng tùy thuộc vào mục đích và yêu cầu của hệ thống. Sử dụng message broker giúp cho các ứng dụng có thể tách biệt và phân tán hơn, giảm thiểu sự phụ thuộc giữa các thành phần và tăng tính tin cậy của hệ thống. Nó cũng giúp tối ưu hóa hiệu suất và tăng khả năng mở rộng của hệ thống.

RabbitMQ là một hệ thống message broker mã nguồn mở, được sử dụng để trung gian trong việc truyền tải dữ liệu giữa các ứng dụng. Nó hỗ trợ nhiều giao thức truyền tải như AMQP (Advanced Message Queuing Protocol), MQTT (Message Queuing Telemetry Transport), STOMP (Simple Text Oriented Messaging Protocol) và HTTP (Hypertext Transfer Protocol). RabbitMQ được thiết kế để xử lý hàng triệu thông điệp mỗi giây, với khả năng mở rộng và độ tin cậy cao. Nó cung cấp các tính năng như hàng đợi (queue), trao đổi (exchange) và routing key để giúp các ứng dụng truyền tải và nhận thông điệp một cách hiệu quả. Một số lợi ích của RabbitMQ bao gồm: độ tin cậy, khả năng mở rộng, tính linh hoạt, tính đa nền tảng và tính tiêu thụ thấp. RabbitMQ được thiết kế để đảm bảo tính đáng tin cậy trong việc truyền tải thông điệp. Nó sử dụng các cơ chế như ACK (acknowledgment) và requeue (đưa lại vào hàng đợi) để đảm bảo rằng thông điệp được gửi và nhận một cách chính xác. Nó có thể mở rộng để xử lý hàng triệu

thông điệp mỗi giây bằng cách sử dụng các cụm RabbitMQ (RabbitMQ cluster) để phân tải công việc. RabbitMQ hỗ trợ nhiều giao thức truyền tải như AMQP, MQTT và STOMP, cho phép các ứng dụng sử dụng các giao thức phù hợp với nhu cầu của mình. Nhiều ngôn ngữ lập trình được hỗ trợ như Java, Python, Ruby, .NET, Go,... giúp các nhà phát triển có thể sử dụng ngôn ngữ lập trình mà họ ưa thích để phát triển ứng dụng của mình. RabbitMQ sử dụng kiến trúc hàng đợi (queue) để lưu trữ và truyền tải thông điệp, giúp giảm thiểu lượng tài nguyên tiêu thụ trong quá trình truyền tải. Tóm lại, đây là một hệ thống message broker mã nguồn mở, cung cấp các tính năng như hàng đợi, trao đổi và routing key để giúp các ứng dụng truyền tải và nhận thông điệp một cách hiệu quả.

3.2 Container hóa các service trong ứng dụng

Quá trình container hóa các service của ứng dụng được chia thành các bước:

- Dựng images: tạo các images để chạy container từ các file Dockerfile.
- Khởi chạy container: sử dụng docker-compose để deploy nhiều container cùng lúc.
- Debugging: kiểm tra các service xem có hoạt động chính xác hay không.
- Đẩy images lên Repository: khi mọi thứ hoạt động chính xác, upload images lên repository để hạn chế việc phải build lại images.

```
root@chau-gitlab-ce:~/microservices-demo/src# ll
total 52
drwxr-xr-x 11 root root 4096 Jul 29 16:18 ./
drwxr-xr-x 10 root root 4096 Jul 29 16:13 ../
drwxr-xr-x 10 root root 4096 Jul 29 16:14 carts/
drwxr-xr-x 11 root root 4096 Jul 29 16:15 catalogue/
-rw-r--r-- 1 root root 4382 Jul 29 16:06 docker-compose.yml
drwxr-xr-x 4 root root 4096 Jul 29 16:15 edge-router/
drwxr-xr-x 9 root root 4096 Jul 29 16:16 front-end/
drwxr-xr-x 10 root root 4096 Jul 29 16:17 orders/
drwxr-xr-x 10 root root 4096 Jul 29 16:17 payment/
drwxr-xr-x 8 root root 4096 Jul 29 16:17 queue-master/
drwxr-xr-x 9 root root 4096 Jul 29 16:18 shipping/
drwxr-xr-x 10 root root 4096 Jul 29 16:18 user/
```

Hình 3.6 Thư mục chứa source code và Dockerfile của từng service

Build

Thực hiện lệnh “docker build” để build images từ Dockerfile, có thể thêm nhiều tùy chọn cho câu lệnh này nhưng đơn giản nhất chỉ cần thực hiện “docker build -t myimages:tag path/to/Dockerfile .”, cờ -t để xác định tên images, tag của images nếu

không được chỉ định thì sẽ được đặt theo mặc định là tag latest và đường dẫn đến thư mục chứa Dockerfile, nếu giao diện dòng lệnh đang ở thư mục đó thì có thể bỏ luôn phần này.

Muốn build images của dịch vụ nào, thực hiện di chuyển đến thư mục chứa source code của dịch vụ được chỉ định. Ví dụ cần build images cho dịch vụ shipping.

```
cd shipping

docker build -t shipping -f docker/shipping/Dockerfile .
```

Thực hiện tương tự với các service còn lại. Ngoài ra cũng có thể build images với docker compose.

Deploy

Có thể chạy từng container từ images bằng lệnh “docker run”, tuy nhiên cần thiết lập mạng chung để các container có thể giao tiếp với nhau và các thiết đặt khác nữa. Việc chạy container một cách đơn lẻ bằng lệnh chỉ phù hợp với những ứng dụng được gói gọn trong một images. Còn đối với những ứng dụng phân tán gồm nhiều thành phần chạy đơn lẻ thì việc triển khai như vậy mất khá nhiều thời gian trong việc thiết lập chạy từng container một, chưa kể còn có những service cần được triển khai khi một hoặc một số các service khác đã được chạy trước đó. Docker-compose chính là giải pháp cho vấn đề này. Nó là một công cụ giúp định nghĩa và chạy các ứng dụng Docker gồm nhiều container. Nó cho phép định nghĩa các service, network và volume của ứng dụng trong một tệp YAML duy nhất, sau đó khởi động, dừng ứng dụng và xóa toàn bộ chỉ bằng một lệnh đơn. Khi cần thay đổi bất cứ thiết lập nào của các container, chỉ cần thay đổi trong file cấu hình, và chạy một lệnh đơn giản để triển khai lại toàn bộ ứng dụng. Docker Compose giúp dễ dàng thiết lập các môi trường phức tạp cho ứng dụng mà không cần bắt đầu và liên kết các container riêng lẻ bằng tay. Có thể chỉ định các mối quan hệ phụ thuộc giữa các dịch vụ, cấu hình thứ tự khởi động của chúng, mạng và các thuộc tính khác. Docker compose không được cài đặt kèm theo với ứng dụng Docker nên muốn sử dụng cần thực hiện cài đặt thêm. Lệnh chạy docker-compose sử dụng tệp cấu hình docker-compose.yml như sau:

```
docker-compose -f microservices-demo/deploy/docker-compose/docker-
compose.yml up -d
```

Trong đó, cờ -f chỉ định file cấu hình đi kèm với đường dẫn tới thư mục chứa nó, docker-compose up để vừa thực hiện build images và chạy images đó, cờ -d để container chạy trong nền.

Kiểm tra tính chính xác của các service:

Trong docker-compose file dùng để deploy các container chạy các service của ứng dụng, volumes được chỉ định ngoài việc có thể trao đổi dữ liệu từ host với container và ngược lại, nó còn chỉ định folder log/ dùng để lưu các logs của các container. Ngoài cách đọc từ các file trong folder logs, docker-compose cũng hỗ trợ người dùng một cách xem logs khác.

- Front-end service:

```
root@chau-gitlab-ce:~/microservices-demo/deploy/docker-compose# docker-compose logs front-end
Attaching to docker-compose_front-end_1
front-end_1      npm info It worked if it ends with ok
front-end_1      npm info using npm@2.15.11
front-end_1      npm info using node@v4.8.0
front-end_1      npm info prestart microservices-demo-front-end@0.0.1
front-end_1      npm info start microservices-demo-front-end@0.0.1
front-end_1      > microservices-demo-front-end@0.0.1 start /usr/src/app
front-end_1      > node server.js
front-end_1      Using local session manager
front-end_1      Warning: connect.session() MemoryStore is not
front-end_1      designed for a production environment, as it will leak
front-end_1      memory, and will not scale past a single process.
front-end_1      App now running in production mode on port 8079
front-end_1      GET /catalogue 200 72.262 ms - -
front-end_1      Received login request
front-end_1      {"user":{"firstName":"User","lastName":"Name","username":"user","id":"57a98d98e4b00679b4a830b2","_links":{"addresses":{"href":"http://user/customers/57a98d98e4b00679b4a830b2/addresses"},"cards":{"href":"http://user/customers/57a98d98e4b00679b4a830b2/cards"},"customer":{"href":"http://user/customers/57a98d98e4b00679b4a830b2"},"self":{"href":"http://user/customers/57a98d98e4b00679b4a830b2"}}}}
front-end_1      57a98d98e4b00679b4a830b2
front-end_1      Merging carts for customer id: 57a98d98e4b00679b4a830b2 and session id: VoBGRU6sXSWLbug-ngAOy8oki1U4BC8n
front-end_1      GET /catalogue 200 6.966 ms - -
front-end_1      Received login request
front-end_1      {"user":{"firstName":"User","lastName":"Name","username":"user","id":"57a98d98e4b00679b4a830b2","_links":{"addresses":{"href":"http://user/customers/57a98d98e4b00679b4a830b2/addresses"},"cards":{"href":"http://user/customers/57a98d98e4b00679b4a830b2/cards"},"customer":{"href":"http://user/customers/57a98d98e4b00679b4a830b2"},"self":{"href":"http://user/customers/57a98d98e4b00679b4a830b2"}}}}
front-end_1      57a98d98e4b00679b4a830b2
front-end_1      Merging carts for customer id: 57a98d98e4b00679b4a830b2 and session id: 18VwO117RH9M9B9VCYgmCOKL3xbmtW7N
front-end_1      Carts merged.
front-end_1      Sent cookies.
front-end_1      GET /login 200 281.055 ms - 13
front-end_1      Carts merged.
front-end_1      Sent cookies.
front-end_1      GET /login 200 412.732 ms - 13
front-end_1      Attempting to delete cart for user: 57a98d98e4b00679b4a830b2
front-end_1      Attempting to delete cart for user: 57a98d98e4b00679b4a830b2
front-end_1      User cart deleted with status: 202
front-end_1      DELETE /cart 202 16.218 ms - -
```

Hình 3.7 Log chạy front-end service container

- Edge-router service:

```
root@chau-gitlab-ce:~/microservices-demo/deploy/docker-compose# docker-compose logs edge-router
Attaching to docker-compose_edge-router_1
edge-router_1    time="2023-07-24T02:18:53Z" level=info msg="Traefik version v1.1.2 built on 2016-12-15 10:21:15AM"
edge-router_1    time="2023-07-24T02:18:53Z" level=info msg="Using TOML configuration file /etc/traefik/traefik.toml"
edge-router_1    time="2023-07-24T02:18:53Z" level=info msg="Preparing server http &{Network: Address:80 TLS:<nil> Redirect:<nil> Auth:<nil> Compress:false}"
edge-router_1    time="2023-07-24T02:18:53Z" level=info msg="Starting provider *provider.File {\"Watch\":true,\"Filename\":\"/etc/traefik/traefik.toml\",\"Constraints\":null}"
edge-router_1    time="2023-07-24T02:18:53Z" level=info msg="Starting provider *main.WebProvider {\"Address\":\":8080\",\"CertFile\":\"\",\"KeyFile\":\"\",\"ReadOnly\":false,\"Auth\":null}"
edge-router_1    time="2023-07-24T02:18:53Z" level=info msg="Starting server on :80"
edge-router_1    time="2023-07-24T02:18:53Z" level=info msg="Server configuration reloaded on :80"
edge-router_1    time="2023-07-24T02:18:54Z" level=warning msg="Error checking new version: Get https://update.traefik.io/repos/containous/traefik/releases: x509: certificate has expired or is not yet valid"
```

Hình 3.8 Log chạy Edge-router service container

- Orders service:

```

root@chuan-gtiab:~/cst-/microservices-demo/deploy/docker-compose# docker-compose logs orders
Attaching to docker-compose_orders_1
orders_1    | [ 2023-07-24 02:18:57.579 INFO [bootstrap]] 6 --- [ main] s.c.a.AnnotationConfigApplicationContext: Refreshing org.springframework.context.annotat
tion.AnnotationConfigApplicationContext@1521c1cf: startup date [Mon Jul 24 02:18:57 GMT 2023]; root of context hierarchy
orders_1    | [ 2023-07-24 02:18:59.426 INFO [bootstrap]] 16 --- [ main] trationDelegateBeanPostProcessorChecker: Bean 'configurationPropertiesRebinderAutoConfiguration' of type [class org.springframework.cloud.autoconfigure.ConfigurationPropertiesRebinderAutoConfiguration$$EnhancerBySpringG
LI1B$5b094F93] is not eligible for getting processed by all BeanPostProcessors (for example: not eligible for auto-proxying)
orders_1    | _____
orders_1    |  \   / ____ _
orders_1    |  /  / |  _ \| | | |
orders_1    | / __/| |_| | | | |
orders_1    |/_||___|___|_|_|_|
orders_1    |:: Spring Boot ::      (v1.4.4.RELEASE)
orders_1    |
orders_1    | [ 2023-07-24 02:19:00.401 INFO [orders_]] 6 --- [ default] m.weave.socks.orders.OrderApplication : No active profile set, falling back to default profiles: default
orders_1    | [ 2023-07-24 02:19:00.531 INFO [orders_]] 6 --- [ main] annotationConfigEmbeddedWebApplicationContext: Refreshing org.springframework.web.servlet.config.annotation.AnnotationConfigEmbeddedWebApplicationContext@8649d299: startup date [Mon Jul 24 02:19:00 GMT 2023]; parent: org.springframework.context.annotat
ionConfigApplicationContext@1521c1cf
orders_1    | [ 2023-07-24 02:19:01.494 INFO [orders_]] 6 --- [ main] o.s.b.f.s.DefaultListableBeanFactory: Overriding bean definition for bean 'man
agementServiceContext' with a different definition: replacing [Root bean: class [null]; scope=abstract=false; lazyInit=false; autowireMode=3; dependencyCheck=0;
autowired=true; primary=false; factoryBeanName=org.springframework.management.endpoint.annotation.ManagementContextConfiguration; factoryMethodName=managementServiceContext; initMethod=null; destroyMethod=None(inferred); defined in class path resource [/org/springframework/boot/actuate/autoconfigure/endpoi
ntWebMvcHypermediaManagementContextConfiguration.class] with [Root bean: class [null]; scope=abstract=false; lazyInit=false; autowireMode=3; dependencyCheck=0;
autowired=true; primary=false; factoryBeanName=org.springframework.management.endpoint.annotation.ManagementContextConfiguration; factoryMethodName=managementServiceContext; initMethod=null; destroyMethod=None(inferred); defined in class path resource [/org/springframework/boot/actuate/autoconfigure/endpoi
ntWebMvcHypermediaManagementContextConfiguration.class]]
orders_1    | [ 2023-07-24 02:19:06.239 WARN [orders_]] 6 --- [ main] o.s.c.a.ConfigurationClassPostProcessor: Cannot enhance @Configuration bean defin
ition 'refreshScope' since its singleton instance has been created too early. The typical cause is a non-static @Bean method with a BeanDefinitionRegistryPostProcessor
as target; consider declaring such methods as 'static'.
orders_1    | [ 2023-07-24 02:19:06.810 INFO [orders_]] 6 --- [ main] o.s.cloud.context.scope.GenericScope : BeanFactory id=25db8119-5df6-3e7e-8ac5-6fabdc42a9ea
orders_1    | [ 2023-07-24 02:19:06.957 INFO [orders_]] 6 --- [ main] trationDelegateBeanPostProcessorChecker: Bean 'org.springframework.cloud.sleuth.it
rument.async.AsyncDefaultAsyncConfigurersSupport' of type [class org.springframework.cloud.sleuth.instrument.async.AsyncDefaultAsyncConfigurersSupport$$EnhancerBySpringG
LI1B$52c2b0e0ff] is not eligible for getting processed by all BeanPostProcessors (for example: not eligible for auto-proxying)
orders_1    | [ 2023-07-24 02:19:08.290 INFO [orders_]] 6 --- [ main] trationDelegateBeanPostProcessorChecker: Bean 'org.springframework.cloud.autoconfigure.ConfigurationPropertiesRebinderAutoConfiguration' of type [class org.springframework.cloud.autoconfigure.ConfigurationPropertiesRebinderAutoConfiguration$$EnhancerBySpringG
LI1B$52c2b0e0ff] is not eligible for getting processed by all BeanPostProcessors (for example: not eligible for auto-proxying)

```

Hình 3.9 Log chạy Orders service container

```

root@chau-gitlab-ce:~/microservices-demo/deploy/docker-compose# docker-compose logs payment
Attaching to docker-compose_payment_1
payment_1      | ts=2023-07-24T02:18:53Z caller=main.go:70 transport=HTTP port=80
payment_1      | ts=2023-07-24T02:19:53Z caller=logging.go:29 method=Authorise result=false took=29.659µs
payment_1      | ts=2023-07-24T02:19:53Z caller=logging.go:29 method=Authorise result=false took=7.811µs
payment_1      | ts=2023-07-24T02:19:54Z caller=logging.go:29 method=Authorise result=true took=1.19µs
payment_1      | ts=2023-07-24T02:19:54Z caller=logging.go:29 method=Authorise result=true took=1.102µs
payment_1      | ts=2023-07-24T02:19:55Z caller=logging.go:29 method=Authorise result=true took=887ns
payment_1      | ts=2023-07-24T02:19:55Z caller=logging.go:29 method=Authorise result=true took=907ns
payment_1      | ts=2023-07-24T02:19:55Z caller=logging.go:29 method=Authorise result=true took=1.04µs
payment_1      | ts=2023-07-24T02:19:55Z caller=logging.go:29 method=Authorise result=true took=1.086µs
payment_1      | ts=2023-07-24T02:19:56Z caller=logging.go:29 method=Authorise result=true took=1.257µs
payment_1      | ts=2023-07-24T02:19:56Z caller=logging.go:29 method=Authorise result=true took=961ns
payment_1      | ts=2023-07-24T02:19:56Z caller=logging.go:29 method=Authorise result=true took=929ns
payment_1      | ts=2023-07-24T02:19:56Z caller=logging.go:29 method=Authorise result=true took=1.188µs
payment_1      | ts=2023-07-24T02:19:56Z caller=logging.go:29 method=Authorise result=true took=1.182µs
payment_1      | ts=2023-07-24T02:19:56Z caller=logging.go:29 method=Authorise result=true took=939ns
payment_1      | ts=2023-07-24T02:19:56Z caller=logging.go:29 method=Authorise result=true took=1.062µs
payment_1      | ts=2023-07-24T02:19:56Z caller=logging.go:29 method=Authorise result=false took=8.389µs
payment_1      | ts=2023-07-24T02:19:57Z caller=logging.go:29 method=Authorise result=false took=6.535µs
payment_1      | ts=2023-07-24T02:19:57Z caller=logging.go:29 method=Authorise result=true took=1.082µs
payment_1      | ts=2023-07-24T02:19:57Z caller=logging.go:29 method=Authorise result=false took=7.227µs
payment_1      | ts=2023-07-24T02:19:57Z caller=logging.go:29 method=Authorise result=true took=949ns
payment_1      | ts=2023-07-24T02:19:57Z caller=logging.go:29 method=Authorise result=true took=1.232µs
payment_1      | ts=2023-07-24T02:19:57Z caller=logging.go:29 method=Authorise result=true took=1.035µs

```

Hình 3.10 Log chạy Payment service container

```

root@chau-gti1ab-ce:/microservices-demo/deploy/docker-compose# docker-compose logs user
Attaching to docker-compose_user_1
user_1      | ts=2023-07-24T02:18:55Z caller=main.go:145 transport=HTTP port=80
user_1      | ts=2023-07-24T02:19:52Z caller=middlewares.go:34 method=Login took=3.481605ms
user_1      | ts=2023-07-24T02:19:52Z caller=middlewares.go:34 method=Login took=1.030379ms
user_1      | ts=2023-07-24T02:19:53Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=680.868µs
user_1      | ts=2023-07-24T02:19:53Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=311.777µs
user_1      | ts=2023-07-24T02:19:53Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=450.422µs
user_1      | ts=2023-07-24T02:19:53Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=374.768µs
user_1      | ts=2023-07-24T02:19:53Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=2.928517ms
user_1      | ts=2023-07-24T02:19:53Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=5.173193ms
user_1      | ts=2023-07-24T02:19:53Z caller=middlewares.go:104 method=GetAddresses id=57a98d98e4b00679b4a830b2 result=1 took=556.329µs
user_1      | ts=2023-07-24T02:19:53Z caller=middlewares.go:104 method=GetAddresses id=57a98d98e4b00679b4a830b2 result=1 took=367.658µs
user_1      | ts=2023-07-24T02:19:53Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=263.057µs
user_1      | ts=2023-07-24T02:19:53Z caller=middlewares.go:134 method=GetCards id=57a98d98e4b00679b4a830b1 result=1 took=365.445µs
user_1      | ts=2023-07-24T02:19:53Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=397.171µs
user_1      | ts=2023-07-24T02:19:53Z caller=middlewares.go:134 method=GetCards id=57a98d98e4b00679b4a830b1 result=1 took=266.306µs
user_1      | ts=2023-07-24T02:19:53Z caller=middlewares.go:34 method=Login took=954.817µs
user_1      | ts=2023-07-24T02:19:53Z caller=middlewares.go:34 method=Login took=1.074766ms
user_1      | ts=2023-07-24T02:19:53Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=424.626µs
user_1      | ts=2023-07-24T02:19:53Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=610.884µs
user_1      | ts=2023-07-24T02:19:53Z caller=middlewares.go:104 method=GetAddresses id=57a98d98e4b00679b4a830b2 result=1 took=409.177µs
user_1      | ts=2023-07-24T02:19:54Z caller=middlewares.go:134 method=GetCards id=57a98d98e4b00679b4a830b1 result=1 took=328.546µs
user_1      | ts=2023-07-24T02:19:54Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=360.987µs
user_1      | ts=2023-07-24T02:19:54Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=284.443µs
user_1      | ts=2023-07-24T02:19:54Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=333.32µs
user_1      | ts=2023-07-24T02:19:54Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=194.027µs
user_1      | ts=2023-07-24T02:19:54Z caller=middlewares.go:104 method=GetAddresses id=57a98d98e4b00679b4a830b2 result=1 took=339.788µs
user_1      | ts=2023-07-24T02:19:54Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=326.576µs
user_1      | ts=2023-07-24T02:19:54Z caller=middlewares.go:134 method=GetCards id=57a98d98e4b00679b4a830b1 result=1 took=220.787µs
user_1      | ts=2023-07-24T02:19:54Z caller=middlewares.go:34 method=Login took=1.264697ms
user_1      | ts=2023-07-24T02:19:54Z caller=middlewares.go:34 method=Login took=8.499492ms
user_1      | ts=2023-07-24T02:19:55Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=426.806µs
user_1      | ts=2023-07-24T02:19:55Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=320.64µs
user_1      | ts=2023-07-24T02:19:55Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=267.621µs
user_1      | ts=2023-07-24T02:19:55Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=395.434µs
user_1      | ts=2023-07-24T02:19:55Z caller=middlewares.go:75 method=GetUsers id=57a98d98e4b00679b4a830b2 result=1 took=236.078µs
user_1      | ts=2023-07-24T02:19:55Z caller=middlewares.go:104 method=GetAddresses id=57a98d98e4b00679b4a830b2 result=1 took=205.315µs

```

Hình 3.11 Log chạy User service container

- Carts service:

```

root@chau-gti1ab-ce:/microservices-demo/deploy/docker-compose# docker-compose logs carts
Attaching to docker-compose_carts_1
carts_1      | 2023-07-24 02:18:57.934 INFO [bootstrap,,] 7 --- [          main] s.c.a.AnnotationConfigApplicationContext : Refreshing org.springframework.context.annotation.AnnotationConfigApplicationContext@51521cc1: startup date [Mon Jul 24 02:18:57 GMT 2023]; root of context hierarchy
carts_1      | 2023-07-24 02:18:59.861 INFO [bootstrap,,] 7 --- [          main] trationDelegate$BeanPostProcessorChecker : Bean 'configurationPropertiesRebinderAutoConfiguration' of type [class org.springframework.cloud.autoconfigure.ConfigurationPropertiesRebinderAutoConfiguration$$EnhancerBySpringCGLIB$$894f39] is not eligible for getting processed by all BeanPostProcessors (for example: not eligible for auto-proxying)
carts_1      | 2023-07-24 02:19:00.908 INFO [carts,,,] 7 --- [          main] works.weave.socks.cart.CartApplication : No active profile set, falling back to default profile: default
carts_1      | 2023-07-24 02:19:01.038 INFO [carts,,,] 7 --- [          main] ationConfigEmbeddedWebApplicationContext : Refreshing org.springframework.boot.context.embedded.AnnotationConfigEmbeddedWebApplicationContext@649d209a: startup date [Mon Jul 24 02:19:01 GMT 2023]; parent: org.springframework.context.annotation.AnnotationConfigApplicationContext@51521cc1
carts_1      | 2023-07-24 02:19:01.573 INFO [carts,,,] 7 --- [          main] o.s.b.f.s.DefaultListableBeanFactory : Overriding bean definition for bean 'managementServletContext' with a different definition: replacing [Root bean: class [null]; scope: abstract=false; lazyInit=false; autowireMode=3; dependencyCheck=0; autowireCandidate=true; primary=false; factoryBeanName=org.springframework.boot.actuate.autoconfigure.EndpointWebMvcHypermediaManagementContextConfiguration; factoryMethodName=managementServletContext; initMethodName=null; destroyMethodName=(inferred); defined in class path resource [org/springframework/boot/actuate/autoconfigure/EndpointWebMvcHypermediaManagementContextConfiguration.class]] with [Root bean: class [null]; scope: abstract=false; lazyInit=false; autowireMode=3; dependencyCheck=0; autowireCandidate=true; primary=false; factoryBeanName=org.springframework.boot.actuate.autoconfigure.EndpointWebMvcAutoConfiguration; factoryMethodName=managementServletContext; initMethodName=null; destroyMethodName=(inferred); defined in class path resource [org/springframework/boot/actuate/autoconfigure/EndpointWebMvcAutoConfiguration.class]]
carts_1      | 2023-07-24 02:19:06.265 WARN [carts,,,] 7 --- [          main] o.s.c.a.ConfigurationClassPostProcessor : Cannot enhance @Configuration bean definition 'refreshScope' since its singleton instance has been created too early. The typical cause is a non-static @Bean method with a BeanDefinitionRegistryPostProcessor return type: Consider declaring such methods as 'static'.
carts_1      | 2023-07-24 02:19:06.876 INFO [carts,,,] 7 --- [          main] o.s.cloud.context.scope.GenericScope : BeanFactory id=063d23b5-fa4f-330e-8afb-cd76f0b608e
carts_1      | 2023-07-24 02:19:07.516 INFO [carts,,,] 7 --- [          main] trationDelegate$BeanPostProcessorChecker : Bean 'org.springframework.cloud sleuth.instrument.async.AsyncDefaultAutoConfiguration$DefaultAsyncConfigurerSupport' of type [class org.springframework.cloud.sleuth.instrument.async.AsyncDefaultAutoConfiguration$DefaultAsyncConfigurerSupport] is not eligible for getting processed by all BeanPostProcessors (for example: not eligible for auto-proxying)
carts_1      | 2023-07-24 02:19:08.669 INFO [carts,,,] 7 --- [          main] trationDelegate$BeanPostProcessorChecker : Bean 'org.springframework.cloud.autoconfigure.ConfigurationPropertiesRebinderAutoConfiguration' of type [class org.springframework.cloud.autoconfigure.ConfigurationPropertiesRebinderAutoConfiguration$$EnhancerBySpringCGLIB$$894f39] is not eligible for getting processed by all BeanPostProcessors (for example: not eligible for auto-proxying)

```

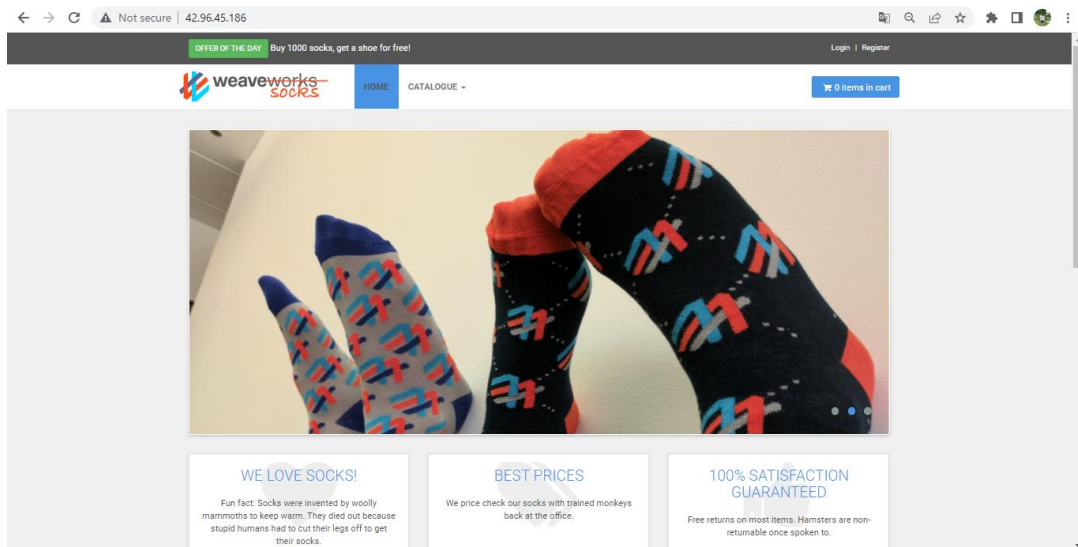
Hình 3.12 Log chạy Carts service container

- Catalogue service:

Kiểm tra các tính năng của ứng dụng

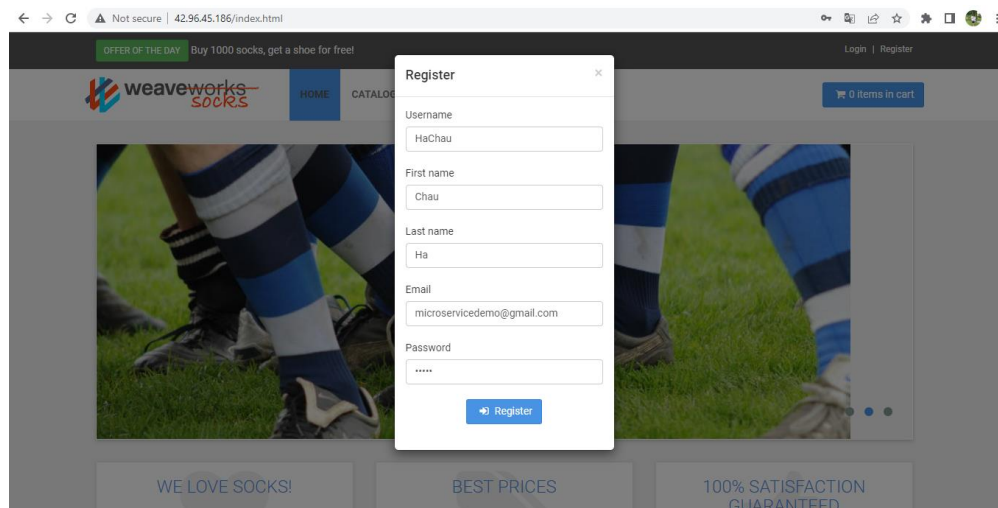
Truy cập giao diện UI của ứng dụng trên [http:// 42.96.45.186](http://42.96.45.186) , 42.96.45.186 là địa chỉ IP của máy chủ chạy docker.

- Giao diện chính của ứng dụng:



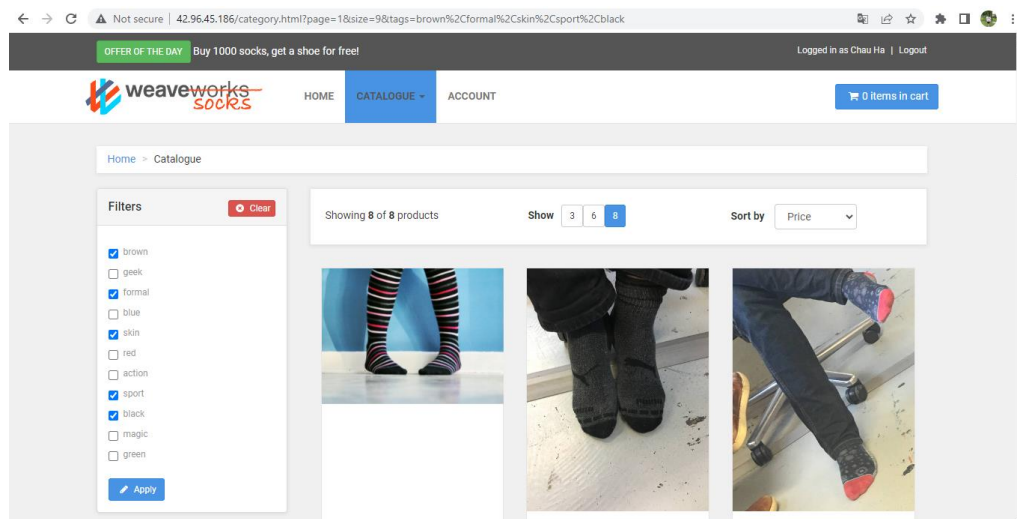
Hình 3.16 Giao diện chính của ứng dụng

- Đăng ký tài khoản:



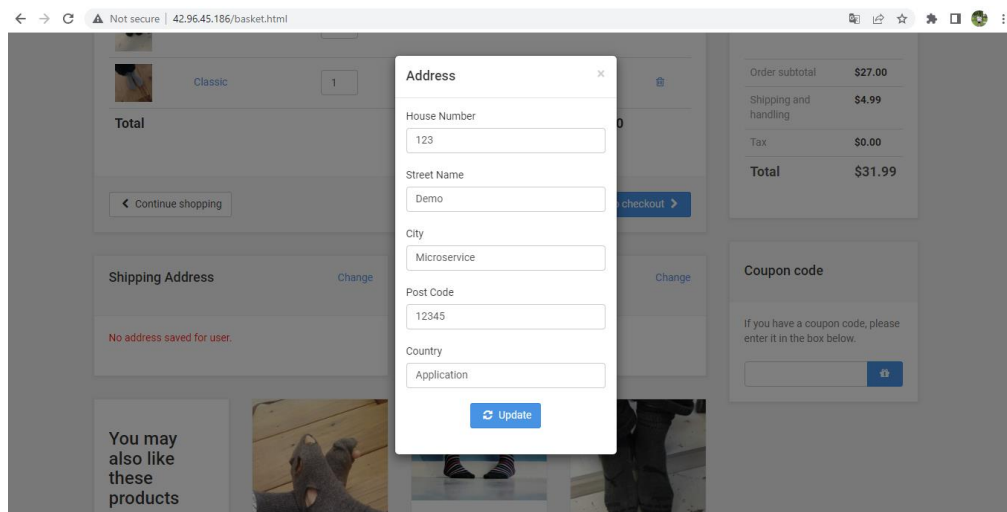
Hình 3.17 Đăng ký tài khoản mua hàng

- Xem danh mục sản phẩm:



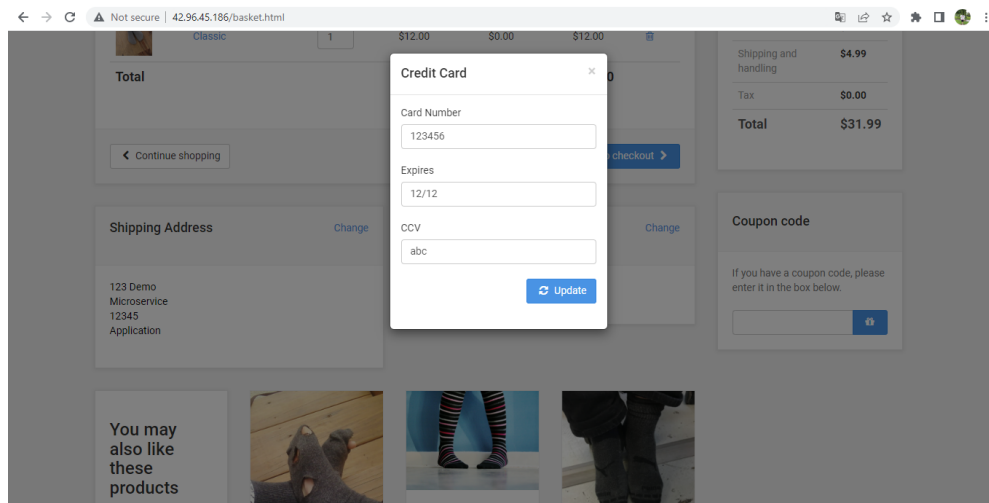
Hình 3.18 Giao diện danh mục sản phẩm

- Add địa chỉ:



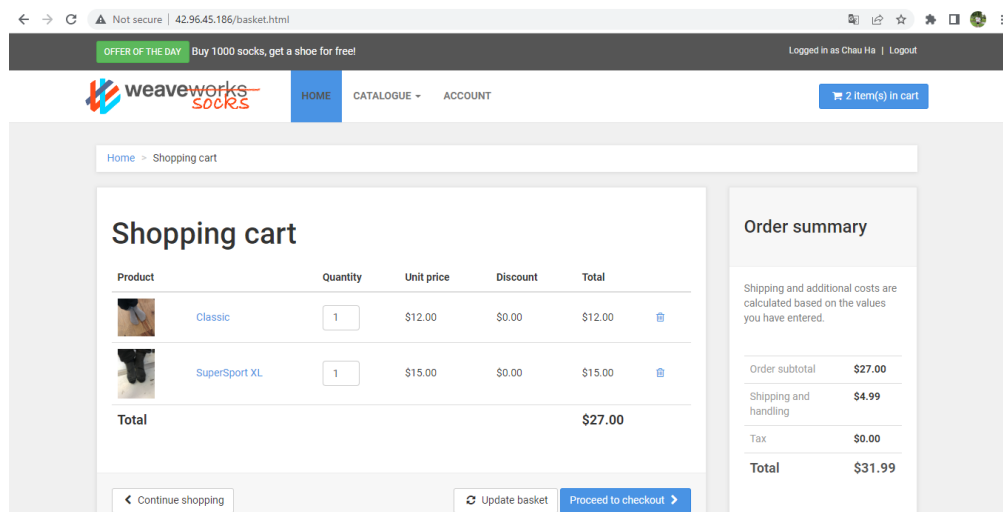
Hình 3.19 Giao diện thêm địa chỉ giao hàng

- Add thẻ:



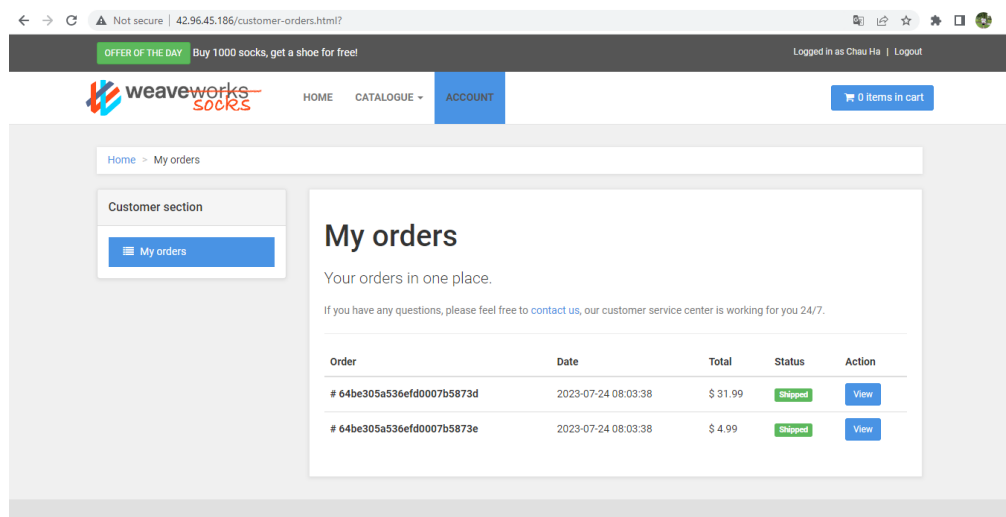
Hình 3.20 Giao diện thêm thông tin thẻ thanh toán

- Đặt hàng:



Hình 3.21 Giao diện order sản phẩm

- Kiểm tra order:



Hình 3.22 Giao diện danh sách các đơn hàng đã đặt

3.3 Triển khai ứng dụng trên Kubernetes

Để triển khai ứng dụng trên Kubernetes Cluster có 2 phương án đó là sử dụng tệp YAML Manifest hoặc Helm. Tệp YAML manifest mô tả các tài nguyên cần thiết cho ứng dụng bao gồm các container, volumes, services, deployments và các thành phần khác. Tệp manifest này sẽ được sử dụng để triển khai ứng dụng trên Kubernetes bằng lệnh `kubectl apply` hoặc `kubectl create`. Helm là một công cụ quản lý gói cho Kubernetes, được sử dụng để triển khai và quản lý ứng dụng trên Kubernetes một cách dễ dàng. Helm cho phép đóng gói ứng dụng vào các bản phân phối có thể tái sử dụng, được gọi là "chart". Trong phần demo này, ứng dụng sẽ được triển khai sử dụng tệp YAML manifest.

3.3.1 Thiết lập cụm Kubernetes cơ bản với minikube

Thiết lập cụm Kubernetes với minikube trên Window 10

Chạy PowerShell với quyền administrator:

```
# Tải tệp cài đặt .exe

New-Item -Path 'c:\' -Name 'minikube' -ItemType Directory -Force

Invoke-WebRequest -OutFile 'c:\minikube\minikube.exe' -Uri
'https://github.com/kubernetes/minikube/releases/latest/download/minik
ube-windows-amd64.exe' -UseBasicParsing

# Thêm minikube.exe vào đường dẫn hiện tại và tiến hành cài đặt:
```

```

$oldPath = [Environment]::GetEnvironmentVariable('Path',
[EnvironmentVariableTarget]::Machine)

if ($oldPath.Split(';') -notincontains 'C:\minikube'){

    [Environment]::SetEnvironmentVariable('Path', $('{0};C:\minikube' -f
$oldPath), [EnvironmentVariableTarget]::Machine)

}

# Kiểm tra cài đặt đã thành công hay chưa:

minikube

# Kết quả được thể hiện ở hình 3.23

# Bật hypervisor, trong phần demo này sử dụng docker

# Tạo cụm Kubernetes

minikube start --memory 6144 --cpus 4

# Có thể thêm các tùy chọn cấu hình khác như cấu hình RAM, CPU,
hypervisor ..., nếu không chỉ cần chạy minikube start, cluster sẽ được
thiết lập theo các cấu hình mặc định và có thể tùy chỉnh sau khi tạo
thành công cụm.

#Kết quả ở Hình 3.24

# Sau khi tạo thành công cluster có thể dùng kubectl để giao tiếp với
cluster, kết quả một số câu lệnh trong Hình 3.25

```

```

C:\Users\lamha>minikube
minikube provisions and manages local Kubernetes clusters optimized for development workflows.

Basic Commands:
  start           Starts a local Kubernetes cluster
  status          Gets the status of a local Kubernetes cluster
  stop           Stops a running local Kubernetes cluster
  delete          Deletes a local Kubernetes cluster
  dashboard      Access the Kubernetes dashboard running within the minikube cluster
  pause          pause Kubernetes
  unpause        unpause Kubernetes

Images Commands:
  docker-env     Provides instructions to point your terminal's docker-cli to the Docker Engine inside minikube.
  (Useful for building docker images directly inside minikube)
  podman-env     Configure environment to use minikube's Podman service
  cache          Manage cache for images
  image          Manage images

Configuration and Management Commands:
  addons         Enable or disable a minikube addon
  config         Modify persistent configuration values
  profile        Get or list the current profiles (clusters)
  update-context Update kubeconfig in case of an IP or port change

Networking and Connectivity Commands:
  service        Returns a URL to connect to a service
  tunnel         Connect to LoadBalancer services

Advanced Commands:
  mount          Mounts the specified directory into minikube
  ssh            Log into the minikube environment (for debugging)
  kubectl        Run a kubectl binary matching the cluster version
  node           Add, remove, or list additional nodes

```

Hình 3.23 Cài đặt thành công minikube

```

C:\Users\lamha>minikube start --memory 6144 --cpus 4
* minikube v1.31.1 on Microsoft Windows 10 Pro 10.0.19044.3086 Build 19044.3086
* Automatically selected the docker driver
* Using Docker Desktop driver with root privileges
* Starting control plane node minikube in cluster minikube
* Pulling base image ...
* Creating docker container (CPUs=4, Memory=6144MB) ...
* Preparing Kubernetes v1.27.3 on Docker 24.0.4 ...
  - Generating certificates and keys ...
  - Booting up control plane ...
  - Configuring RBAC rules ...
* Configuring bridge CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
! Executing "docker container inspect minikube --format={{.State.Status}}" took an unusually long time: 2.2284503s
* Restarting the docker service may improve performance.
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner

! C:\Program Files\Docker\Docker\resources\bin\kubectl.exe is version 1.25.9, which may have incompatibilities with
Kubernetes 1.27.3.
  - Want kubectl v1.27.3? Try 'minikube kubectl -- get pods -A'
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default

C:\Users\lamha>

```

Hình 3.24 Kết quả lệnh thiết lập cluster

```

C:\Users\lamha>kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready     control-plane   84s   v1.27.3

C:\Users\lamha>kubectl get namespace
NAME                STATUS    AGE
default             Active    94s
kube-node-lease     Active    94s
kube-public         Active    94s
kube-system         Active    94s

C:\Users\lamha>kubectl get pod -A
NAMESPACE   NAME                                     READY   STATUS    RESTARTS   AGE
kube-system  coredns-5d78c9869d-2fjkk              1/1     Running   0           2m10s
kube-system  etcd-minikube                         1/1     Running   0           2m22s
kube-system  kube-apiserver-minikube               1/1     Running   0           2m22s
kube-system  kube-controller-manager-minikube      1/1     Running   0           2m22s
kube-system  kube-proxy-d7qss                     1/1     Running   0           2m11s
kube-system  kube-scheduler-minikube               1/1     Running   0           2m22s
kube-system  storage-provisioner                   1/1     Running   1 (119s ago) 2m14s

C:\Users\lamha>

```

Hình 3.25 Nodes, deployment và pod hệ thống

3.3.2 Triển khai ứng dụng trên Kubernetes

Chuyển đến thư mục chứa tệp YAML manifest

```
C:\Users\lamha\microservices-demo-master\deploy\kubernetes>dir
Volume in drive C is Windows 10
Volume Serial Number is 9604-2FE2

Directory of C:\Users\lamha\microservices-demo-master\deploy\kubernetes

24/07/2023  11:56 SA    <DIR>          .
24/07/2023  11:56 SA    <DIR>          ..
24/07/2023  11:48 SA             17.814 manifests-sock-shop.yaml
               1 File(s)             17.814 bytes
               2 Dir(s)  52.196.675.584 bytes free

C:\Users\lamha\microservices-demo-master\deploy\kubernetes>
```

Hình 3.26 Thư mục chứa tệp cấu hình manifest

Chạy lệnh:

```
kubectl create -f manifests-sock-shop.yaml
```

Kết quả trên Hình 3.27

```
C:\Users\lamha\microservices-demo-master\deploy\kubernetes>kubectl create -f manifests-sock-shop.yaml
namespace/sock-shop created
Warning: spec.template.spec.nodeSelector[beta.kubernetes.io/os]: deprecated since v1.14; use "kubernetes.io/os" instead
deployment.apps/carts created
service/carts created
deployment.apps/carts-db created
service/carts-db created
deployment.apps/catalogue created
service/catalogue created
deployment.apps/catalogue-db created
service/catalogue-db created
deployment.apps/front-end created
service/front-end created
deployment.apps/orders created
service/orders created
deployment.apps/orders-db created
service/orders-db created
deployment.apps/payment created
service/payment created
deployment.apps/queue-master created
service/queue-master created
deployment.apps/rabbitmq created
service/rabbitmq created
deployment.apps/session-db created
service/session-db created
deployment.apps/shipping created
service/shipping created
deployment.apps/user created
service/user created
deployment.apps/user-db created
service/user-db created
```

Hình 3.27 Kết quả triển khai khai ứng dụng sử dụng tệp manifest

Kiểm tra pods:

```
kubectl get pod -n sock-shop # Hình 3.28
```

```
C:\Users\lamha\microservices-demo-master\deploy\kubernetes>kubectl get pod -n sock-shop
```

NAME	READY	STATUS	RESTARTS	AGE
carts-5bb979cb6d-x4mbn	1/1	Running	0	11m
carts-db-6d88679f9d-9td7v	1/1	Running	0	11m
catalogue-7c89c4b8b7-xctgb	1/1	Running	0	11m
catalogue-db-6d76c95d76-vztg4	1/1	Running	0	11m
front-end-575ff98689-m6pcm	1/1	Running	0	11m
orders-d5f745cc6-4s9vj	1/1	Running	0	11m
orders-db-688cc755dd-bwt9l	1/1	Running	0	11m
payment-66d9c6c5c8-2xp1l	1/1	Running	0	11m
queue-master-78b6f85bb7-82qqr	1/1	Running	0	11m
rabbitmq-55c946cb56-99kbg	2/2	Running	0	11m
session-db-9dc55b5b-28dnm	1/1	Running	0	11m
shipping-78db6c6958-snggh	1/1	Running	0	11m
user-5c8d59bcd4-wnmtp	1/1	Running	0	11m
user-db-758477f574-zs2tj	1/1	Running	0	11m

Hình 3.28 Các pod của ứng dụng

Kiểm tra deployment:

kubectl get deployment -n sock-shop # Hình 3.29

```
C:\Users\lamha\microservices-demo-master\deploy\kubernetes>kubectl get deployment -n sock-shop
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
carts	1/1	1	1	22m
carts-db	1/1	1	1	22m
catalogue	1/1	1	1	22m
catalogue-db	1/1	1	1	22m
front-end	1/1	1	1	22m
orders	1/1	1	1	22m
orders-db	1/1	1	1	22m
payment	1/1	1	1	22m
queue-master	1/1	1	1	22m
rabbitmq	1/1	1	1	22m

Hình 3.29 Các deployment của ứng dụng

Kiểm tra service:

kubectl get svc -n sock-shop # Hình 3.30

```
C:\Users\lamha\microservices-demo-master\deploy\kubernetes>kubectl get svc -n sock-shop
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
carts	ClusterIP	10.101.231.231	<none>	80/TCP	23m
carts-db	ClusterIP	10.105.19.175	<none>	27017/TCP	23m
catalogue	ClusterIP	10.97.208.47	<none>	80/TCP	23m
catalogue-db	ClusterIP	10.111.1.168	<none>	3306/TCP	23m
front-end	NodePort	10.99.64.175	<none>	80:30001/TCP	23m
orders	ClusterIP	10.97.158.160	<none>	80/TCP	23m
orders-db	ClusterIP	10.98.159.108	<none>	27017/TCP	23m
payment	ClusterIP	10.108.39.65	<none>	80/TCP	23m
queue-master	ClusterIP	10.103.53.78	<none>	80/TCP	23m
rabbitmq	ClusterIP	10.111.242.240	<none>	5672/TCP,9090/TCP	23m
session-db	ClusterIP	10.104.36.66	<none>	6379/TCP	23m
shipping	ClusterIP	10.99.45.86	<none>	80/TCP	23m
user	ClusterIP	10.105.210.121	<none>	80/TCP	23m
user-db	ClusterIP	10.102.12.13	<none>	27017/TCP	23m

Hình 3.30 Các service chạy trên cluster

Thêm dashboard. Kubernetes Dashboard là một giao diện người dùng dựa trên web cho việc quản lý các cụm Kubernetes. Nó cho phép người dùng

triển khai và quản lý các ứng dụng, cũng như giám sát và khắc phục sự cố cho cụm Kubernetes của họ. Bảng điều khiển cung cấp một biểu đồ đồ họa về các tài nguyên đang chạy trong cụm, bao gồm các pod, dịch vụ, triển khai và cấu hình.

minikube dashboard

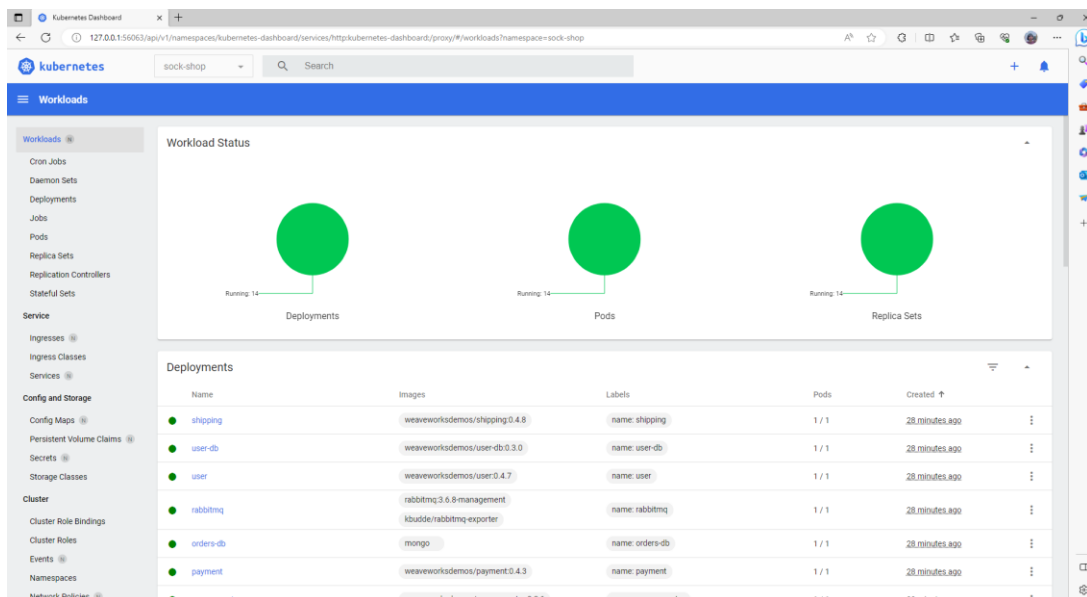
Kết quả ở Hình 3.31

```
C:\Users\lamha\microservices-demo-master\deploy\kubernetes>minikube dashboard
* Enabling dashboard ...
- Using image docker.io/kubernetesui/dashboard:v2.7.0
- Using image docker.io/kubernetesui/metrics-scraper:v1.0.8
* Some dashboard features require the metrics-server addon. To enable all features please run:

    minikube addons enable metrics-server

* Verifying dashboard health ...
* Launching proxy ...
* Verifying proxy health ...
* Opening http://127.0.0.1:56063/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashbo
rd:/proxy/ in your default browser...
```

Hình 3.31 Kết quả chạy lệnh thêm Kubernetes Dashboard



Hình 3.32 Giao diện Kubernetes Dashboard

Truy cập vào các service của ứng dụng chạy thông qua NodePort. Sử dụng lệnh `minikube service <service-name>` để mở trình duyệt web và truy cập ứng dụng thông qua dịch vụ NodePort.

Truy cập tất cả service

`minikube service --all -n sock-shop`

Truy cập dashboard

```
minikube service kubernetes-dashboard -n kubernetes-dashboard
```

```
# Truy cập giao diện front-end
```

```
minikube service front-end -n sock-shop
```

NAMESPACE	NAME	TARGET PORT	URL
sock-shop	carts		http://127.0.0.1:57392
sock-shop	carts-db		http://127.0.0.1:57394
sock-shop	catalogue		http://127.0.0.1:57396
sock-shop	catalogue-db		http://127.0.0.1:57398
sock-shop	front-end		http://127.0.0.1:57400
sock-shop	orders		http://127.0.0.1:57402
sock-shop	orders-db		http://127.0.0.1:57404
sock-shop	payment		http://127.0.0.1:57406
sock-shop	queue-master		http://127.0.0.1:57408
sock-shop	rabbitmq		http://127.0.0.1:57410
sock-shop	session-db		http://127.0.0.1:57411
sock-shop	shipping		http://127.0.0.1:57413
sock-shop	user		http://127.0.0.1:57415
sock-shop	user-db		http://127.0.0.1:57417

Hình 3.33 URL của các service

```
C:\Users\lamha\microservices-demo-master\deploy\kubernetes>minikube service kubernetes-dashboard -n kubernetes-dashboard
```

NAMESPACE	NAME	TARGET PORT	URL
kubernetes-dashboard	kubernetes-dashboard		No node port

* service kubernetes-dashboard/kubernetes-dashboard has no node port
* Starting tunnel for service kubernetes-dashboard.

NAMESPACE	NAME	TARGET PORT	URL
kubernetes-dashboard	kubernetes-dashboard		http://127.0.0.1:58001

* Opening service kubernetes-dashboard/kubernetes-dashboard in default browser...
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.

Hình 3.34 URL của Dashboard service

```
C:\Users\lamha>minikube service front-end -n sock-shop
```

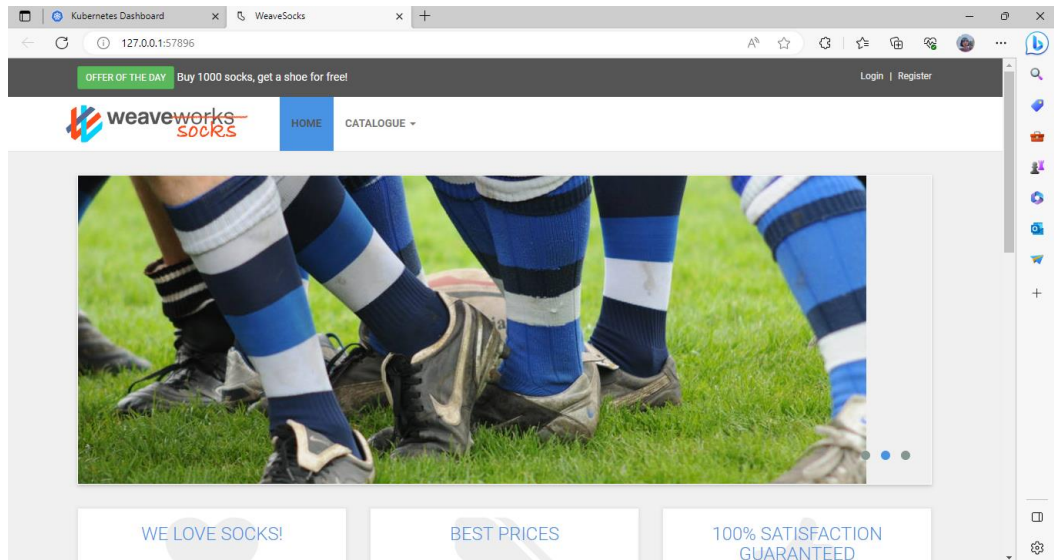
NAMESPACE	NAME	TARGET PORT	URL
sock-shop	front-end	80	http://192.168.49.2:30001

* Starting tunnel for service front-end.

NAMESPACE	NAME	TARGET PORT	URL
sock-shop	front-end		http://127.0.0.1:57896

* Opening service sock-shop/front-end in default browser...
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.

Hình 3.35 URL Front-end service



Hình 3.36 Giao diện chính của ứng dụng chạy trên cụm Kubernetes

3.4 Đánh giá

Sau khi thực hiện triển khai ứng dụng microservice sử dụng hai giải pháp công nghệ là Docker và Kubernetes. Dưới đây là một số đánh giá về việc sử dụng Docker và Kubernetes trong triển khai kiến trúc microservices.

Đầu tiên, giải pháp triển khai này có những ưu điểm như sau:

- Tăng tính di động và độ linh hoạt: Docker cho phép đóng gói ứng dụng và các thành phần phụ thuộc của nó vào một container độc lập, giúp việc triển khai ứng dụng trở nên dễ dàng và nhanh chóng hơn. Kubernetes cung cấp một cách để quản lý và triển khai các container này trên nhiều máy chủ, giúp tăng tính di động và độ linh hoạt của ứng dụng.
- Tăng tính khả dụng và độ tin cậy: Việc triển khai kiến trúc microservices trên Kubernetes giúp tăng khả năng sẵn sàng và độ tin cậy của ứng dụng. Nếu một container bị lỗi, Kubernetes có thể tự động khởi động lại container đó hoặc triển khai một phiên bản mới của nó.
- Tiết kiệm tài nguyên: Docker cho phép nhiều container chạy trên cùng một máy chủ, giúp tiết kiệm tài nguyên máy chủ và giảm chi phí.
- Dễ dàng quản lý phiên bản: Việc triển khai kiến trúc microservices trên Kubernetes giúp dễ dàng quản lý các phiên bản khác nhau của các dịch vụ. Kubernetes có thể tự động triển khai và quản lý các phiên bản mới của các dịch vụ và giữ cho ứng dụng luôn chạy trên phiên bản mới nhất.

Tuy nhiên, vẫn còn một số nhược điểm cần được xem xét đó là:

- Việc học cách sử dụng Docker và Kubernetes để triển khai kiến trúc microservices có thể đòi hỏi một số thời gian và nỗ lực, đặc biệt là đối với những người mới bắt đầu.
- Docker và Kubernetes là hai công nghệ phức tạp và có nhiều tính năng, điều này có thể khiến việc triển khai và quản lý kiến trúc microservices trở nên phức tạp hơn so với các phương pháp truyền thống.

Tổng kết lại, chương này đã thực hiện triển khai ứng dụng microservice áp dụng công nghệ dựa trên nền tảng container. Giúp thử nghiệm mô hình kiến trúc, nắm được cách các service giao tiếp với nhau, hiểu rõ hơn về quy trình container hóa một ứng dụng trong thực tế và các mà các công nghệ như Docker và Kubernetes giúp cho việc triển khai và vận hành ứng dụng trở nên dễ dàng và linh hoạt hơn.

KẾT LUẬN

Kết luận chung

Trong đồ án này, em đã thực hiện tìm hiểu về kiến trúc microservice và triển khai ứng dụng microservice sử dụng các giải pháp công nghệ đang phát triển hiện nay như Docker và Kubernetes. Những kiến thức và kỹ năng mà em đã tiếp thu được như:

- Kiến thức về kiến trúc hệ thống và kiến trúc phần mềm
- Các giải pháp công nghệ đang phát triển hiện nay
- Kiến thức về các phần mềm mã nguồn mở, cách cài đặt, tùy chỉnh cấu hình và deploy chúng.
- Có thêm kiến thức và trải nghiệm với các phần mềm như là Docker, Kubernetes, ...
- Cách tổ chức làm việc theo module
- Kỹ năng tự học
- Cải thiện thêm về ngoại ngữ
- Kỹ năng trình bày thông tin một cách khoa học

Tuy nhiên, em cảm thấy bản thân mình vẫn còn đó nhiều hạn chế và còn cần cải thiện nhiều hơn.

Hướng phát triển

Đồ án này tuy đã giải quyết được việc đi vào tìm hiểu các kiến trúc hệ thống và tập trung và kiến trúc microservice. Triển khai được một ứng dụng microservice cơ bản dựa trên công nghệ container hóa trên nền tảng Container Orchestration. Tuy nhiên, việc tìm hiểu vẫn chưa thực sự đi sâu vào kiến trúc và các đặc điểm của nó, ứng dụng demo còn tương đối đơn giản, chưa triển khai được ứng dụng trên một cụm Kubernetes quy mô lớn. Chưa có các bài test tính năng mở rộng của hệ thống khi số lượng yêu cầu tăng thêm. Chưa cài đặt công cụ giám sát hoạt động và thu thập log của hệ thống. Sau đây là một số hướng phát triển của đồ án:

- Nghiên cứu thêm các kiến trúc và giải pháp mới
- Thêm vào các bài test khả năng scale up của hệ thống khi số lượng request tăng
- Thiết lập hệ thống giám sát hoạt động của phần mềm

- Nghiên cứu các phương án di chuyển ứng dụng lên cloud

TÀI LIỆU THAM KHẢO

- [1] <https://martinfowler.com/articles/microservices.html>
- [2] Mumshad Mannambeth, *Kubernetes-for-Beginners.pdf*.
- [3] <https://microservices.io/patterns/microservices.html>
- [4] <https://medium.com/@SoftwareDevelopmentCommunity/what-is-service-oriented-architecture-fa894d11a7ec>
- [5] <https://www.baeldung.com/cs/service-discovery-microservices>
- [6] <https://blog.bitsrc.io/circuit-breaker-pattern-in-microservices-26bf6e5b21ff>
- [7] <https://microservices.io/patterns/apigateway.html>
- [8] <https://www.baeldung.com/cs/saga-pattern-microservices>
- [9] <https://medium.com/design-microservices-architecture-with-patterns/cqrs-design-pattern-in-microservices-architectures-5d41e359768c>
- [10] What is Container Orchestration? | Benefits and Tools (k21academy.com)
- [11] <https://www.devopsschool.com/blog/what-is-minikube/>
- [12] <https://kubernetes.io>
- [13] Suren Machiraju, Rajneesh Sidhu, Tarun Arora, *Containerized Docker Application Lifecycle with Microsoft Platform and Tools*
- [14] Cesar de la Torre, Bill Wagner, Mike Rousos, *.NET Microservices: Architecture for Containerized .NET Applications*
- [15] <https://microservices-demo.github.io/api/index.html>
- [16] <https://github.com/microservices-demo>
- [17] <https://www.hadii.ca/insights/microservice-communication>
- [18] <https://docs.docker.com/get-started/overview/>
- [19] <https://www.linkedin.com/pulse/key-reasons-using-api-gateway-microservices-architecture-chad-ng>

PHỤ LỤC